

PATRICK ANDREI

CLAUDE

CODE ✨

ARCHITECT



Master SKILL.md, Subagents, and Hooks That
Transform Claude Code Into a Fully Engineered,
Production Agentic System

CLAUDE CODE

ARCHITECT

Copyright © 2026 ANDREI Media. All rights reserved.

No part of this book may be reproduced or transmitted in any form
or by any means, electronic or mechanical, including

photocopying, recording, or by any information storage and retrieval system, without permission in writing from the publisher.

The information in this book is provided for educational and informational purposes only. Claude Code, Claude, and Anthropic are trademarks of Anthropic, PBC. This book is not affiliated with, endorsed by, or sponsored by Anthropic, PBC.

*For the developers who treat their tools the way
craftsmen treat theirs:
as something worth shaping, slowly, until it fits.*

CONTENTS

Introduction.....	6
--------------------------	----------

PART I *Foundations: The Three Pillars of Claude Code Architecture*

Chapter 1 Why Claude Code Alone Isn't Enough.....	11
Chapter 2 The Anatomy of Claude Code.....	21
Chapter 3 Skills, Subagents, and Hooks: The Three-Pillar Framework.....	31

PART II *Skills: Claude's Extensible Knowledge System*

Chapter 4 Skills Fundamentals: The SKILL.md Anatomy.....	43
Chapter 5 Writing Your First Skill: A Hands-On Build.....	55
Chapter 6 Skill Description Engineering: The Art of Triggering.....	67
Chapter 7 Multi-File Skills and Bundled Resources.....	78
Chapter 8 Skill Composition and Collision: When Skills Work Together.....	90
Chapter 9 The Personal Skill Library: Your First Ten Skills.....	100

PART III *Subagents: Claude's Parallel Specialist System*

Chapter 10 Subagents Fundamentals: Why Single-Threaded Claude Isn't Enough.....	115
Chapter 11 Designing Effective Subagents: Role, Tools, and Boundaries.....	126
Chapter 12 Parallel Subagents and the Orchestrator Pattern.....	138
Chapter 13 The Subagent Library: Eight Specialists Every Developer Needs.....	149
Chapter 14 Subagents and Skills: The Composition Pattern.....	162

PART IV *Hooks: Claude's Deterministic Enforcement System*

Chapter 15 Hooks Fundamentals: Deterministic Behavior in a Probabilistic System.....	174
Chapter 16 Hook Design Patterns: Gates, Guards, and Notifiers...185	
Chapter 17 Security and Permission Hooks: Protecting Your System.....	197
Chapter 18 Hook Chains and Event Pipelines.....	210

PART V *The Composition Layer: Building a Self-Governing Development Environment*

Chapter 19 The Three-Pillar Architecture: Designing a Complete Claude Code System.....	224
---	------------

Chapter 20 Case Study: Building a Full-Stack Development Environment.....235

Chapter 21 Case Study: The Solo Technical Founder's Claude Code Setup.....255

PART VI *Shipping, Sharing, and Evolving*

Chapter 22 Distribution, Team Adoption, and the Claude Code Marketplace.....274

APPENDICES

Appendix A Troubleshooting and Debugging.....286

Appendix B Hook Events Quick Reference.....292

Appendix C Beyond the Three Pillars: Plugins and Agent Teams.....295

Appendix D Key Terms Index.....299

INTRODUCTION

You are holding a book about a tool that did not exist three years ago. Claude Code shipped to general availability in early 2025, and within twelve months it had transformed how a substantial slice of working software engineers spend their days. The transformation is still in progress. Most engineers using Claude Code today are using it the way most people use a new tool: by typing requests at it and accepting whatever it produces. The handful of engineers who have moved past that posture, who have learned to engineer their environment rather than merely use it, are working at a different level of leverage entirely.

This book is for the engineers who want to make that move. The ones who have noticed that their session quality varies wildly day to day. The ones who have wished they could teach Claude their team's conventions just once and have those conventions stick. The ones who have watched Claude make the same kind of mistake three times in one afternoon and wondered whether there is a way to make it stop. The ones who suspect that the tool is capable of more than they are getting from it, and who are tired of suspecting and want to start engineering.

The argument of this book is that Claude Code, properly understood, is not a chat interface. It is a programmable substrate with three distinct customization mechanisms, and the engineers who learn to use those mechanisms in combination produce results that engineers using the chat interface alone cannot match. The three mechanisms have specific names. Skills are units of reusable capability that teach Claude how to do specific kinds of work consistently. Subagents are isolated specialist contexts that handle particular roles in parallel. Hooks are deterministic shell commands that fire at lifecycle moments and impose mechanical guarantees the model cannot evade. Each of these is documented in Anthropic's official materials. None of them, individually, is the secret. The secret, if there is one, is in their composition.

What this book teaches, across twenty-two chapters, is the architecture for that composition. Part One lays out the framework. Parts Two through

Four go deep into each pillar. Part Five shows the composed architecture at work in two full case studies, one for a senior engineer at a small product company and one for a solo SaaS founder running an entire business out of a single Claude Code installation. Part Six closes with the practical concerns of shipping, sharing, and evolving an environment over time. The four appendices give you reference material to come back to.

A short word about who this book is not for. If you are looking for a quick introduction to Claude Code, this is not it. The official Anthropic quickstart is better for that purpose, and it is free. If you are looking for a tour of every feature in the product, this is not that either. The book covers the three customization pillars in depth and treats other features only when they intersect with the three pillars. If what you want is a comprehensive feature catalog, the official documentation is more current than any book can be, and it should be your first stop.

What this book offers, that the documentation does not, is a coherent way of thinking about how the pieces fit together. The documentation explains each piece accurately. It does not, and cannot, explain why the three pieces sit at different layers of the architecture, what kinds of failures each pillar prevents, in what order to build them, or how to compose them into a working environment. That synthesis is what this book exists to provide. The pieces are public. The architecture is what you are paying for.

A note on how to read the book. The chapters are designed to be read in order, because each pillar builds on the conceptual vocabulary established in the previous one. A reader who jumps directly to the hooks chapters before reading the skills chapters will find the hooks discussion harder than it needs to be, because the deterministic-versus-probabilistic framing that organizes the hooks treatment is set up earlier. That said, after a first read, the book is meant to be returned to non-linearly. The appendices are reference material. The case studies are templates to come back to when you are designing your own environment. Many readers will read the book once cover to cover, then keep it within reach as they build, opening it to specific sections when specific design questions surface.

A note about pronouns. The hypothetical developer throughout the book is referred to with feminine pronouns. This is a deliberate choice and is meant to be read as one. Software engineering is more diverse than its

conventional pronouns suggest, and a book written in 2026 ought to reflect that reality. Readers who find this jarring may want to take a moment to consider why. The book itself proceeds without further comment.

A note about the present moment. This book is being written and shipped at a particular point in the evolution of AI-assisted development tooling. The specific features the book describes are accurate as of early 2026. Some of those features will change. New features will appear. The framework, however, is meant to be durable, because it is rooted in the structure of engineering work itself rather than in the specifics of any particular tool. A reader returning to this book in 2028 will find that some of the syntactic details have moved on, but that the architectural principles still apply, because the underlying division of capability, specialization, and enforcement is a property of the work and not of the tool.

Now to the work. The first chapter opens at the moment most engineers experience as the trigger for taking their tooling more seriously: a Friday afternoon, three different kinds of failure in quick succession, and the dawning recognition that the way they have been using the tool is not going to scale to the work they actually need to do.

PART ONE

Foundations: The Three Pillars of Claude Code Architecture

The gap between using Claude Code and engineering with Claude Code.

CHAPTER ONE

Why Claude Code Alone Isn't Enough

“Tools are not merely aids to the hand. They are extensions of the mind, and the mind that fails to shape them is shaped by them instead.”

Marshall McLuhan, paraphrased

There is a specific moment, familiar to anyone who has lived with Claude Code for more than a few weeks, when the honeymoon ends. The tool still feels magical. The responses are still sharp. But the reader begins to notice something quieter underneath the surface: the same instructions are being typed again. The same three warnings are being issued before every refactor. The same linting step is being requested after every edit. The same four sentences of project context are being pasted into the first message of every new session. The magic is still there, but it has started to feel like a magic that requires a great deal of bookkeeping.

This book was written for that moment. Not the moment of discovery, when Claude Code first opens in a terminal and writes a working function from a vague sentence. Not the moment of curiosity, when the reader begins to explore what the tool can and cannot do. The moment this book was written for is the moment of friction. The moment when the gap between using the tool and being mastered by the tool becomes visible, and when the reader begins to suspect, correctly, that there must be a better way.

The better way is not a better prompt. It is not a longer `CLAUDE.md` file. It is not a collection of tricks. The better way is a shift in how the reader relates to the tool itself: from a user of Claude Code to an engineer of Claude Code. This shift is the central promise of the book, and this chapter is where that promise gets defined.

Consider a concrete scene. A developer sits down on a Tuesday morning to continue work on a medium-sized web application. She opens Claude Code. The first thing she types, as she has typed roughly forty times before, is a reminder that this codebase uses TypeScript with strict mode, that components should be written as functional components with hooks, that tests are written in Vitest rather than Jest, that commit messages follow the conventional commits format, and that any changes to the authentication layer require her personal review. She types this reminder because she has learned, through painful experience, that if she does not type it, Claude Code will occasionally produce perfectly reasonable code that violates every one of those rules.

She then asks Claude Code to add a new feature. The code appears. It is, as always, impressive. She reads it. She notices that the test file is missing, because she forgot to ask for it. She asks for tests. The tests appear.

She notices that one test uses a matcher that exists in Jest but not Vitest. She corrects Claude, who apologizes and fixes it. She commits the change. She notices, twenty minutes later, that her commit message does not follow the conventional commits format, because she wrote it herself in a hurry. She amends the commit.

The Working Developer's Shape

This is a good developer. She is getting real work done. Claude Code is, objectively, saving her hours. And yet, look closely at the shape of her morning. It is full of tiny, avoidable frictions. Instructions she typed that should have been remembered. Checks she ran that should have fired automatically. Specialists she consulted in her own head, one after another, because her tool did not know how to consult them for her. She is doing the work that a well-engineered environment should be doing on her behalf.

Now imagine the same morning in a different configuration of the same tool. She opens Claude Code. She does not type a reminder about TypeScript or Vitest or commit messages, because the environment already knows. When she asks for a new feature, the code appears with tests, because a skill for writing features in this codebase has been installed and knows that tests are part of the definition of done. Before anything is written to disk, a pre-edit hook silently checks the file path against the codebase's protected paths. After the code is written, a post-edit hook runs the linter and the type checker, and if they fail, Claude Code sees the failure and corrects the code before returning control to her. When she is ready to commit, a pre-commit hook runs the full test suite in the background, and a subagent dedicated to commit messages produces a conventional commit message from the diff. She reviews the message, accepts it, and moves on.

The work done in this second configuration is the same work. The tool is the same tool. What has changed is the scaffolding around it. The developer has stopped being the carrier of every rule and every check, and has become, instead, the supervisor of a small, patient, consistent system that carries those rules and checks on her behalf.

The distance between those two mornings is the distance between using Claude Code and engineering with it. And that distance is not a matter of talent, intelligence, or time spent with the tool. It is a matter of knowledge. The second developer knows something the first one does not. She knows

that Claude Code is not a chat interface to a language model. She knows that Claude Code is a programmable environment, and she knows the three systems that make the programming possible.

Those three systems are Skills, Subagents, and Hooks. Every power-user setup, every company-wide Claude Code deployment, every open-source configuration pack that developers are quietly building and trading in private Slacks right now, is built out of some combination of these three primitives. They are the primitives of Claude Code mastery. They are also, surprisingly, the primitives that the overwhelming majority of Claude Code users have never consciously touched.

This is the strange fact at the center of the tool. Anthropic shipped Claude Code with all three of these systems built in from an early stage. The documentation exists. The configuration files are well-designed. The examples in the official repositories are good. And yet, walking into any developer forum or any Claude Code community channel, the overwhelming pattern of conversation is about prompt wording. About trying different `CLAUDE.md` phrasings. About sharing clever one-liners. The three systems that could transform a workflow go almost entirely unmentioned, because they sit one level below the surface, and almost no one has taken the time to descend.

Three Systems, One Architecture

The reader is about to descend. But first, it is worth naming the three systems clearly, so that every subsequent chapter has a stable vocabulary to refer back to.

A Skill is a reusable capability. It is a folder containing a `SKILL.md` file, and possibly supporting scripts or reference material, that teaches Claude how to perform a specific task consistently. When a user asks Claude to perform that task, Claude recognizes the request, reads the skill, and carries it out according to the skill's definition. Skills are how a developer stops repeating themselves. They are the mechanism by which institutional knowledge enters the environment and stays there.

A Subagent is a specialized parallel worker. It is a separate Claude process, with its own system prompt, its own restricted set of tools, and its own isolated context window. The main Claude session delegates work to a

subagent, the subagent performs the work, and only the final result comes back. Subagents are how a single developer becomes the orchestrator of a small team of specialists without paying the cognitive cost of switching roles. A code-reviewer subagent reviews code. A test-writer subagent writes tests. A security-auditor subagent audits security. They do not interrupt each other, they do not pollute each other's thinking, and they do not consume the main session's context window.

A Hook is a deterministic event trigger. It is a shell command, bound to a specific moment in the Claude Code lifecycle, that executes automatically whenever that moment occurs. A hook fires when a tool is about to be used, or just after it was used, or when a session starts, or when it ends. Hooks are how a probabilistic system gets wrapped in deterministic edges. They are how the developer stops trusting Claude to remember to run the tests and starts trusting the hook that always runs the tests whether Claude remembers or not.

Each of these systems, on its own, is useful. A skill saves the developer from typing the same instructions twice. A subagent saves the context window from drowning in noise. A hook saves the workflow from the failure mode of forgetting. But the real power, the power that this book is built around, does not come from the three systems in isolation. It comes from the composition. It comes from building an environment in which all three work together, reinforcing each other, filling each other's gaps, turning Claude Code into something that is less like a tool the developer uses and more like a development environment that the developer has engineered to fit the shape of her work.

That phrase, fully engineered, is not an exaggeration. By the time the reader has finished this book, the reader will have installed skills that teach Claude how to write code the way the reader wants it written. The reader will have defined subagents that handle specialized work without disturbing the main session. The reader will have configured hooks that enforce the boundaries and the rituals of the reader's workflow without any conscious effort. The three systems will be humming together, quietly, behind every prompt the reader types.

Engineering as Platform

There is a word for this in software engineering: platform. A platform is the set of conventions, tools, and automations that shape how work is done within an organization, so that individual engineers do not have to carry all of that shaping in their own heads. What this book teaches, in the end, is how to build a personal platform on top of Claude Code. Not a company platform, necessarily, though the same techniques scale. A personal platform. One developer, one machine, one Claude Code installation, and a set of skills, subagents, and hooks that together make the environment feel as if it was built specifically for the way that one developer thinks.

This is not a small thing to promise. And it is worth being honest about what it costs. Building a personal Claude Code platform requires an investment. Not a large one, but a real one. The developer has to understand the tool well enough to see which frictions are worth automating and which are not. The developer has to be willing to write configuration as carefully as she writes production code, because the configuration is, in effect, production code for her own workflow. The developer has to treat her `.claude` directory with the same respect she treats her source code, versioning it, reviewing changes, and improving it over time.

In exchange for that investment, the developer gets something that compounds. Every skill installed pays dividends for every future session in which that skill fires. Every subagent defined saves the context window on every future task that would otherwise have polluted it. Every hook installed protects every future action from the failure mode it was designed to prevent. A year of thoughtful configuration turns into an environment that is noticeably more productive than any colleague's environment who has not made the same investment. Two years of it turns into a compounding lead that becomes difficult to close.

This is the pitch of the book. It is not a pitch about tricks. It is a pitch about building a platform. The reader who follows the path laid out in these chapters will finish with a platform of her own, and with the understanding of how to extend that platform indefinitely as her work changes and as Claude Code itself evolves.

The rest of Part One is about laying the foundations for that journey. Chapter Two takes the reader under the hood of Claude Code, because no serious engineering is possible without an accurate mental model of the

system being engineered. Chapter Three introduces the three-pillar framework formally, explains how the pillars map to the real problems of daily development, and establishes the composition principle that will guide every subsequent part of the book. By the end of Part One, the reader will have the vocabulary and the mental model needed to understand why the rest of the book says what it says.

It is worth pausing here on a question the reader may already be asking. If the three systems are so valuable, why are they underused? The question has a few honest answers, and each of them is worth naming, because naming them helps the reader avoid the fate of the developers who have not benefited from what the tool already offers.

Why So Few Have Done It

The first answer is visibility. Prompts are visible. A developer sees her own prompts every day, and she sees the prompts that other developers share in public channels, and she forms an intuition that prompting is where the action is. Configuration is invisible. A developer does not see another developer's `.claude` directory by default. It does not show up in screencasts. It does not appear in most tutorials. It is the dark matter of Claude Code: the thing that holds the visible workflow together, but that the visible workflow never calls attention to. The natural result is that developers imitate what they can see, and what they can see is mostly prompts.

The second answer is that configuration feels like overhead. A developer who is trying to ship a feature does not want to stop shipping features in order to design a skill library. The investment looks like a distraction from the work, and the returns are deferred. This feeling is real, and it is also wrong. The returns are not deferred by very long. A well-designed skill pays for itself within a week of normal use, because it runs automatically for every matching task and saves its entire cost every time. A hook pays for itself the first time it prevents a failure that would have taken an hour to clean up. The problem is not that the returns are slow; it is that the returns are distributed across many future sessions, rather than concentrated in the session where the investment was made, so they are psychologically harder to see.

The third answer is cultural. The habit of configuring tools carefully used to be widespread in software engineering, and it has eroded.

Developers customize their editors, but they do not usually customize their terminals, their shells, or their build systems with the same care that was once standard. Claude Code is a tool that rewards that older discipline. It is designed to be bent and shaped, and it gives back, in long-term productivity, whatever the developer is willing to put in during the shaping. Readers who bring the older discipline back with them into this new tool will find themselves at a significant advantage over those who do not.

One more anchor is worth placing before this chapter ends. Consider what Claude Code would feel like if all three pillars were ignored. A developer in that situation is using Claude Code as a conversational interface to a language model. She gets good outputs when she writes good prompts. She gets bad outputs when she writes bad prompts. Her productivity rises and falls with her attention, her memory, and her patience. Now consider the same developer after six months of thoughtful configuration. Her productivity no longer rises and falls in the same way, because the environment has absorbed much of the burden that used to sit on her attention. The ceiling of what she can produce in a day has risen, because the floor has also risen. On her worst days, the environment carries her. On her best days, the environment amplifies her. The variance has fallen and the average has climbed. This is, in the end, the simplest case for engineering rather than merely using: a reduction in variance and an increase in average, across every future session the developer will ever run.

There is a deeper argument to be made, too, about what this kind of configuration does to a developer over time. A developer who has spent six months shaping her Claude Code environment has, in the course of doing so, answered a series of questions about her own work that she might otherwise never have answered explicitly. What is my preferred format for a pull request description? What counts as a complete definition of done for a new feature in my codebase? Which files should never be edited without manual confirmation, and why? What does a good commit message look like in this project, precisely? Every skill, subagent, and hook the developer builds is an answer to one of these questions, captured in a form the tool can use. The side effect of building the environment is that the developer ends up with a much clearer articulation of how she actually wants to work than she had before she started. The environment is a mirror of the developer's craft, and the craft becomes sharper by being mirrored. This is a

benefit that never appears on any productivity chart, but it is one of the most valuable outcomes of the whole exercise, and readers who go through the work will recognize it when it arrives.

The remaining frustration of the Tuesday morning described at the start of this chapter is not an inherent property of working with Claude Code. It is a property of working with Claude Code without having engineered the environment around it. That environment, the one the second developer was enjoying, is buildable by anyone. The building is what this book is about. The next step in the building is understanding the machine we are about to shape, and that is the subject of the chapter that follows.

The Anatomy of Claude Code

“If I had an hour to solve a problem, I would spend the first fifty-five minutes understanding the problem, and the last five solving it.”

attributed to Albert Einstein

To engineer something, one must first see it clearly. This chapter is about seeing Claude Code clearly. Not as a conversational interface, which is how it feels from the outside, but as a layered system with a specific shape, a specific set of moving parts, and a specific vocabulary for the relationships between those parts. Without this clarity, every later chapter in the book will feel like a collection of incantations. With it, every later chapter will feel like the deliberate, ordinary application of a few simple principles to a machine whose behavior is already understood.

The first thing to understand about Claude Code is that, from the moment a session begins, a small but crucial amount of context is being assembled on the user’s behalf, every single turn, before the user ever sees a response. The user types a prompt. That prompt is not sent in isolation to the language model. It is sent as part of a carefully constructed bundle. The bundle includes a system prompt that defines who Claude is and what it is authorized to do within this environment. It includes a description of the tools that Claude is allowed to call, including the file system tools, the shell, the web, and whatever other tools the installation has made available. It includes contextual information about the project, typically harvested from files like `CLAUDE.md` or from explicit references the user has made. It includes, when relevant, the definitions of the skills and the subagents that apply to this session, and it includes the configuration that tells Claude which hooks will fire at which points. Only after that entire bundle is constructed is the user’s prompt appended, and the whole assembly is sent to the language model for a response.

This matters because it means that the user’s prompt is not the sole cause of Claude’s behavior. The user’s prompt is one ingredient among many, and in many cases, it is not even the most important ingredient. The system prompt is more important. The `CLAUDE.md` content is often more

important. The skills that are recognized as applicable are often more important. A developer who believes that the only way to change Claude's behavior is to change the wording of her prompts is a developer operating with a faulty mental model of where behavior comes from. The model is mostly not in the prompt. The model is in the bundle.

The second thing to understand is that Claude Code is agentic. The word agentic has been overused in the last two years to the point of fatigue, but in this specific context it has a precise and useful meaning. An agent, in the sense that matters here, is a language model that has been given the ability to call tools, observe the results of those tool calls, and decide, based on those observations, what to do next. Claude Code is an agent in exactly this sense. When a user asks Claude Code to add a new feature to a codebase, Claude does not produce the code in a single shot and then stop. Claude produces a plan, inspects files to verify assumptions about the codebase, reads surrounding code to understand conventions, writes a first draft of the new code, sometimes runs tests or type checks, observes any failures, and adjusts the code in response. This loop of propose, act, observe, adjust is the beating heart of the tool, and every subsequent system in this book is ultimately a way of shaping what happens inside that loop.

The Loop

The loop is worth naming because it will be referred to constantly. The loop has three phases per iteration. In the first phase, Claude thinks, which is to say, the language model produces text that constitutes its next reasoning step. In the second phase, Claude acts, which is to say, Claude chooses a tool to call and provides the arguments to that tool. In the third phase, Claude observes, which is to say, the result of the tool call is read back into the context window, and Claude decides whether it is done or whether another iteration is needed. Think, act, observe. Think, act, observe. The loop runs until Claude decides, based on the current state of the context window, that the task is complete.

This loop is the reason subagents exist, the reason hooks exist, and the reason skills matter. Subagents are a way of saying, this part of the loop is a different kind of work, and it deserves its own isolated loop with different rules. Hooks are a way of saying, whenever the act phase is about to call this specific tool, run this shell command first, or after, to enforce

something deterministic that the model alone cannot be trusted to enforce. Skills are a way of saying, this kind of task recurs often enough that it deserves a standing set of instructions, so that when the loop encounters it, the right behavior is already present in the context bundle.

The third thing to understand is the file system. Claude Code is not a disembodied agent floating above a project. It runs inside a specific working directory, and it is deeply aware of the structure of that directory. A set of conventions controls what Claude sees and how Claude configures itself. The most important of these conventions is the `.claude` directory. The `.claude` directory sits at the root of a project, or in the user's home folder, or in both, and it is where configuration lives. Skills live in `.claude/skills/` as subfolders. Subagent definitions live in `.claude/agents/` as files. Hook definitions live in the settings file at `.claude/settings.json`, or in the corresponding user-level settings file. The structure of this directory is, in effect, the source code of the developer's Claude Code environment.

There is a hierarchy at play. User-level configuration, which lives in the home directory, applies to every Claude Code session the user opens, on every project. Project-level configuration, which lives in the `.claude` directory at the root of a specific project, applies only when Claude Code is opened inside that project. When both are present, the project-level configuration generally takes precedence for project-specific concerns, while the user-level configuration provides a stable personal baseline. A developer who works on five different projects does not need to redefine her personal commit-message skill five times. She defines it once at the user level, and it travels with her. When a specific project needs a project-specific skill, a pre-commit hook that runs that project's specific test suite, or a subagent trained on that codebase's peculiar conventions, those live in the project's `.claude` directory and are committed to source control alongside the code itself.

Where Configuration Lives

This last point deserves emphasis. The project-level `.claude` directory is part of the codebase. It is as much part of the codebase as the source files, the package manifest, or the README. It should be committed, reviewed in pull requests, and evolved deliberately. When a developer joins a team, the team's `.claude` directory travels to her automatically when she clones the

repository. She inherits the team's skills, the team's subagents, and the team's hooks at the moment she starts working. She does not have to be told, orally, what the team's conventions are, because the conventions are encoded as configuration that Claude enforces for her. This is how a well-engineered environment spreads. Not through wikis. Not through onboarding videos. Through version-controlled configuration that the tool itself honors on the first session.

The fourth thing to understand is `CLAUDE.md`. This file is a special convention that Claude Code looks for automatically, and whose contents are added to the system context at the start of every session. `CLAUDE.md` is the place for information that applies to every interaction within a project. It is where one writes, this codebase uses TypeScript with strict mode; tests are in Vitest; authentication changes require careful review. `CLAUDE.md` is not a skill, and it is not a substitute for one. A skill is targeted; it activates when its description matches the user's intent. `CLAUDE.md` is ambient; its contents are always loaded. The right things to put in `CLAUDE.md` are things that need to be true at all times. The wrong things to put there are things that only matter when a specific kind of work is being done, because those things should be skills instead, and should be activated only when they are needed.

This distinction, ambient versus targeted, is one of the design questions the reader will be making over and over again throughout the book. When a piece of knowledge belongs in `CLAUDE.md`, and when it belongs in a skill, is a judgment call. A rule of thumb that will serve well: if the knowledge is relevant to almost every action in the project, put it in `CLAUDE.md`. If the knowledge is relevant only to a specific kind of action, put it in a skill, and write the skill's description so that the skill fires when that kind of action is being requested. This rule will be refined as the book proceeds, but it is a good starting place.

The fifth thing to understand is the context window. Every model has a finite context window, and every session spends that window progressively as it runs. The context window contains the system prompt, the assembled bundle of skills and subagents and hooks that apply to this session, the `CLAUDE.md` content, the history of the conversation so far, and the outputs of every tool call Claude has made. The context window fills up. When it fills up, the quality of the model's responses degrades, because the model has to

work harder to find the relevant information among the noise. Eventually, the context window fills up entirely, and something has to give.

Context as a Scarce Resource

This is why subagents matter so much. A subagent, when it runs, has its own fresh context window. The main session dispatches a task to the subagent, the subagent does the work in its own private context, and only the subagent's final output is added back to the main session's context. The dozens of tool calls the subagent may have made, the files it may have read, the drafts it may have discarded, none of it enters the main session. The main session stays clean. The developer who designs subagents well is, in effect, designing a context budget, a plan for what deserves to live in the main session's precious memory and what should be delegated to a subprocess that will return only its conclusion.

The sixth thing to understand is the lifecycle. A Claude Code session is not a single event. It has stages. There is the session start, when the environment is assembled and the initial context is loaded. There is the prompt submission phase, when the user's input is appended to the context. There is the tool use phase, when Claude decides to call a tool; this phase is composed of an approval step, the actual tool call, and the observation of its result. There is the stop phase, when Claude decides the task is done. There is the session end, when the process terminates. Each of these stages is a moment at which something can be made to happen. The hook system is precisely the mechanism by which a developer attaches behavior to these moments. When a developer wants something to happen before every tool use, she attaches a `PreToolUse` hook. When she wants something to happen after Claude finishes speaking, she attaches a `Stop` hook. The lifecycle is the grid of available moments, and hooks are the pins that attach behavior to the grid.

A seventh observation, and the last of the anatomy, concerns the way Claude Code thinks about safety. Every tool use is, by default, a point of trust. The language model decides, based on the user's prompt and the context bundle, that a certain tool should be called with certain arguments. If that decision is wrong, and if the tool has consequences in the outside world, the wrong decision becomes a real event. Claude Code mitigates this in several ways that the reader should be aware of. Tool permissions can be

restricted at the session, project, or user level, so that certain tools are never available in certain contexts. Approvals can be required for specific categories of tool use, so that the developer retains a human check on high-impact operations. And hooks, as will be seen in the fourth part of this book, can intercept tool uses before they happen and reject them on any grounds the developer can encode in a shell command. Safety, in this model, is not a property of the language model alone. It is a property of the configured environment, and the developer is the one who configures it. This responsibility is not heavy; it mostly consists of making good defaults and then forgetting about them. But it is real, and the reader who takes the time to set good defaults early will never have to think about them later, which is the best outcome a safety system can offer.

This is the whole machine, seen from enough height to be understood in a single glance. A language model, wrapped in an agent loop, wrapped in a file-system-aware configuration system, wrapped in a lifecycle of named events, wrapped in a session that the user drives by typing prompts. Every sophisticated use of Claude Code that the reader will encounter in this book, and every sophisticated use she will design on her own in the years to come, is some configuration of this machine. There is no hidden trick. There is no magic. There is a well-defined system, and the question is only how well the developer understands that system and how deliberately she shapes it.

Composition by Default

There is also a subtler property of Claude Code worth surfacing while the anatomy is fresh: the environment is not static. Anthropic has been evolving the tool continuously, adding new lifecycle events, refining the skill system, expanding what subagents can do, broadening the matchers available to hooks. The developer who engineers her environment in this tool is aiming at a moving target, and that is not a drawback. It is a feature. The principles that the book is about to teach are stable principles, rooted in the shape of the loop and the lifecycle rather than in any specific version of the tool. A developer who understands skills, subagents, and hooks in the sense this book defines them will be able to absorb new features as they ship, because she will recognize each new feature as a variation on a theme she already knows. The framework is, in that respect, a defense against the churn of the underlying software. The developer who has an opinion about how her

environment should be shaped is the developer who can evaluate new features on their merits, keep what serves her work, and ignore what does not. The developer who has no such opinion will be pushed around by whatever the current release notes emphasize.

A short diagnostic exercise is worth performing here, because it will make the abstractions of this chapter much more concrete. The reader is encouraged, when she has a moment at her own terminal, to open a project in Claude Code and explicitly map its anatomy. What is in the project's `CLAUDE.md` file, if one exists? What skills live in the project's `.claude/skills` directory, and what do their descriptions say? What subagents are defined, and what do their system prompts establish about their specialization? What hooks are configured in the project's settings file, and at which lifecycle events do they fire? This exercise, which should take only fifteen or twenty minutes, often reveals two things. First, it reveals that the environment the developer has been operating in has more shape than she realized, because previous developers or the team standard has already put some configuration in place. Second, it reveals the gaps, which is to say, the places where no configuration exists and where the developer has been filling the gap with her own attention, session after session. Those gaps are the raw material for the rest of the book. Every gap is a candidate for a skill, a subagent, or a hook, and by the end of the book the reader will have a considered view on which is appropriate for which gap.

One final note is worth making before this chapter closes. The word configuration has been used many times in these pages, and it is a word that can sound dry and administrative. It is not. In the world of modern software, configuration is power. The most influential tools in any developer's life are tools whose default behavior can be bent by configuration to fit that developer's specific needs. A text editor is an example; the raw editor is impressive, but an editor configured over years with the right extensions, keybindings, and snippets becomes something much more powerful than the sum of its defaults. A shell is another; the raw shell is capable, but a shell configured with the right aliases, functions, and prompt becomes an extension of the developer's intentions rather than a tool she is constantly wrestling with. Claude Code is, in this respect, more like a modern editor or a well-configured shell than like a chat interface. It is meant to be configured. It is designed with configurability as a first-class

concern. The developer who treats Claude Code as a tool to be configured, rather than merely used, is the developer who will extract the full value from it. Everything in this book is an attempt to teach that configuration seriously and well.

With the anatomy clear, the chapter that follows introduces the three-pillar framework that organizes all of that configuration into a coherent system of thought. The framework is the single most important concept in this book, and the rest of the book is, in a sense, a careful elaboration of it. The reader who has internalized the anatomy described here is now ready to see the framework for what it is.

Skills, Subagents, and Hooks: The Three-Pillar Framework

“Architecture is the art of how to waste space beautifully, or how to save it meaningfully. It is never neutral.”

Philip Johnson, adapted

Every discipline eventually discovers that its power comes less from any single technique and more from the framework that organizes its techniques into a coherent whole. In medicine, the framework of anatomy organizes the study of the body. In music, the framework of harmony organizes the study of chords. In software engineering, frameworks of architecture organize patterns that would otherwise feel like a disordered bag of tricks. This chapter introduces the framework that organizes everything in this book. It is a simple framework with only three parts, but the simplicity is deceptive. Once the reader has internalized these three parts and the relationships between them, every subsequent chapter in the book reads as a natural elaboration. Before the reader has internalized them, the chapters look like a catalog of features. After, they look like a single unified argument.

The framework is this. Claude Code mastery is built on three pillars. The first pillar is Skills, and it supplies capability. The second pillar is Subagents, and they supply parallelism and specialization. The third pillar is Hooks, and they supply determinism. Every sophisticated workflow is some combination of these three, and the real art of mastery lies not in any one of them in isolation but in how they are composed.

Begin with the first pillar. A Skill is the answer to a specific question: how does a developer teach Claude to do something consistently, across sessions, across projects, without repeating herself? The question looks simple, but it is the root of an enormous amount of developer frustration. A developer writes a prompt that produces excellent output. She is pleased. She moves on. Three weeks later, she wants that same kind of output again, and she cannot remember exactly how she phrased the prompt. She tries to

reconstruct it. The result is not quite as good as before. She keeps trying. Eventually, the reconstruction converges to the original, but she has wasted thirty minutes. This is a daily experience for every Claude Code user who has not adopted skills. Skills are the cure.

A skill is, in its essence, a promise. It is a promise that whenever a certain kind of task is requested, Claude will remember how to do that task well, because the instructions for doing it well have been captured in a file that Claude will read. The file is called `SKILL.md`. It lives in a folder named after the skill, inside either the user's personal skill library or the project's skill library. It has a structured head and a free-form body. The head declares the skill's name, its description, and the tools it is allowed to use. The body contains the instructions themselves, in plain prose, with examples, references, and any other material that would help a capable reader do the task well for the first time.

When Claude Code starts a session, it discovers all of the available skills. It does not load their full contents at once, because that would waste the context window. It loads only the descriptions, because the descriptions are enough to tell Claude when each skill is relevant. When a user issues a prompt, Claude scans the descriptions of the available skills and recognizes which ones, if any, apply to the prompt. If a skill applies, Claude reads its body and uses those instructions to shape its response. This is what progressive disclosure means in the context of skills: only what is relevant gets read, at the moment it becomes relevant, and nothing more.

This mechanism is what makes skills practical. A developer can accumulate dozens or even hundreds of skills without paying a context-window tax for the ones she is not using. Only the skills that match the current request cost tokens. Every other skill sits quietly in the library, waiting for the kind of request that would activate it. A skill, in this sense, is a dormant capability, and a skill library is a collection of dormant capabilities. When the library has been designed well, the developer's work activates the right capabilities at the right moments, and a substantial portion of the developer's daily knowledge work runs itself.

Skills in Practice

What does a skill look like in practice? Consider a simple example. A developer writes a lot of pull request descriptions, and has developed a

specific format she likes: a one-sentence summary, a bulleted list of what changed, a note about testing, and a note about risk. She has typed this format into prompts hundreds of times. She sits down one evening and writes a skill called pr-description. In `SKILL.md`, she writes a description that says, activate this skill whenever the user asks for a pull request description, PR description, or PR body. In the body, she lays out the format in detail, with an example filled in. She saves the file, restarts her Claude Code session, and the next time she types, give me a PR description for this branch, the skill fires. The description that comes back follows her format exactly. She does not explain the format in the prompt. She does not need to. The skill remembers for her.

This is a small example, but it is representative. Every skill is, at its core, the codification of something that used to live only in the developer's head and her prompting habits. Skills make that knowledge durable. They are how a developer turns her accumulated expertise into configuration, and through configuration, into compounding leverage.

Now turn to the second pillar. A Subagent is the answer to a different kind of question: how does a developer do many kinds of work at once, without drowning her main session in the noise of all those kinds of work, and without losing the specialized focus that each kind of work deserves?

Every non-trivial development task is really several tasks in a trench coat. A feature request involves reading existing code, designing new code, writing the new code, writing tests for it, updating documentation, and reviewing the whole for quality. A single Claude session can do all of these things in sequence, and it often does, but there is a cost. Every file Claude reads in the course of understanding the existing code takes up space in the context window. Every false start, every draft, every experiment pollutes the history of the session. By the time Claude is writing the tests, the context window is crowded with exploration residue that has nothing to do with testing. The tests it writes, as a result, are sometimes shaped by irrelevant context that it cannot easily ignore.

A subagent solves this by giving a specific kind of work its own isolated environment. When the main session needs tests written, it dispatches a test-writer subagent. The subagent has its own system prompt, tuned for the craft of writing tests. It has its own set of allowed tools, which

probably includes reading code and writing new test files but probably excludes making commits or touching production configuration. It has its own context window, which starts empty and fills only with the information needed for the specific test-writing task at hand. When the subagent finishes, it returns its output to the main session. The main session receives the tests. The entire exploration the subagent did to produce those tests does not enter the main session's context. The main session stays focused.

This architecture has several virtues that are worth naming. First, it saves context, which directly improves the quality of everything else the main session does. Second, it allows specialization, because each subagent can be tuned for its specific kind of work, with a role definition and tool restrictions appropriate to that work. Third, it allows parallelism, because multiple subagents can run at the same time, each on an independent piece of a larger task, and the main session can synthesize their outputs. A developer with a well-designed subagent library is able to dispatch a security review, a performance audit, a documentation pass, and a test-coverage check simultaneously on a single pull request, and receive four independent reports back. A single-threaded session could never do this well.

Subagents in Practice

Subagents, like skills, are defined in the `.claude` directory. Each subagent is a file that declares its name, its description, its allowed tools, and its system prompt, the last of which is where the specialization actually lives. The system prompt of a code-reviewer subagent is different from the system prompt of a test-writer subagent, because code review and test writing are genuinely different crafts. The art of subagent design, which later chapters will treat in depth, is in part the art of writing these focused system prompts well.

There is one further property of subagents that deserves mention before leaving the second pillar. Subagents can themselves invoke skills. A test-writer subagent, when it starts, can have access to a skill that encodes the project's test conventions, and it can activate that skill just as the main session would. This is where the framework begins to compose. A subagent is not merely a specialist; it is a specialist who inherits the knowledge the developer has encoded into the environment. The specialist-plus-skill

combination is strictly more powerful than either alone, and the real art of building a Claude Code platform involves designing these combinations deliberately.

The third pillar. A Hook is the answer to the third question: how does a developer ensure that the things that must happen always happen, not merely most of the time?

Language models are probabilistic. This is a statement of design, not of failure. A probabilistic system produces different outputs for the same inputs, and this is what allows a language model to handle the open-ended variety of human requests. The trade-off is that a probabilistic system cannot be trusted to always do anything. It can be coaxed, in most cases, into doing a thing reliably, but reliably is not the same as always. For the things that must happen always, probabilistic behavior is not enough.

Consider the category of things that must happen always. Tests must be run before commits. Linters must be run after edits. Secrets must never be written to disk. Production configuration files must never be edited without explicit confirmation. Long sessions must not terminate silently without a summary being logged somewhere the developer can review. Each of these is a requirement where the acceptable failure rate is zero. A ninety-eight percent success rate is not enough. A developer who tells Claude, by prompt alone, to always run tests before committing, will find that in ninety-eight out of a hundred cases Claude does exactly that. In the other two, something else happens. Usually nothing bad. Occasionally something very bad. The whole point of the rule was to prevent the very bad thing, and a rule that fails two percent of the time is barely a rule at all.

A hook is the mechanism by which a developer can, at specific moments in the Claude Code lifecycle, insert a deterministic shell command that runs whether Claude remembers or not. If the hook fails, the lifecycle is interrupted. If the hook succeeds, the lifecycle continues. The language model does not get to override the hook, because the hook is not part of the language model's loop. The hook is outside the loop. The hook is wrapped around the loop.

The Hook Layer

This placement, outside the loop and around it, is the whole source of the hook's value. Hooks do not replace the language model's judgment; they constrain the environment in which the language model is making its judgments. A hook that refuses to let any tool use touch the `.env` file does not make the model smarter. It makes it unnecessary for the model to be perfectly vigilant about the `.env` file, because the `.env` file is protected by a rule the model cannot break. This frees the model to think about the work that does require judgment, while the work that requires only enforcement is handled by the hook. The probabilistic engine is left to do the probabilistic work, and the deterministic requirements are handled deterministically, each in its proper place.

Hooks are defined in the `settings.json` file, at either the user level or the project level. Each hook declares which lifecycle event it responds to, an optional matcher that narrows when it should fire, and the command to run. The command can be any shell command. It can be a one-liner or a call to a full script. It can read the context of the hook from standard input in a structured format, and it can signal approval or rejection of the current action through its exit code and output. The design is deliberately simple at the configuration layer, so that the complexity can live in the scripts themselves, where it belongs.

Having named the three pillars, the natural next question is how they relate to each other. The short answer is that they are complementary, and they become far more powerful together than any of them could be alone. The longer answer is the composition principle, which is the central design principle of the book, and which deserves its own short treatment before this chapter closes.

The composition principle states that any sophisticated Claude Code workflow will use all three pillars, because each pillar addresses a different class of problem, and real workflows contain all three classes of problem. Skills address the problem of how to do a task well. Subagents address the problem of how to keep different kinds of work separated. Hooks address the problem of how to enforce what must happen. A workflow that uses only skills is a workflow that can teach Claude to do good work but has no way to isolate different kinds of work and no way to guarantee enforcement. A workflow that uses only subagents has specialization but no stored knowledge and no enforcement. A workflow that uses only hooks

has enforcement but no capability and no specialization. Each pillar, alone, is a partial answer. All three, together, form a complete answer.

The composition can be illustrated with a simple image. Imagine a developer whose morning routine includes reviewing the previous day's pull requests. She has a skill called pr-review that encodes the specific kind of review she likes to write: substantive comments, clear suggestions, and a summary judgment. She has a subagent called security-auditor, specialized for flagging potential security issues, with access to the file system and to a small set of security-focused reference material. She has a hook on `PreToolUse` that blocks any attempt to approve a pull request that touches the authentication layer without her explicit confirmation.

When she sits down in the morning and asks Claude Code to review a specific PR, the skill fires because the description matches. Claude reads the skill and structures its review accordingly. During the review, Claude dispatches the security-auditor subagent in parallel, so that security issues are surfaced by a specialist rather than by Claude's generalist attention. The subagent returns its findings. Claude synthesizes the review, including the security findings, according to the format the skill has established. If Claude, perhaps through a subsequent prompt, attempts to approve the PR and the PR touches the authentication layer, the hook fires, blocks the approval, and informs Claude that explicit human confirmation is required. At no point has the developer had to remember to do any of this. The environment remembered. The environment composed its three pillars on her behalf, quietly, exactly when each was needed.

Engineering in Miniature

This is what it looks like, in miniature, to engineer with Claude Code. Skills provide the knowledge. Subagents provide the specialized attention. Hooks provide the guarantees. The developer provides the architecture, once, and then spends the subsequent months and years harvesting the dividends of that architecture in every session she runs.

The remaining five parts of this book are, at heart, a detailed elaboration of this framework. Part Two teaches skills in depth, from their anatomy to their composition, and sends the reader out of it with a working personal skill library. Part Three teaches subagents, from their fundamentals to the orchestration patterns that unlock parallelism, and sends the reader out of it

with a working team of specialists. Part Four teaches hooks, from their event model to the design patterns that make them reliable, and sends the reader out of it with a hardened, secured, deterministic environment. Part Five is the composition layer, where the three pillars are assembled into a full personal platform, and two end-to-end case studies walk the reader through building that platform for specific kinds of work. Part Six closes the book by teaching how to ship, share, and evolve a Claude Code platform over time.

The reader who has followed the argument of this chapter now has everything she needs to understand why the book is organized the way it is. The three pillars are the organizing structure. Every chapter is an elaboration of one of them, or of the composition between them. The journey is straightforward, and the destination is a Claude Code environment that the reader has genuinely engineered to fit the shape of her work.

A closing thought, offered in the spirit of calibrating expectations, is worth including here. The framework in this book is deliberately simple. Three pillars, one composition principle, a small number of recurring design questions. This simplicity is a strength, not a weakness. A framework that can be held entirely in working memory is a framework that a developer can actually use, in real time, in the middle of the day, while she is trying to decide whether a given piece of workflow friction should become a skill, a subagent, a hook, or nothing at all. A more elaborate framework, with a dozen categories and a dozen principles, would be impressive to read about and useless to apply. The three-pillar framework is sized for the working life of a human being who also has other things to do. That sizing is part of the design.

The simplicity also has an implication for the reader's relationship with the book. None of the chapters that follow require memorization. They require understanding, and the understanding is built on the same three pillars, considered from different angles. A reader who has genuinely absorbed this chapter can, in principle, reason her way to many of the conclusions the later chapters will reach. The later chapters will spare her the effort, and will also introduce specific techniques, patterns, and configurations that would be hard to derive from first principles. But the

backbone of the thinking is already in place. What remains is the detailed craft of applying it.

Part One closes here. Part Two begins with the deepest single topic in the book, which is the anatomy of the `SKILL.md` file, and it is there that the real work begins.

PART TWO

Skills: Claude's Extensible Knowledge System

Knowledge that belongs in your head should not stay there. Move it.

CHAPTER FOUR

Skills Fundamentals: The SKILL.md Anatomy

“The structure of a thing is the meaning of a thing. Change the structure and you have changed the meaning, whether you intended to or not.”

Christopher Alexander, paraphrased

This chapter teaches the anatomy of a skill. Not the philosophy, not the strategy, not the high-level reasons skills exist. The anatomy. By the end of this chapter, the reader will be able to look at any `SKILL.md` file in the wild and explain, line by line, what every part of it does, why it is shaped the way it is, and how it will behave when Claude encounters it in a real session. The reader will also be able to write a `SKILL.md` file from a blank page without consulting documentation, because every field, every section, and every convention will be in working memory.

Begin with the simplest possible definition. A skill is a folder. The folder lives in either the user's personal skills directory or in the project's skills directory. The folder contains, at minimum, one file named `SKILL.md`. The folder may also contain subfolders for supporting material, but those are optional. The skill's name is the name of the folder. That is the entire structural definition.

The folder location matters, because location determines scope. A skill placed in the user's personal skills directory, which by convention lives at `~/claude/skills/` on Unix-like systems, is available in every Claude Code session that user opens, regardless of which project that session is operating on. A skill placed in a project's skills directory, which lives at `.claude/skills/` at the root of that project, is available only when Claude Code is run inside that project. This split exists for an obvious reason. Some skills are personal; they reflect the way a particular developer prefers to work, and they should travel with that developer across every project. Other skills are project-specific; they encode conventions that apply only to a specific codebase, and they should travel with that codebase wherever it goes, including to teammates who clone the repository fresh.

The folder name becomes the skill name. There are conventions worth following. Skill folder names should be lowercase, hyphenated, and descriptive. A skill that drafts pull request descriptions might live in a folder called `pr-description`. A skill that scaffolds new React components might live in `component-scaffold`. The name is what Claude will see when it loads the skill's metadata at startup, and it is what the developer will see in any tooling that lists installed skills. A clear, short, descriptive name is, in this respect, a small but real act of good engineering.

The `SKILL.md` File

Inside the folder sits SKILL.md. This file has two parts, and the parts are separated by a fence of three hyphens on a line of their own. The first part is YAML frontmatter, which carries structured metadata about the skill. The second part is the markdown body, which carries the instructions Claude will follow when the skill is active. The frontmatter is short. The body is as long as it needs to be, within sensible limits that this chapter will discuss in due course.

```
---
name: pr-description
description: Drafts a pull request description in the conventional format for this team. Use
whenever the user asks for a PR description, pull request description, PR body, or asks "write a
PR for this branch" or similar phrasing.
---

# PR Description

When asked to write a pull request description, follow this format exactly:

## Summary
A single sentence that explains what this PR does, written in the imperative mood.

## What changed
A bulleted list of the concrete changes, one bullet per logical change.

## Testing
A short paragraph describing how the change was tested. If new tests were added, mention them.

## Risk
A single sentence categorizing risk as low, medium, or high, with one short reason.

Always read the diff before writing the description. Do not invent changes that do not appear in
the diff.
```

The example above is a complete, working skill. It has a name. It has a description. It has a body. It can be saved into a folder named pr-description, that folder can be placed inside `~/claude/skills/`, and on the next Claude Code session it will be discoverable. When the user later types something like give me a PR description for this branch, Claude will recognize that the skill applies, load its body into context, and produce output that follows the format the body specifies.

Look closely at the frontmatter. There are exactly two required fields. The first is name, which must match the folder name and is what Claude uses to refer to the skill internally. The second is description, which is the field that determines whether and when the skill is invoked. The description is the most important field in the entire file, and the next chapter is devoted entirely to writing it well, because no other single decision affects skill behavior more.

There is also an optional third field worth knowing about now: `allowed-tools`. This field, when present, restricts the set of tools Claude is permitted to use while operating under this skill. A skill that exists only to compose written prose, for example, might restrict tools to Read and Grep and exclude Edit, Write, and Bash, so that the act of using the skill cannot accidentally modify the file system. The `allowed-tools` field is a Claude Code feature; when skills are used through the Anthropic API or the SDK, tool access is controlled at the application level instead. For most readers of this book, who are running skills inside Claude Code, `allowed-tools` is a useful safety mechanism that should be reached for whenever a skill has no legitimate reason to write to disk or run shell commands.

A Worked Example

A second example, slightly larger, will demonstrate `allowed-tools` in context.

```
---
name: explain-codebase
description: Produces a high-level architectural summary of a codebase using only read
operations. Use when the user asks "explain this codebase," "give me a tour of this repo," "what
does this project do," or any request to understand a codebase without modifying it.
allowed-tools: Read, Grep, Glob
---

# Explain Codebase

When the user asks for an explanation of a codebase, follow this procedure:

1. Read the README and any top-level documentation files first.
2. List the top-level directories and infer the architectural pattern from their names.
3. Read package.json or the equivalent dependency manifest to understand the technology stack.
4. Identify the entry point of the application by checking the manifest for a "main" or "bin" field.
5. Read the entry point file and trace the first one or two layers of imports.
6. Produce a summary with these sections:
```


- Stack
- Architecture
- Entry point and request flow
- Notable patterns or conventions
- Open questions or things you would need to read more to understand

Never modify any files while operating under this skill. The allowed-tools restriction enforces this, but the instruction is repeated for clarity.

The reader should pause and notice what just happened. With twenty-five lines of plain text in a folder, the reader's Claude Code installation has been taught a procedure that is now repeatable, consistent, and safe. The next time the reader joins a new project and asks Claude to explain the codebase, the same procedure will run, in the same order, with the same output structure, and Claude will be physically incapable of writing to any file while doing so. This is the leverage skills offer. A small, careful piece of configuration produces a substantial and ongoing improvement in the developer's daily experience.

The body of a skill is not free-form, but it is flexible. There is no required template. Headings, lists, examples, code samples, references to other files, are all allowed. The two principles that govern good skill body design are clarity and concision. Clarity, because Claude is going to follow the body's instructions literally, and ambiguity in the body produces ambiguity in the output. Concision, because the body counts against the context window every time the skill activates, and a verbose body wastes tokens that could be spent on the actual work.

A useful rule of thumb is that a `SKILL.md` body should stay under five hundred lines. If a skill seems to want more than that, the right move is almost always to factor the surplus content into separate files within the skill folder and reference them from the body. A skill folder by convention may contain three kinds of supporting material, organized into three subfolders. The scripts folder contains executable code that Claude can run as part of the skill. The references folder contains additional documentation that Claude should read when relevant, but that does not need to live in `SKILL.md`. The assets folder contains files that Claude will use as templates or fixtures during the skill's execution. These three subfolders are conventions established by Anthropic in the official skills repository, and

following them makes the skill folder predictable to read for anyone who joins later, including future versions of the developer who wrote it.

The progressive disclosure principle, mentioned in the previous chapter, is what makes the bundled-files pattern practical. When Claude starts a session, it loads only the descriptions of available skills, not their bodies. When Claude decides that a specific skill applies, it loads that skill's `SKILL.md` body into context. Only at that point, if the body references a file in references or asks for the execution of a script in scripts, does Claude reach further into the skill folder to pick up what is needed. The total token cost of having forty skills installed, none of which is currently relevant, is roughly the cost of forty short descriptions. The cost only rises when a skill becomes relevant, and even then, the rise is bounded by what the body and any referenced files actually contain.

Skills as a System

Take a moment to consider the architecture from this angle. The skill system is a three-layer cache. The first layer, always loaded, is the metadata: name and description for every skill. The second layer, loaded on demand when the skill activates, is the `SKILL.md` body. The third layer, loaded only when the body asks for it, is whatever lives in scripts, references, and assets. Each layer costs more tokens than the one above it, and each layer is reached only when the layer above has decided that more detail is required. This cascade is what allows a Claude Code installation to host a substantial library of skills without paying a substantial token tax for them.

There is a final aspect of skill anatomy that deserves attention before this chapter closes, and it concerns the relationship between skills and Claude Code's older slash-command system. Slash commands are the explicit, user-invoked counterpart to skills. A user types a slash followed by a command name, and Claude executes the corresponding command. Slash commands have existed in Claude Code longer than skills have, and many developers have built up libraries of them. As of recent updates, slash commands and skills have been unified at the implementation level, which means that a file at `.claude/commands/deploy.md` and a skill at `.claude/skills/deploy/SKILL.md` both register a `/deploy` slash command and behave the same way when invoked explicitly. The practical

recommendation that emerges from this convergence is to prefer the skill format for any new work, because skills support the full feature set, including bundled files and automatic invocation, whereas the older command format is a subset.

Automatic invocation is the property that makes skills feel different from slash commands in daily use. A slash command requires the user to remember the command and to type it. A skill, when its description is well written, requires nothing of the user except that she describe her task in natural language. Claude reads the descriptions, recognizes which skills apply, and activates them silently. The user does not need to know which skills exist or which is firing at any given moment. The environment simply gets better at the user's requests, in ways that match the configuration the developer has installed. This silent, automatic activation is the property that makes skills, in the words of one prominent commentator, possibly a bigger deal than MCP. The barrier between intention and capability collapses to zero. A user describes what she wants, and the right capability arrives on its own.

Before this chapter closes, a third worked example is worth examining, because it shows the anatomy at its most expressive. The previous two examples were small. Real skills, the ones that earn their place in a working library, often carry a little more weight. The example below is a skill called `sql-review`, and its job is to review a SQL query for performance, correctness, and security issues before it goes anywhere near a production database.

```
---
name: sql-review
description: Reviews a SQL query for performance, correctness, and security issues. Use
whenever the user pastes a SQL query and asks "review this query," "is this efficient," "will this
scale," "any issues with this SQL," or describes wanting feedback on a query before running it.
This skill is for review only; it does not execute the query.
allowed-tools: Read
---

# SQL Review

When asked to review a SQL query, follow this procedure exactly.

## Step 1: Read the query carefully
```

Read every clause. Identify the tables touched, the join conditions, the filter conditions, the projection, and any subqueries or CTEs.

Step 2: Performance review

Check for:

- Missing or non-sargable WHERE clauses (functions wrapping indexed columns)
- Joins on non-indexed columns
- SELECT * in production paths
- Cartesian products from missing or wrong join conditions
- Subqueries that could be CTEs or joins for clarity and performance
- Aggregations over large tables without filters

Step 3: Correctness review

Check for:

- NULL handling, especially in joins and aggregations
- Off-by-one errors in BETWEEN and date ranges
- Implicit type coercion that could mask bugs
- LIMIT without ORDER BY, which produces nondeterministic results

Step 4: Security review

Check for:

- String concatenation building queries (SQL injection vector)
- Missing parameterization where user input is involved
- Permissions assumed but not verified

Step 5: Output format

Produce the review in this format:

Verdict

One sentence: safe to run, run with caution, or do not run.

Performance

A short bulleted list of issues, each with a one-line explanation and a one-line fix.

Correctness

Same format.

Security

Same format.

Suggested rewrite

If issues are significant, provide a rewritten query in a fenced code block. If the original is fine, omit this section.

Do not invent issues. If the query is good, say so plainly. A short clean review is better than a padded one.

Multi-File Skills

That example deserves a moment of attention. It is twice as long as the earlier examples, and the additional length is doing real work. The body is broken into named steps, each step has a focused checklist, and the output format is specified down to the section headings. A skill written this way produces strikingly consistent output, because almost every degree of freedom that could produce variability has been pinned down. The query is read in the same order every time. The same five performance issues are looked for in the same order. The output is structured the same way every time. The reader who runs this skill against ten different queries over the course of a week will get ten reviews that feel like they came from the same careful reviewer, because in a meaningful sense they did.

This is a property worth dwelling on. Skills do not merely save typing. They produce consistency. A developer working alone can have ten different commit messages on a Monday and forty different commit messages by Friday, all of them written by her, none of them following the same internal logic. The same developer with a commit-message skill installed will have one hundred commit messages by the end of the month that all follow the same format, because the format is no longer carried by her attention but by the skill. Consistency, in this sense, is a separate benefit from speed, and it compounds in a different way. Speed saves time per commit. Consistency makes the entire history of commits more useful, because future readers can rely on the format being stable.

Two more anatomical details deserve attention before this chapter closes, because they come up often in real skill libraries and are worth knowing the first time the reader encounters them. The first detail concerns the relationship between the description and the body. A common confusion among new skill authors is the temptation to repeat in the body what the description already says. This is a waste of context, because the description

is already loaded into context whenever the skill is being considered. The body should pick up where the description leaves off. The description tells Claude when to fire. The body tells Claude what to do once it has fired. Treating the two as having distinct jobs prevents the kind of bloated `SKILL.md` file in which the first paragraph of the body is a restatement of the description.

The second detail concerns the cost of having many skills. A natural worry is that installing fifty skills will somehow degrade the performance of every Claude Code session. In practice, the cost is bounded by progressive disclosure. Fifty skills means fifty descriptions loaded at startup. A typical description is two or three sentences, perhaps thirty to fifty words. Fifty descriptions is therefore on the order of two thousand words, or roughly three thousand tokens. That is a real cost, but it is a small one compared to the size of a modern context window, and it is paid once at session start rather than on every prompt. The body of any specific skill is loaded only when that skill activates, and only one or two skills typically activate per turn. The cost of skill ownership scales gracefully. A library of fifty good skills is not noticeably slower than a library of five.

This is the anatomy of a skill, complete enough to begin building. A folder, named after the skill, in either the user or project skills directory. A `SKILL.md` file at the root of that folder, with frontmatter declaring at minimum a name and a description, optionally restricting tools, and a markdown body containing the instructions. Optional subfolders for scripts, references, and assets, picked up by Claude only when the body refers to them. A clear, automatic activation model that does not require the user to remember anything. With this anatomy in working memory, the reader is ready for the chapter that follows, in which a real skill is built from the empty folder forward, with every decision made explicit and every common mistake catalogued. The work begins now.

Writing Your First Skill: A Hands-On Build

“The shortest distance between a developer and a working system is the smallest example that ships.”

Folk wisdom of the open source world

This chapter is a build. By the end of it, the reader will have a working skill installed in her own Claude Code environment, will have used it in a real session, and will have debugged the most common failure mode skills exhibit. The build is small on purpose. The point is not the impressiveness of the example. The point is that every step of the path from empty directory to working skill becomes familiar, so that subsequent skills, larger and more sophisticated, can be built without having to relearn any of it.

The skill being built in this chapter is called commit-message. Its job is to take the staged changes in a git repository and produce a single commit message in the conventional commits format. This is a skill that almost every developer would benefit from, because almost every developer has, at some point, committed a change with a message that her future self struggled to interpret. The skill will fix this once and for all, by ensuring that every commit message Claude writes follows a consistent format, includes a meaningful summary, and surfaces any breaking changes prominently.

Before opening any files, do the architectural thinking. The decision is, first, where the skill should live. This is a skill that the developer will want available in every project, not only in one. Therefore the skill belongs in the personal skills directory, at `~/.claude/skills/`. The decision is, second, what the skill needs to be able to do. To inspect staged changes, the skill needs the ability to run shell commands, specifically `git diff`. To compose the message, the skill needs to be able to think about the diff. It does not need to write to any file directly, because the user will take the produced message and pass it to `git commit` herself. Therefore the `allowed-tools` field will include Bash but not Edit or Write.

The decision is, third, what the description should say. This is where novice skill authors most often fail, because the description has two jobs that pull in opposite directions. It must be specific enough that Claude only fires the skill when it genuinely applies, and it must be broad enough to fire on the variety of phrasings real users actually employ. A description that is too narrow will fail to fire on requests that should clearly trigger it. A description that is too broad will fire when it should not. The next chapter is devoted to the craft of description writing in depth, but for this first build, a reasonable working approach is to draft a description that lists the canonical request, then enumerates a few alternative phrasings the user might use, and finally states what kind of input the skill expects to be working from.

Setting Up the Folder

With these decisions made, the build can begin. Open a terminal. Create the folder.

```
mkdir -p ~/.claude/skills/commit-message
cd ~/.claude/skills/commit-message
```

Inside that folder, create a file named SKILL.md. The file will contain frontmatter, then a body. Open the file in any editor and type the following.

```
---
name: commit-message
description: Generates a conventional commits message from the currently staged changes in a git repository. Use whenever the user asks for a commit message, asks to "write a commit," asks "what should I commit this as," or otherwise indicates they have staged changes and want a message written. Always inspect the diff before writing the message.
allowed-tools: Bash, Read
---

# Commit Message

When asked to produce a commit message, follow this procedure exactly.

## Step 1: Inspect the staged changes

Run `git diff --cached` to see what is currently staged. If nothing is staged, run `git diff` to see unstaged changes and tell the user that no changes are staged but here is what would be committed if they were. Do not proceed to write a message until you have actually read the diff.
```


Step 2: Choose the type

The commit type is the first part of a conventional commit and must be one of the following:

- feat: a new feature visible to the user
- fix: a bug fix
- refactor: a change in code that does not alter behavior
- docs: documentation only
- test: tests only
- chore: build tooling, dependencies, configuration
- perf: a change focused on performance
- style: formatting only, no code change

Choose the most specific type that fits. If a single commit spans multiple types, choose the most user-visible one and mention the others in the body.

Step 3: Choose the scope

The scope is an optional one-word noun in parentheses indicating which area of the codebase changed. Examples: feat(auth), fix(api), refactor(parser). Use a scope when the diff is concentrated in a recognizable area. Omit it when changes are diffuse.

Step 4: Write the summary

The summary is the rest of the first line. Keep it under 72 characters. Use the imperative mood: "add user authentication," not "added user authentication." Do not capitalize. Do not end with a period.

Step 5: Write the body, if needed

If the change is non-trivial, add a blank line after the summary, then write a short paragraph explaining what changed and why. Keep it under five lines. Skip the body for trivial changes.

Step 6: Mark breaking changes

If the diff contains anything that breaks an existing API or changes default behavior, add a line at the bottom that begins with `BREAKING CHANGE:` followed by an explanation. This line is required for breaking changes and must not be omitted.

Step 7: Output

Output the final commit message in a single fenced code block, with no commentary before or after. The user will copy it directly into git commit.

Examples

A good message:

```

feat(auth): add password reset endpoint

Adds POST /auth/reset that accepts an email and sends a reset link.

Token expires in one hour. Tested with Postman.

```

A bad message that this skill must avoid:

```

fixed stuff

```

Save the file. The skill is now installed. To verify, restart Claude Code so that the new skill is discovered at the next startup. Open Claude Code in any git repository where some changes are staged. Type, in the most natural phrasing the developer can think of, something like give me a commit message for this. Claude should recognize that the commit-message skill applies, run `git diff` cached on its own, read the diff, and produce a commit message that follows the seven-step procedure laid out in the body. The whole interaction should feel as if the developer has briefed a careful collaborator, once, and the collaborator now silently applies the same procedure every time the request is made.

The first time a developer builds a skill, the most common failure mode is not what one might expect. The failure is not that the skill misbehaves. The failure is that the skill does not fire at all. The developer types a request that should clearly activate the skill, and Claude proceeds to write a commit message in some other format, ignoring the skill entirely. This failure has one cause in roughly nine cases out of ten, and the cause is the description.

Failure Modes and Fixes

Walk through what is happening when the failure occurs. At session start, Claude loaded the description of the commit-message skill into its context. When the developer typed her request, Claude scanned the descriptions of the available skills and assessed whether any of them matched the request. If the description does not contain language that resembles the developer's request closely enough, the assessment fails, and the skill does not fire. The skill is installed correctly. The body is written correctly. But the description

has not done its job, which is to bridge the gap between the developer's natural phrasing and the skill's identity.

When this happens, the fix is almost always to expand the description. Add the phrasings the developer actually uses. Add synonyms. Add the verbs and nouns from the developer's tone of voice. The description above already includes several alternatives: the canonical "asks for a commit message," and the alternatives "write a commit," "what should I commit this as," and "indicates they have staged changes and want a message written." These multiple framings are not redundant. They are the surface area of the skill, the places where natural language can land and still be recognized as a request for this skill.

A useful diagnostic exercise, when a skill fails to fire, is to ask Claude directly. Type into the session, why did the commit-message skill not fire on my last request? Claude will often produce a useful answer, naming the gap between the request and the description. That answer is then a precise instruction for what to add to the description. After making the change, restart the session for the change to take effect, and try again. Within two or three iterations, almost every triggering problem can be solved by description refinement.

There are two other failure modes worth naming, though both are less common than the triggering problem. The first is the over-firing problem, in which the description is so broad that the skill activates on requests that should not have triggered it. A commit-message skill whose description merely says "anything related to git" will fire when the user asks how to revert a commit, which is the wrong behavior. The fix is to narrow the description by adding negative phrasings or tighter scope language. The second failure mode is the body problem, in which the skill fires correctly but produces output that does not match what the body specifies. This is almost always a sign that the body is ambiguous in some way that the developer did not notice when writing it. The fix is to read the body as if reading it for the first time, find the place where instruction and output diverge, and tighten the language. Examples in the body, especially good and bad examples placed side by side, are extraordinarily effective for this.

The Skill in Real Use

The skill is now built, installed, and verified. The developer has, in roughly twenty minutes of focused work, installed a piece of automation that will pay her back every single time she commits code for the rest of her time using Claude Code. The investment is small. The dividend is daily, indefinite, and silent. There will be no celebration in the terminal when the hundredth commit message gets written by this skill. The developer will simply notice, weeks from now, that her commit history reads more cleanly than it used to, and that she stopped having to think about commit messages somewhere along the way. This is what skills, in aggregate, do to a developer's working life. Each one is small. The compounding is large.

A worked debugging walkthrough is worth including here, because the experience of debugging a skill is one of the moments that most reliably separates developers who go on to build serious skill libraries from those who give up after one or two attempts. The walkthrough is from a real session. The skill author had built a skill called `explain-error`, intended to produce structured debugging guidance from a stack trace. She tested it by pasting a Python traceback and asking, what does this mean. The skill did not fire. Claude produced a reasonable answer, but not the structured output the skill body specified. The author was confused, because the description seemed clear. She typed into the same session, why did the `explain-error` skill not fire on my last request. Claude responded with a short paragraph: the description mentions errors and stack traces, but the user's request used the word `means` rather than any of the canonical phrasings, and the description had no language that bridged the gap between asking what something means and asking for an error explanation.

This was a precise diagnosis. The author edited the description, adding the phrasing `what does this mean`, `what is going on here`, and `what is happening` to the trigger language. She restarted the session. She pasted the same traceback and asked, what does this mean. The skill fired. The output followed the structured format the body specified. The total time from first failure to working skill was roughly four minutes. The lesson the author took away was not about this specific skill. The lesson was about the general method: when a skill fails to fire, ask Claude why, edit the description in light of the answer, restart, and try again. This loop converges quickly, almost always within two or three iterations. Developers who

internalize the loop stop being intimidated by description writing, because the feedback cycle is fast enough that experimentation is cheap.

A second worked build will reinforce the procedure. The skill being built is called slack-update. Its job is to take a description of what the developer did today and produce a short, well-formatted Slack message suitable for posting to a team channel as an end-of-day update. This is a skill that fits a specific context but appears in almost every working developer's life: at four-thirty in the afternoon, the developer wants to summarize what she did today, in a format that fits the team's norms, with the right level of detail for an audience who was not in her head while she did the work.

Reflection on the Build

The architectural thinking, applied to this skill, runs like this. The skill belongs at the user level, because end-of-day updates are personal and follow the developer across projects. The skill needs the ability to read recent commits or recent file changes, because those are the raw material from which the update is composed; that argues for Bash and Read in allowed-tools. The skill does not need write access, because the developer will paste the result into Slack herself. The description should fire on phrasings like give me a Slack update, what should I post in standup, write my end-of-day message, and similar.

```
---
name: slack-update
description: Produces a short, well-formatted Slack message summarizing what the developer
accomplished today. Use whenever the user asks for "a Slack update," "an end-of-day message,"
"what should I post in standup," "a daily update," "summarize my day," or describes wanting to
share progress with the team. This skill is for outbound updates the developer is writing, not for
summarizing other people's messages.
allowed-tools: Bash, Read
---

# Slack Update

When asked for a Slack update, follow this procedure.

## Step 1: Gather material
```

If the user has described what they did today, use that as the primary material. If they have not, run ``git log --since="this morning" --author="$(git config user.email)"`` to surface today's commits, and ask the user to confirm or add any uncommitted work.

Step 2: Categorize

Group the work into at most three categories: shipped, in progress, and blocked. If a category is empty, omit it.

Step 3: Format

Produce the message in this format:

Today

- [bullet, one short line per shipped item]

In progress

- [bullet, one short line per in-progress item, with a note on what's next]

Blocked

- [bullet, what's blocked and what would unblock it]

Use Slack markdown: asterisks for bold, no headings, plain bullets. Keep each bullet under 100 characters. The whole message should fit in a single screen without scrolling.

Step 4: Tone

Match the developer's tone. If the work was routine, keep it factual. If something genuinely went well, allow a single short positive note. Do not invent enthusiasm or use phrases like "exciting progress" unless the developer has signalled that mood.

Do not include items the developer did not mention or that do not appear in the day's commits.

The slack-update skill, like the commit-message skill before it, took roughly ten minutes to write once the architectural decisions had been made. The body is longer because the task has more structure, but the discipline is the same: name the steps, specify the format down to the punctuation, and leave no ambiguity about tone or content. The skill, once installed, will compose end-of-day updates that fit a consistent shape across every working day, and the developer will stop spending the four minutes per day she used to spend on this small but real task.

There is a final point worth making about the experience of building skills, which only becomes visible after several have been built. The first

skill is intimidating. The second skill is easier. By the fifth skill, the act of building has become almost automatic: the developer thinks of a recurring friction, knows immediately that a skill could absorb it, and writes the skill in fifteen minutes. The barrier to building the next skill, after the first few, is no longer technical. It is a matter of recognizing the friction in the moment and choosing to invest fifteen minutes in eliminating it permanently. Most developers do not invest those fifteen minutes, because they are busy with whatever produced the friction in the first place. The developers who do invest those fifteen minutes accumulate, week by week, an environment that becomes increasingly tailored to the way they actually work, and the gap between their environment and the environment of a colleague who does not invest grows wider over time.

A small habit is worth recommending here. The reader who is just beginning her skill library should keep a running list, in a plain text file, of frictions she notices during her working day. The list does not need to be sorted, prioritized, or organized. It just needs to exist. At the end of each working week, she opens the list and picks one or two items to convert into skills over the weekend. The list keeps her honest about which frictions are real and recurring versus which are rare and one-off. The weekly review keeps the library growing at a sustainable pace, neither too fast nor too slow. After three months of this practice, most developers find that they have a library of fifteen to twenty skills, all of them addressing real frictions, all of them used regularly, and all of them earning their place. This is the steady-state pattern that produces lasting results, and it requires no more than thirty minutes per week of deliberate investment.

The next chapter takes the most important field in any skill, the description, and gives it the attention it deserves. The triggering problem encountered in this chapter has a craft, and the craft can be taught. The reader who masters the craft of writing descriptions will, from then on, build skills that fire reliably on the first try, and will spend her time on the body, where the substance of the skill actually lives. That is the subject ahead.

Skill Description Engineering: The Art of Triggering

“To name a thing accurately is to make it findable. To name it inaccurately is to lose it forever in plain sight.”

a librarian, anonymous

This is the most important chapter in Part Two. The body of a skill is what Claude does when the skill fires. The description is what determines whether the skill fires at all. A perfect body attached to a poor description is a skill that almost never runs. A modest body attached to an excellent description is a skill that runs reliably across the full range of natural phrasings real users employ. Between the two, the description matters more, because a skill that does not fire might as well not exist. This chapter is dedicated to the craft of description writing, and the goal is that by the end of it, the reader will produce descriptions that fire correctly on the first attempt, every time.

Begin with what a description actually does mechanically. At the start of a Claude Code session, the system loads the metadata of every available skill, and the description is the most substantive part of that metadata. When the user submits a prompt, Claude evaluates whether any of the loaded descriptions match the intent of the prompt. The matching is semantic, not literal. Claude is not running a regular expression over the description. It is reading the description in the same way it reads any other natural language, and forming a judgment about whether the prompt fits what the description is talking about. If the judgment is yes, the skill body is loaded into context, and the response is shaped by the body. If the judgment is no, the skill is ignored.

This mechanism has two implications that govern everything in this chapter. The first implication is that the description must be a piece of writing that informs Claude’s judgment. It is not a label. It is not a tag. It is a short paragraph that should help Claude decide, accurately, whether a given prompt is the kind of prompt the skill exists to handle. The second

implication is that the description is read alongside the descriptions of every other installed skill. Skills compete for attention. A description that distinguishes its skill clearly from neighboring skills will fire correctly. A description that overlaps with another skill's territory will produce uncertainty, and uncertainty produces inconsistent firing.

With those implications in mind, here is the working framework this chapter teaches. A good description has four parts, and they appear in roughly this order. The first part is the canonical statement of what the skill does. The second part is the trigger language: the kinds of requests, phrasings, or contexts that should activate the skill. The third part is the boundary language: what the skill is not for, where its territory ends. The fourth part is the input expectation: what the skill assumes the environment will look like when it runs. Not every description needs every part in equal measure, but every description benefits from at least considering each of the four.

Begin with the canonical statement. This is the one-sentence version of the skill, the version that would appear in a directory listing. It should answer the question, what does this skill do, in twelve to twenty words. Not all of the words. Just enough to make clear, to a reader who has never seen the skill before, what the skill exists to accomplish. The canonical statement of the commit-message skill from the previous chapter would be: generates a conventional commits message from the currently staged changes in a git repository. That is the skill, named accurately and concisely. Everything that follows refines it.

Trigger Language

The trigger language is the part of the description that does the heaviest lifting. It is where the skill author lists the actual phrasings real users employ when they want this skill to run. Trigger language must be derived from observation, not from imagination. The skill author should write trigger language by asking, what would I actually type when I wanted this skill to run, and what would my colleagues type, and what would a developer who has never used this skill before type if she wanted the same outcome. The variety of phrasings real humans employ is broader than any single author intuits. Useful trigger phrasings often include the canonical

request, several casual paraphrases, the verb form and the noun form of the request, and any common abbreviations.

description: Drafts a pull request description in the team's standard format. Use whenever the user asks for a PR description, pull request description, PR body, or asks "write a PR for this branch," "make me a PR description," "what should the PR say," or describes wanting to summarize a branch's changes for review. Always read the diff before writing.
description: PR description writer.

The first description fires reliably. The second description fires only when the user happens to use the exact words PR description writer, which almost no one does. The difference between the two is not a difference of quality. It is a difference of trigger surface area. The first description offers many places where natural language can land. The second offers one. A skill with one trigger surface and a perfect body is, in practice, a skill that almost never runs.

The boundary language matters when the skill operates near the territory of other skills. If the developer also has a skill called pr-review, which reviews other people's pull requests, then the pr-description skill needs language that distinguishes it. A line such as this skill is for writing the description of a PR you are creating, not for reviewing one that someone else wrote, prevents collision between the two. Without that line, an ambiguous request such as help me with this PR could activate either skill, and the developer's experience becomes unreliable.

The input expectation is the part of the description that prevents the skill from firing in contexts where it cannot succeed. A skill that operates on staged git changes, like the commit-message skill, does no good when there are no staged changes. The description benefits from a phrase such as assumes the user has staged changes in a git repository. This phrase informs Claude's judgment in two ways. First, it makes the skill more likely to fire when the user is clearly in the right context. Second, it makes the skill less likely to fire in contexts where the assumption obviously does not hold, such as a non-git directory or a fresh checkout with no changes.

There is one further technique worth introducing here, which comes from Anthropic's own documentation on skill authoring. Claude has a tendency to under-trigger skills, that is, to err on the side of not firing them when in doubt. The recommended counter-technique is to make

descriptions slightly assertive. A description that says use this skill whenever the user mentions X is more effective than a description that merely describes the topic of X. The verb use, applied to the user's phrasing rather than to the topic, gives Claude a clearer signal that the skill is meant to be activated rather than merely consulted. This is a small change with substantial consequences. Adding the phrase use whenever the user mentions to a description often raises its trigger reliability noticeably without affecting any other behavior.

Apply all of this to a worked example. The skill is called explain-error, and its job is to take an error message or stack trace and produce a clear, prioritized explanation, including likely cause and recommended next debugging step. A weak description for this skill might read, helps explain errors. That description is too short, has no trigger language, no boundary, no input expectation, and no use whenever phrasing. It will under-fire constantly. A strong description would read as follows.

description: Explains a given error message or stack trace and proposes the most likely cause and a concrete next debugging step. Use whenever the user pastes an error, stack trace, traceback, exception, or compiler output and asks "what is this," "why is this happening," "how do I fix this," or otherwise indicates they want help interpreting an error. Do not use this skill for general "my code is broken" requests where no error message is present; in those cases, ask the user for the error first.

A Worked Description

Notice the structure. The first sentence is the canonical statement. The middle of the description lists trigger language, including five concrete phrasings the user might employ. The end carries explicit boundary language: do not use this skill for, followed by the specific case where the skill should defer to a clarifying question instead. This kind of description fires correctly across the full natural variety of how developers actually report errors.

Two final practices round out the craft. The first is the trigger test. After writing a description, the skill author should ask herself whether she can describe the skill's task in three meaningfully different ways and have all three of them fire the skill. If yes, the description has good coverage. If no, the description should be expanded with the missing phrasings. This three-way test takes a minute to perform and prevents a category of bug that

would otherwise only surface weeks later, in a real session, at a moment when the developer is least equipped to debug it.

The second practice is the false-trigger test, which is the inverse. After writing a description, the skill author should think of two requests that look superficially similar to the skill's domain but should not fire the skill. If the description, as written, would clearly fire on those requests, the description is too broad and needs boundary language. The commit-message skill, for example, should not fire when the user asks how to revert a commit, even though the request mentions commits. A boundary line such as this skill is for writing new commit messages, not for git history operations, prevents the false trigger.

Description writing is, in the end, a discipline of imagining the user. The author who writes good descriptions is the author who has thought concretely about how real people will phrase requests in real moments. There is no formula that substitutes for this imagination, but the four-part framework, the use whenever phrasing, the trigger test, and the false-trigger test together produce descriptions that fire correctly on the first attempt in the large majority of cases.

A worked rewrite case study makes the framework concrete. The skill is called api-doc, and its job is to write OpenAPI documentation for a given route. The first draft of the description, written by a developer who had not yet read this chapter, looked like this.

description: Writes API documentation.

Iterating on a Failing Description

This description fired roughly one time in five. Reading it through the four-part framework explains why. The canonical statement is acceptable, but it is the only thing the description contains. There is no trigger language, no boundary, no input expectation. A developer who types document this endpoint, write OpenAPI for this route, or generate Swagger for this handler will find that none of those phrasings clearly match the description, because none of them are the words API documentation. The skill exists, but it does not fire when its time comes.

A second draft, after the developer noticed the under-firing problem and tried to fix it, looked like this.

description: Writes API documentation. Use when the user asks for documentation, OpenAPI, Swagger, or wants to document a route.

This is meaningfully better. The trigger language now includes three of the most common phrasings, and the verb use signals that the skill should be activated rather than merely consulted. Firing rate rose noticeably with this change. But the description still has problems. There is no boundary language, which means it will fire on requests like document this codebase, where the right skill might be the explain-codebase skill from Chapter Four, not api-doc. There is no input expectation, which means it will fire even when the user has not pointed Claude at any specific route. The result is occasional false triggers, where the skill fires on requests it cannot actually serve well.

The third draft, after the four-part framework was applied in full, looked like this.

description: Writes OpenAPI documentation for a single API route or handler. Use whenever the user asks to "document this endpoint," "document this route," "write OpenAPI for this," "add Swagger for this," "generate API docs for this handler," or otherwise indicates they want OpenAPI documentation produced for a specific route. This skill is for documenting a single route at a time, not for documenting an entire codebase. Assumes the user has identified the route or handler to document, either by pasting it or by referring to a specific file.

This description fires reliably and false-triggers rarely. The canonical statement is precise: a single API route, not a codebase. The trigger language enumerates five concrete phrasings the user might employ. The boundary language makes the single-route scope explicit and prevents collision with broader documentation skills. The input expectation establishes that the user has identified what to document, which prevents the skill from firing in contexts where it has nothing to work with. The total length is roughly seventy words, which is on the longer side for a description but well within the range that descriptions can usefully occupy. The trade between length and reliability falls clearly in favor of length here. A description three times longer than necessary that fires correctly is much better than a description one third the length that fires inconsistently.

The case study illustrates a general truth. Description quality is a continuous variable, not a binary. A first-draft description usually works some of the time. Each subsequent revision narrows the gap between when the skill should fire and when it does fire. After two or three revisions, the gap is small enough that the skill becomes reliable in daily use. The reader who is patient with this iteration will, over the course of building her first ten skills, internalize the four-part framework so deeply that her first drafts begin to look like other people's third drafts.

A Rubric for Description Quality

A small rubric is worth keeping in working memory while writing descriptions. A description scores well when it answers four questions clearly. What does this skill do, in one sentence. What kinds of phrasings should fire it. Where does this skill stop and another skill begin. What does this skill assume about the input. A description that does not answer any one of these four questions has a corresponding weakness, and the weakness will manifest in a corresponding failure mode. Under-firing usually traces back to weak trigger language. Over-firing usually traces back to weak boundary language. Firing in the wrong context usually traces back to weak input expectations. Firing on the wrong topic entirely usually traces back to a weak canonical statement.

One additional practice is worth introducing here, although its full development belongs in a more advanced treatment of skill design. Anthropic's own skill-creator skill includes an evaluation procedure in which the skill author writes twenty test queries, mixing prompts that should fire the skill with prompts that should not, and checks whether the description correctly distinguishes between them. The procedure is straightforward to perform manually. After writing a description, the author lists ten phrasings that should clearly fire the skill and ten that should clearly not. She reads each one and asks, would my description, as written, lead Claude to fire the skill on this query. If the answer disagrees with the intended firing in more than one or two cases out of twenty, the description needs work. If it agrees in all twenty cases, the description is probably ready.

This systematic check transforms description writing from a craft based on intuition into a craft based on evidence. The intuition is still valuable; it

suggests the initial language, the candidate phrasings, the likely failure modes. But the check ensures that the intuition has produced the right result. The author who runs the twenty-query check on every description she writes builds, over time, a sense of which kinds of phrasings tend to be dangerous, which kinds of boundaries tend to be necessary, and which kinds of input expectations tend to matter. The sense becomes intuitive, and her first-draft descriptions begin to pass the twenty-query check on the first attempt. This is the path to becoming a skilled skill author. There is no shortcut, but the path is short, and the reader who walks it deliberately reaches the destination within a working week.

Description writing pays out at every layer of the skill library. Better descriptions mean more reliable firing, which means the developer trusts the library, which means she invests in extending it, which means the library grows, which means the compounding accelerates. The single largest difference between a developer with a thriving skill library and a developer with a stalled one is, almost always, description quality. The body of the skill is the visible substance, but the description is the invisible gate that determines whether the body ever runs. Master the gate, and the rest follows.

One last observation is worth keeping in mind. Descriptions, like any other piece of writing, age. The phrasings a developer uses today are not necessarily the phrasings she uses in six months, because vocabulary drifts as projects change and as new tools enter daily use. A description that fired reliably six months ago may begin under-firing now, not because anything in the skill changed but because the developer has started phrasing her requests differently. The remedy is the same in both cases: add the new phrasings to the description. Treat descriptions as living documents, revisited whenever a skill seems to be firing less reliably than it used to. The cost of revising is small. The benefit is that the library stays in tune with the developer who depends on it.

With description engineering in hand, the rest of the work of building skills can focus on the body, where the substance of the skill lives. That work expands in the next chapter, which moves beyond the single-file skill into the world of bundled resources, scripts, and references that turn a skill from a paragraph of instructions into a substantial piece of installed capability.

Multi-File Skills and Bundled Resources

“A small thing well-organized is a tool. A large thing poorly organized is an obstacle. The line between them is layout.”

Edwin Bartholomew, woodworker

The skills built so far have been single-file skills. `SKILL.md` is the only file in the skill folder, and the entire skill, frontmatter and instructions and any examples, lives within that one document. This works perfectly for skills whose total content is small. For many skills, single-file is exactly the right structure, and adding more files would only complicate something that does not need complication.

But many of the most valuable skills cannot reasonably fit in a single file. A skill that scaffolds a new React component might need a dozen template files: one for the component itself, one for the test file, one for the Storybook story, one for the CSS module, one for the index file, one for each variant of the component the skill supports. A skill that runs a deterministic data extraction over a PDF might need to bundle the Python script that does the extraction, because asking the language model to do that work would be slow, expensive, and unreliable compared to running real code. A skill that encodes the conventions of a complex API might need a fifty-page reference document that the language model can consult when needed but should not load by default. None of these skills can be built well with a single file. All of them benefit from the multi-file structure that is the subject of this chapter.

The convention for multi-file skills, established by Anthropic and adopted across the open-source skills ecosystem, is straightforward. A skill folder may contain three special subfolders, each with a specific role. The `scripts` subfolder contains executable code. The `references` subfolder contains additional markdown documents that the `SKILL.md` body can ask Claude to read on demand. The `assets` subfolder contains files that are used as templates, fixtures, or other inputs during the skill’s execution. Any of the three subfolders may be present or absent in any given skill. Some skills

use all three. Most use one or two. The folder structure for a skill that uses all three would look like this.

```
react-component/  
├── SKILL.md  
├── scripts/  
│   └── generate.sh  
├── references/  
│   ├── component-conventions.md  
│   └── testing-conventions.md  
└── assets/  
    ├── component.template.tsx  
    ├── test.template.tsx  
    └── story.template.tsx
```

The naming of the three folders matters because it makes the skill folder predictable to read. A developer encountering the skill for the first time can look at the folder layout and form an immediate, accurate guess about what each piece does. Scripts are for running. References are for reading. Assets are for using as material in the skill's output. This predictability compounds across a skill library; once a developer has internalized the convention, every skill she encounters reveals its structure at a glance.

Begin with references, which is the most common form of bundled material. A reference is a markdown file that lives inside the skill folder and contains content that the skill body wants to be available, but that does not need to be loaded into context unless it is actually needed. The classic example is a long format specification. If a skill produces output in a complex format, the format specification might be ten pages. Putting the entire specification in `SKILL.md` would mean that every time the skill activates, ten pages of spec land in the context window, even if only a small section is relevant to the current task. The better pattern is to keep `SKILL.md` short, place the full specification in a reference file, and have `SKILL.md` instruct Claude to read the reference when relevant.

How Loading Works

The mechanics are simple. The skill body refers to the reference file by relative path, often within an instruction line. Claude, on encountering that instruction, knows that it can read the reference file using its standard file-

reading tools. Here is what the relevant portion of a `SKILL.md` file might look like, illustrating reference usage in practice.

```
---
name: api-doc
description: Writes OpenAPI documentation for a given route. Use whenever the user asks to
"document this endpoint," "write OpenAPI for this," "add Swagger docs," or otherwise indicates
they want API documentation generated.
---

# API Documentation

When asked to document an API endpoint, follow this procedure.

1. Read the route handler the user is referring to.
2. If you are unsure about the project's documentation conventions, read `references/openapi-
conventions.md` for the full style guide.
3. Read `references/error-format.md` only if the endpoint can return errors that are not the
standard 4xx or 5xx codes.
4. Produce the OpenAPI fragment in the format described in the conventions file.

Do not load the references unless the situation calls for them. The conventions file is long, and
loading it every time wastes context.
```

Several details in that example are worth highlighting, because they reflect the discipline of good multi-file design. First, the body is short. The body's job is to describe the procedure at the highest level. The detail lives in the references, where it does not cost context unless needed. Second, the body explicitly tells Claude when to load each reference. The instruction `Read X if Y` is the language pattern that triggers progressive disclosure correctly. Without that instruction, Claude might either always load the references, defeating their purpose, or never load them, leaving important information unused. Third, the body is honest about the cost of loading references. The closing line, `do not load the references unless the situation calls for them`, is a small but real piece of meta-instruction that helps Claude make the right call.

Now turn to scripts. Scripts are executable code bundled inside a skill folder, intended to be run by Claude as part of carrying out the skill. Scripts solve a category of problem that prose instructions handle poorly: deterministic, repeatable operations that real code can perform faster, cheaper, and more reliably than a language model can simulate. Sorting a

list, parsing a binary file, querying a database, calling an external API with a known schema. All of these are operations that should be code, not prose instructions to a model. The skill author writes the script once, places it in the scripts folder, and the skill body instructs Claude to run it at the appropriate moment.

A useful pattern is to combine a thin script with prose instructions about how to interpret its output. The script does the deterministic work. Claude reads the script's output and applies judgment to it. The combination is faster and more reliable than either approach alone. Anthropic's own document-handling skills follow this pattern. The pdf skill, for example, bundles a Python script that extracts form fields from a PDF without loading the PDF into the language model's context. The body of the skill then describes how to use the extracted data to fill or analyze the form. The script handles the part that requires precision. The body handles the part that requires understanding.

Assets, the third bundled resource type, are files used as material in the skill's output. Templates are the dominant case. A skill that scaffolds a new file from a template can keep the template as a separate file in the assets folder, and the body of the skill can instruct Claude to copy the template, fill in the variable parts, and write the result to the appropriate location in the project. This pattern is common in code-generation skills, document-generation skills, and any skill where the structure of the output is large and stable, but the specific values change with every invocation.

A Worked Multi-File Skill

A worked example brings the three resource types together. Consider a skill called scaffold-feature, intended to generate the scaffolding for a new feature in a TypeScript codebase. The scaffolding includes a route file, a service file, a repository file, a test file, and an OpenAPI fragment. The skill folder might contain templates for each of those file types in assets, a small Python script in scripts that runs the test scaffolding through a formatter, and a reference file in references that explains the codebase's naming conventions. The `SKILL.md` body would describe the procedure: ask the user for the feature name, read the conventions reference, copy each template from assets while substituting the feature name, run the formatter script over the result, and write the files into their proper locations. Each of the

three resource types is doing what it does best, and the combined skill produces a substantial scaffolding operation in seconds.

A practical limit is worth stating. `SKILL.md` should stay under approximately five hundred lines, even with bundled resources available. The body is read every time the skill fires, and a body that grows past five hundred lines starts to consume noticeable context on every activation. When a body approaches that limit, the right move is to factor surplus content into reference files. The body becomes a router that points Claude to the right reference at the right moment. This is, again, the progressive disclosure principle, applied recursively. The skill keeps its small footprint when it is not needed. It expands gradually as the work demands.

Token budgeting matters at a skill-library level too. A library of forty skills, each with a thirty-line description, a hundred-line body, and several hundred lines of references, might amount to ten thousand or more lines of total content. Loaded all at once, that would dominate any reasonable context window. Loaded progressively, with only metadata at startup and only the relevant body and references on activation, the total cost on any given turn is small. The progressive disclosure architecture is what allows skill libraries to scale without becoming a tax on every prompt. The discipline of building skills that respect this architecture is, in effect, the discipline of writing skills that play well with their neighbors.

A complete worked example of a multi-file skill will make the structure tangible. The skill being built here is called scaffold-react. Its job is to scaffold a new React component complete with its test file, its Storybook story, and an index export. The skill is exactly the kind that benefits from bundled resources. The templates for each generated file are stable and substantial, the conventions for component organization are project-specific and worth documenting, and the body itself can stay short by referring outward to the supporting files.

The folder structure for the skill looks like this.

```
scaffold-react/  
├── SKILL.md  
├── references/  
│   └── component-conventions.md  
├── assets/  
└── component.template.tsx
```

```
├── test.template.tsx
├── story.template.tsx
└── index.template.ts
```

Inside the Skill

The `SKILL.md` file is the entry point. It is short, because the heavy material lives in the references and assets folders. Its job is to coordinate: to fire on the right requests, gather the input it needs from the user, consult the conventions reference when relevant, and copy the appropriate templates with substitutions. Here is what the body looks like.

```
---
name: scaffold-react
description: Scaffolds a new React component, complete with test file, Storybook story, and index
export, in this codebase's conventional layout. Use whenever the user asks to "scaffold a
component," "create a new component," "set up a React component," "make a new component
called X," or describes wanting the standard files for a new component to be created.
allowed-tools: Read, Write, Glob
---
```

Scaffold React Component

When asked to scaffold a component, follow this procedure.

Step 1: Get the component name

If the user has stated a name, use it. Convert to PascalCase if needed. If the user has not stated a name, ask once for the name and the directory it should live in. Default directory is ``src/components/``.

Step 2: Read the conventions

Read ``references/component-conventions.md`` to confirm the current conventions for this codebase, including naming, prop typing, and CSS handling. Apply any conventions that are relevant to the templates.

Step 3: Generate the files

For each of the four template files in ``assets/``, copy it to the target directory and substitute the placeholder ``__COMPONENT_NAME__`` with the PascalCase name.

The four files produced are:

- ``<ComponentName>/index.ts` (from index.template.ts)`
- ``<ComponentName>/<ComponentName>.tsx` (from component.template.tsx)`
- ``<ComponentName>/<ComponentName>.test.tsx` (from test.template.tsx)`
- ``<ComponentName>/<ComponentName>.stories.tsx` (from story.template.tsx)`

Step 4: Confirm

After writing the files, list the four files created and offer to open the component file for editing.

Do not modify the templates themselves. Substitution happens at copy time. If the conventions in `references/component-conventions.md` conflict with what the templates produce, mention the conflict to the user and ask which to follow.

The `SKILL.md` is roughly forty lines. By itself, that is short. But the skill is not actually small. The total content of the skill, including templates and conventions reference, might run two or three hundred lines. The body is short because the body is doing what bodies do best, which is coordination. The substance lives in the supporting files.

The `component-conventions.md` reference might look like this.

React Component Conventions

This codebase follows these conventions for React components.

Naming

- Component files and folders are PascalCase.
- Hook files start with ``use`` and are camelCase.
- Test files match the component file with a ``test.tsx`` suffix.

Props

- Define props as a named interface above the component, e.g. ``interface ButtonProps``.
- Use destructured props in the function signature.
- Default values go in the destructuring, not in `defaultProps`.

Styling

- Use CSS modules. The CSS module file matches the component name with a ``module.css`` suffix.
- Avoid inline styles except for dynamic values.
- Class names are camelCase to match the CSS module convention.

Exports

- The component is the default export of its main file.
- The folder's index.ts re-exports the component as a named export so that imports can use either form.

Tests

- Use React Testing Library, not Enzyme.
- Test rendering, user interactions, and accessibility.
- Mock external dependencies with `vi.mock`` (or `jest.mock`` if the project uses Jest).

The conventions reference is fifty lines. Loaded into context, it costs roughly seven hundred tokens. If the body of `SKILL.md` included these conventions inline, every activation of the skill would pay that cost. With the conventions in a separate reference, the cost is paid only when the body explicitly asks for it, which is once per scaffolding operation rather than on every prompt where the skill is even considered. The savings compound across the entire skill library.

The templates in the assets folder are similarly self-contained. The component template might be twenty lines of TypeScript with the component name marked by a placeholder string. The test template might be fifteen lines with the same placeholder. Each template file is small in isolation, but together they add up to a substantial amount of generated code per scaffolding operation, and keeping them as separate files makes them easy to read, easy to edit, and easy to version-control.

Skill as Small Program

A property of this design that deserves attention: the skill is now a small program. The `SKILL.md` body is the controller. The conventions reference is the configuration. The asset templates are the data. This is the same architectural shape that has organized successful software systems for decades. The configuration is separate from the code. The data is separate from the logic. The components are independently meaningful and independently editable. A developer who needs to update the conventions edits one file. A developer who needs to add a fifth template adds one file. A developer who needs to change the procedure edits the body. None of these changes ripples into the others, because the boundaries between the components are clean.

This architectural cleanliness is not an accident, and it is not unique to this skill. The three-folder convention, scripts and references and assets, exists precisely because it produces this kind of clean separation. A skill that follows the convention is a skill that has been organized for editability, for review, and for evolution over time. Skills that ignore the convention can still work, but they tend to be harder to maintain, harder to share, and harder to extend, because the responsibilities are tangled together in ways that make any single change risk affecting things that should be independent of it.

A final note for this chapter concerns the path forward for any individual skill. Most skills do not start as multi-file skills. They start as single-file skills, often less than fifty lines long, written in fifteen minutes to address a specific friction. Over time, as the developer uses the skill and notices its rough edges, the skill grows. Some of the growth fits naturally inside SKILL.md. Some of the growth pushes outward, into reference files when the developer wants to add long-form documentation, into asset files when the developer wants templates, into scripts when the developer wants to delegate deterministic work to real code. The multi-file structure is the destination, not the starting point. Skills that grow toward it gradually, in response to actual need, end up with structures that fit their actual use. Skills that begin life as elaborate multi-file constructions before any real use has been made of them often turn out to have been over-engineered, with bundled resources that nobody ever loads and templates that never get applied to real work. The discipline is to start simple, use the skill, and add complexity only when the skill itself is asking for it.

A useful test for whether a skill is ready to expand into multiple files is the printout test. Print the SKILL.md body on paper. Read it through. If, while reading, the eye keeps wanting to skip past large chunks of detail because those chunks are not relevant to the immediate task, those chunks are candidates for relocation into reference files. If, while reading, the body begins to feel less like a procedure and more like a manual, the body has grown too large and needs to be split. Conversely, if the body fits comfortably on a single printed page and reads as a clean procedure from beginning to end, the skill does not yet need to expand. The printout test is informal but reliable, and it spares the developer from the trap of expanding skills before they have earned the right to expand.

The chapter has covered the structural facts. The next chapter takes the natural next step. With the ability to build skills that span multiple files, the developer has the raw material for a library. The question becomes how skills behave when more than one of them might apply to a given request, how they compose with each other when their work overlaps, and how to design libraries in which skills cooperate rather than collide. Composition is what turns a collection of skills into a system, and it is the subject of the chapter ahead.

Skill Composition and Collision: When Skills Work Together

“The library is more than the sum of the books on its shelves only when the shelves themselves have been arranged with thought.”

Jorge Luis Borges, paraphrased

A single skill, well written, improves a developer’s daily life. A library of skills, well composed, transforms it. The transformation does not happen automatically. Skills do not naturally cooperate. Left to their own devices, skills can also collide: two skills that both think they apply to the same request, with overlapping descriptions, produce inconsistent firing and unpredictable behavior. The discipline of skill composition is the discipline of designing libraries in which the skills clearly belong to their own territories, hand off cleanly when work crosses a boundary, and amplify each other when more than one is needed for a single task. This chapter is about that discipline.

Begin with how Claude actually decides which skill to fire. When a user submits a prompt, Claude evaluates the prompt against the descriptions of every available skill. Each description is a piece of evidence about whether the corresponding skill is relevant. Claude integrates the evidence and makes a judgment. In the simplest case, only one skill’s description matches the prompt, and that skill fires. In more interesting cases, two or more skills’ descriptions match, and Claude must choose. The choice is influenced by how confidently each description seems to match, by how specific each description is, and by any boundary language the descriptions contain. When two descriptions are equally plausible, the firing becomes a judgment call that may go either way on different sessions, which is the practical face of skill collision.

Collision is bad not because the wrong skill fires, although that is annoying. Collision is bad because the developer cannot rely on which skill will fire, and reliability is the entire point of installing skills in the first place. A skill library in which firing is reliable is a library that the developer

can trust. A library in which firing is uncertain is a library the developer must verify on every use, which negates the value of having installed the skills at all.

Three patterns produce most collisions in real skill libraries. The first pattern is overlapping topic language. Two skills mention the same topic in their descriptions, and Claude cannot tell which one a given request belongs to. A pr-description skill and a pr-review skill both mention pull requests, and a vague request such as help me with this PR could plausibly fire either one. The fix is boundary language: each description should explicitly state what its skill is and is not for. The pr-description skill says, this skill is for writing the description of a PR you are creating, not for reviewing one. The pr-review skill says, this skill is for reviewing PRs that someone else wrote, not for drafting your own. Each description now contains both an inclusion criterion and an exclusion criterion, and the collision evaporates.

The second pattern is mismatched specificity. One skill has a very specific description; another has a very general description that happens to overlap with it. The general one tends to win in cases where the specific one should have. The fix is to either make the general skill more specific by adding boundary language, or to recognize that the general skill is poorly designed and should be split into several more focused skills. A general skill called code-quality that fires on anything related to code quality is almost always a mistake. It would compete with a refactor-helper skill, a code-review skill, a static-analysis skill, and several others that overlap with parts of its territory. The right response is usually to dissolve the general skill into the specific ones, each of which has a clear description.

Naming Drift

The third pattern is naming drift. The skill author writes the description in one vocabulary and the body in another. The user, naturally, employs vocabulary that matches the body more than the description, because the body is closer to the actual work. The skill under-fires because the description does not contain the words the user actually uses. The fix is to bring the description into alignment with the working vocabulary of the body. A useful exercise is to read the body of the skill, list the ten words that appear most frequently and seem most central to what the skill does,

and check that those words appear in the description. If they do not, the description should be rewritten to include them.

With the failure modes named, turn to composition itself. Composition is the positive case: two or more skills cooperating on a single request to produce something better than either could produce alone. Several patterns of composition are worth knowing.

The first composition pattern is sequential composition. Two skills fire in sequence on the same request, with the output of the first becoming the context for the second. A common case: the user asks for a refactoring plus a commit. The refactor-helper skill fires first, executes the refactor, and produces a description of what changed. The commit-message skill fires next, reads the changes, and produces a commit message that accurately reflects them. Sequential composition often happens implicitly: the developer asks Claude to do two things in one prompt, and Claude activates both relevant skills as it works through the prompt. The skill author can encourage this by writing skill bodies that conclude with a clean handoff. The refactor-helper skill, after completing its work, might end its body with a line such as if the user wants a commit message for these changes, the commit-message skill is appropriate. Claude reads that hint and is more likely to fire the second skill at the right moment.

The second composition pattern is parallel composition. Two or more skills fire on the same request and contribute different perspectives to the same output. A code-review request might activate a security-focused skill, a performance-focused skill, and a style-focused skill simultaneously, with the final review weaving together all three perspectives. Parallel composition is more common when the request is broad and the skills are narrow, because each narrow skill contributes its own piece of the broad answer. The skill author encourages parallel composition by writing skills that produce well-bounded, additive output rather than total reports. A security-focused review skill that produces a self-contained section labeled Security and stops there composes well with other focused skills. A security-focused review skill that produces an entire end-to-end review report does not, because its output overlaps with what the other skills want to produce.

The third composition pattern is the meta-skill. A meta-skill is a skill whose job is to orchestrate other skills. It has a description that fires on broad requests, and its body contains instructions that direct Claude to invoke specific other skills in a specific order. Meta-skills are useful for workflows that recur often enough to deserve a single trigger but that involve several distinct steps. A release-prep meta-skill might fire on a request to prepare a release and direct Claude to invoke a changelog-generator skill, a version-bump skill, a release-notes skill, and a tag-and-push skill, in sequence. The meta-skill is a small orchestrator, and the actual work is done by the focused skills it directs.

A Practical Guideline

A practical guideline ties the patterns together. Build narrow skills first. A library of twenty narrow, focused skills is more useful than a library of five general ones. Narrow skills fire reliably because their descriptions can be precise. Narrow skills compose because they have well-bounded outputs. Narrow skills are easier to debug because their failure modes are concentrated. The temptation to build broad skills usually comes from a desire to reduce the number of skills in the library, but this is the wrong objective. The right objective is to maximize reliability and composability, both of which point toward narrow skills.

A second guideline concerns the boundary between skills and the body of `CLAUDE.md`. Some knowledge belongs in `CLAUDE.md`, where it is always loaded. Some knowledge belongs in a skill, where it is loaded on demand. The dividing line is roughly: knowledge that is true at all times in this project belongs in `CLAUDE.md`; knowledge that is true only when a specific kind of work is being done belongs in a skill. The rule sounds simple, and in practice it requires some judgment, but applying it consistently produces a Claude Code environment where ambient knowledge is always present, targeted knowledge is loaded only when relevant, and the boundary between the two is principled rather than arbitrary.

There is one more compositional possibility worth naming, although its full treatment belongs in Part Three. Skills can be invoked by subagents as well as by the main session. A subagent dispatched for a specific kind of work inherits the skill library available in the project and can activate any

skill whose description matches the work the subagent is doing. This is a powerful pattern, because it lets the skill library and the subagent library reinforce each other. A test-writer subagent that has access to a project's test-conventions skill is more reliable than either the subagent or the skill alone. The full implications of this composition will be developed in Chapter Fourteen, after subagents have been introduced. For now, it is enough to know that the skills built in this part of the book will continue to do work even after the boundaries of skills as a topic are crossed.

This chapter has dealt with the politics of a skill library: how skills relate to each other, how they collide and how they cooperate, and what the author can do to encourage the second over the first.

A practical exercise will reinforce the principles. Imagine a developer who, over six months of casual skill building, has accumulated a library of fifteen skills. She has noticed that some of them seem to fire reliably and others fire inconsistently. She decides to audit the library against the principles in this chapter.

The Audit Procedure

The audit procedure is straightforward. She lists the fifteen descriptions side by side. She reads each one and scores it against the four-part framework from Chapter Six: canonical statement, trigger language, boundary language, input expectation. She also reads each pair of descriptions and asks whether either pair could plausibly fire on the same request. Of the fifteen skills, three pairs jump out as potential collisions. A pr-description skill and a pr-review skill, both mentioning pull requests. A code-explain skill and an explain-codebase skill, both mentioning code explanation. A bug-report skill and a debug-helper skill, both mentioning bugs.

Each pair gets resolved differently. The first pair is the cleanest case. Each skill clearly has a different job, but neither description names the difference explicitly. She adds a single sentence of boundary language to each. The pr-description description gains the line, this skill is for writing the description of a PR you are creating, not for reviewing one. The pr-review description gains the symmetrical line, this skill is for reviewing PRs that someone else wrote, not for drafting your own. The two skills now have non-overlapping territories. Subsequent firing becomes reliable.

The second pair is more interesting. Reading them carefully, she realizes that code-explain and explain-codebase are not really separate skills. Code-explain operates on a piece of code that has been pasted or pointed to. Explain-codebase operates on a whole repository. The original intent was that they would handle different scales of explanation. But the descriptions have not made the scale distinction clear, and in practice users have been triggering whichever description Claude happened to match first, regardless of which scale the user actually wanted. She rewrites both descriptions to make the scale explicit. Code-explain says, explains a single function, file, or pasted snippet. Explain-codebase says, produces a high-level architectural summary of an entire codebase. Now the descriptions are not just non-overlapping; they are explicitly complementary. A user who pastes a function gets code-explain. A user who asks for a tour of the repo gets explain-codebase. The collision evaporates.

The third pair turns out to be a deeper problem. The bug-report skill formats a structured bug report. The debug-helper skill walks the user through diagnosing a bug. Both fire on requests that mention bugs. After looking at how she actually used the two skills over the past month, she realizes that debug-helper has fired roughly four times, all of them in cases where bug-report would also have been a reasonable choice, and none of those four sessions produced output she actually used. Debug-helper, in retrospect, was an attempt to build a skill that did too much: a generic debugging companion. Its description was broad because its job was broad, and broad jobs do not fit the skill model well. She decides to retire debug-helper and replace it, eventually, with two or three narrower skills that cover specific debugging scenarios. For now, removing it cleans up the library and removes the collision with bug-report. The library shrinks from fifteen skills to fourteen, and quality goes up rather than down.

This kind of audit is worth performing once a quarter on any skill library that has grown past ten skills. The principles are simple: list the descriptions, score them against the four-part framework, look for collisions, and either resolve the collisions through boundary language, refactor through clearer scope language, or retire skills that have proven to be too broad to do their job well. A library that has been audited recently is a library that the developer can trust, and trust is what makes the library worth using.

Composition Across Domains

A second concrete scenario, which surfaces a different design pattern, involves a developer building a skill library for a team. The team has a shared `.claude/skills/` directory in their project, and three developers are contributing to it. Each developer has, naturally, drafted a few skills in her own style. The result, after a month, is a library that is technically functional but stylistically inconsistent. Some descriptions follow the four-part framework. Others are one-liners. Some bodies use the step-numbered procedure pattern. Others use prose. Some skills assume the user will provide context. Others have built-in steps to gather it.

The team decides to standardize. They sit down together for thirty minutes and agree on a small set of conventions. Every description must include canonical, trigger, boundary, and input expectation language. Every body must use a step-numbered procedure for any skill whose work is sequential. Every skill that needs context must include a step-one section that gathers that context, rather than relying on the user to provide it preemptively. The conventions are written into a short `README.md` at the root of the `.claude/skills/` directory, so that any future contributor encounters them at the moment she opens the directory for the first time.

The conventions take effect immediately for new skills, and existing skills are gradually retrofitted as they need attention for other reasons. Within a month, the library has a coherent house style. Skills feel like they belong together. New contributors can write a new skill in fifteen minutes because the conventions answer most of the design questions before the contributor even asks them. The library has become, in the genuine sense of the word, a library, with a unifying organizing principle, rather than a heap of files in a folder.

A final thought on composition before this chapter ends. The most powerful skill libraries in the wild, the ones that earn the title library rather than mere collection, share a property that is worth naming. They are designed for the user's actual workflow, not for the developer's sense of what skills should exist. A skill that addresses a friction the developer has experienced personally, in real work, will almost always fit cleanly into a library. A skill that the developer built because she imagined someone might want it, without having needed it herself, will almost always feel out

of place. The audit principle that follows from this is to be ruthless about skills that have not been used. A skill that has not fired in three months is a skill that does not belong in the library, even if its description is technically excellent. The cost of keeping it is small but real, and the noise it adds to the firing decision marginally degrades every other skill that competes for the same attention.

A pattern that separates great libraries from merely good ones is the deliberate cultivation of trigger overlap with the developer's natural vocabulary. The developer who pays attention notices, after a few months, that she has habits of phrasing that recur. She tends to say make me a, when she wants something generated. She tends to say walk me through, when she wants something explained. She tends to say tighten this up, when she wants something refined. Each of these recurring phrasings is a hook, and skills whose descriptions explicitly include the developer's recurring phrasings fire more reliably than skills whose descriptions use generic alternatives. Building this kind of personal-vocabulary alignment into descriptions is a small practice, but it converts a library from one that fires correctly to one that fires effortlessly. The library starts to feel less like a tool the developer is using and more like an extension of her own way of speaking. This is the felt sense of a mature library, and it cannot be achieved on the first day. It is the gradual product of attention paid to the developer's own habits, applied iteratively across descriptions over time.

The next chapter shifts from politics to inventory. With the principles of composition in hand, the reader will build a personal library of ten skills that are immediately useful in daily work, with each skill written, debugged, and ready to install. The chapter ahead is the most practical in the book. By its end, the reader's Claude Code environment will be measurably more capable than it was at the start of Part Two, and the value of the principles taught in the chapters before it will be visible in every session that follows.

The Personal Skill Library: Your First Ten Skills

“A library is not a collection of books. A library is a set of decisions about which books to keep at hand.”

Umberto Eco, paraphrased

This chapter is the most practical in Part Two. By the time the reader finishes it, ten new skills will be installed in her Claude Code environment, each of them ready for immediate use, each of them solving a specific recurring friction in daily development work. The skills are presented in a uniform format. Each one begins with the problem it solves, then shows the complete `SKILL.md` file ready to copy, then explains the design choices that shaped it, then notes the most common pitfall and the recommended way to extend the skill once the developer has lived with it for a while.

The selection of ten skills is deliberate. Each skill represents a category of friction common to almost every developer using Claude Code. The categories are: composing structured documents from raw material, generating standardized scaffolding, performing focused review, capturing knowledge from work that just happened, formatting and conforming to project conventions, and coordinating between code and surrounding artifacts such as commits and pull requests. The reader who installs all ten will have her core daily workflow substantially automated. The reader who installs even three or four will feel the difference within a working week.

Before beginning the build, a small piece of housekeeping. All ten skills in this chapter are intended to be personal skills, installed at the user level, and therefore live in `~/.claude/skills/`. Each one needs its own folder, named to match the skill. The folder structure for the full library will look like this when complete.

```
~/.claude/skills/  
├── commit-message/  
├── pr-description/  
├── humanize-text/  
├── refactor-readability/  
└── jest-test/
```

```
|— pr-review/  
|— code-explain/  
|— bug-report/  
|— decision-log/  
|— changelog-entry/
```

Setting Up Each Skill

Each of the ten skills below requires the same setup: create the named folder, create a `SKILL.md` file inside it, paste the contents shown. After installing all ten, restart Claude Code so that the new skills are discovered. From that point forward, the skills will be available in every session.

The first skill is `commit-message`. It was built in detail in Chapter Five and serves as the canonical reference for what a small skill looks like. Its job is to produce conventional commit messages from staged changes. If the reader has not already installed it, the `SKILL.md` file from Chapter Five can be used directly.

The second skill is `pr-description`, used as an example in earlier chapters. It produces pull request descriptions in a consistent format.

```
---  
name: pr-description  
description: Drafts a pull request description in the team's standard format. Use whenever the user  
asks for a PR description, pull request description, PR body, or asks "write a PR for this branch,"  
"make me a PR description," "what should the PR say." This skill is for drafting the description of  
a PR you are creating, not for reviewing one someone else wrote.  
allowed-tools: Bash, Read, Grep  
---  
  
# PR Description  
  
When asked for a PR description, follow this procedure.  
  
1. Identify the diff. If the user mentions a branch, run `git log` and `git diff` against the merge  
base. If no branch is mentioned, use the current branch against main.  
2. Read the diff in full.  
3. Produce the description in this exact format:  
  
## Summary  
A single sentence in the imperative mood describing the change.
```

What changed

A bulleted list of concrete changes, one bullet per logical unit.

Testing

A short paragraph describing how the change was tested.

Risk

One sentence categorizing risk as low, medium, or high with a one-line reason.

Do not invent changes that do not appear in the diff. If the diff is large, summarize at the file level rather than the line level.

The third skill is humanize-text. Its job is to take written content that sounds machine-generated, formal, or stilted, and to rewrite it in a more natural human voice. This skill is especially valuable for developers who write documentation, blog posts, or release notes alongside their code, and who have noticed that AI-generated drafts have a recognizable tone that benefits from editing.

```
---
name: humanize-text
description: Rewrites text to sound more human, less formal, and less machine-generated. Use
whenever the user asks to "humanize this," "make this less AI-sounding," "rewrite in a more
natural voice," "make this less formal," or pastes a draft and asks for a more natural rewrite.
allowed-tools: Read
---
```

Humanize Text

When asked to humanize a piece of text, apply these transformations.

- Remove em-dashes. Replace with commas, periods, or sentence breaks.
- Remove or replace AI-flavored vocabulary: "delve," "leverage," "robust," "navigate," "in conclusion," "it is important to note."
- Vary sentence length deliberately. Short sentences punctuate. Long sentences explore. Avoid runs of medium-length sentences.
- Cut adverbs that do not earn their place: "really," "very," "quite," "extremely."
- Remove rule-of-three constructions. If a sentence lists three things in parallel for rhythm rather than meaning, replace with one or two.
- Cut filler openers: "It is worth noting that," "It should be mentioned that."
- Use contractions where the surrounding tone permits them.
- Prefer concrete nouns over abstract ones.

Return the rewritten text in a fenced code block. After the block, list the most significant changes you made, no more than five bullets.

The Refactor Skill

The fourth skill is refactor-readability. Its job is to take a function or section of code that works correctly but reads poorly, and propose a refactor focused on clarity rather than functionality.

```
---
name: refactor-readability
description: Proposes a readability-focused refactor of a given function or section of code,
without changing its behavior. Use whenever the user asks to "make this more readable,"
"refactor this for clarity," "this is hard to read," "clean this up," or pastes code and asks for a
structural improvement that preserves behavior.
allowed-tools: Read, Edit
---

# Refactor for Readability

When asked to refactor code for readability, follow this procedure.

1. Read the code in full.
2. Identify the top three readability problems. Common candidates: deeply nested conditionals,
names that do not communicate intent, single functions doing multiple things, magic numbers
without explanation, repeated logic that could be factored, comments compensating for unclear
code rather than explaining intent.
3. Propose a refactor that addresses those three problems while preserving behavior exactly.
4. Show the refactor as a complete replacement of the original, not as a diff.
5. After the code, list the three problems addressed in plain language.

Do not change behavior. If you notice a bug in the original code, mention it separately, do not
silently fix it as part of the refactor.
```

The fifth skill is jest-test. Its job is to write a Jest or Vitest test file for a given function or module, following sensible defaults for both frameworks.

```
---
name: jest-test
description: Writes a test file in Jest or Vitest for a given function or module. Use whenever the
user asks to "write tests for this," "add unit tests," "test this function," "give me Jest tests," "give
me Vitest tests," or otherwise indicates they want tests authored for a piece of code.
allowed-tools: Read, Write
---

# Jest or Vitest Test
```

When asked to write tests, follow this procedure.

1. Read the file under test.
2. Detect the test framework. If the project uses `vitest` (check package.json), use Vitest. Otherwise default to Jest.
3. Identify the public surface of the file: exported functions and classes.
4. For each exported function, write tests that cover at minimum: the happy path, one boundary condition, and one error condition.
5. Use `describe` blocks per function and `it` blocks per scenario.
6. Mock external dependencies. Do not make real network calls or write to real files in tests.
7. Place the test file alongside the file under test, with the suffix `.test.ts` (or `.test.js` if the project is JavaScript).

Output the complete test file. Do not abbreviate.

The sixth skill is pr-review. Its job is to produce a substantive review of someone else's pull request, complete with concrete suggestions and a summary judgment.

```
---
name: pr-review
description: Reviews someone else's pull request and produces a structured review with
comments, suggestions, and a summary judgment. Use whenever the user asks to "review this
PR," "give me a PR review," "look at this pull request," "what do you think of this change." This
skill is for reviewing other people's PRs, not for writing the description of your own.
allowed-tools: Bash, Read, Grep
---
```

PR Review

When asked to review a PR, follow this procedure.

1. Identify the PR or diff to review. If the user provides a branch name, diff against the merge base.
2. Read the diff in full before forming any opinions.
3. Group your observations into four sections:

Substantive comments

Issues that affect correctness, security, or maintainability. Each comment should reference a file and line and explain the issue and suggested change.

Suggestions

Improvements that are not strictly required but would make the change better. Same format as substantive comments.

Questions

Things that are unclear from the diff and would require clarification from the author.

Summary

A two- or three-sentence judgment. Approve, request changes, or defer with reasons.

Do not invent issues. If the change is good, say so plainly. A short positive review is better than a long padded one.

The seventh skill is code-explain. Its job is to explain a piece of code at a level appropriate to a colleague who knows the language but does not know this codebase. This skill is especially useful when joining a new project, when reading unfamiliar parts of a familiar project, or when preparing notes to share with teammates.

```
---
name: code-explain
description: Explains a piece of code at the level of a colleague who knows the language but not
this codebase. Use whenever the user asks "what does this do," "explain this code," "walk me
through this," "I don't understand this function," "help me read this," or pastes code and asks for
an explanation.
allowed-tools: Read
---
```

Code Explanation

When asked to explain code, follow this procedure.

1. Read the code in full before writing anything.
2. Identify the purpose of the code in one sentence.
3. Identify the inputs, the outputs, and any side effects.
4. Explain the algorithm or pattern at a high level. Use plain language. Avoid restating the code line by line.
5. Note any non-obvious choices the author made and what alternatives they might have considered.
6. End with a short list of questions you would ask the original author if you could.

Do not pad the explanation with general background. Assume the reader knows the language. Focus on what is specific to this code.

The Bug Report Skill

The eighth skill is bug-report. Its job is to take a half-articulated description of a problem and produce a structured, actionable bug report that a developer or team could file directly into a tracker.

```
---
name: bug-report
description: Turns an informal description of a problem into a structured bug report ready to file
in an issue tracker. Use whenever the user describes a bug, says "something is broken," "this isn't
working," "X is happening when Y should happen," or asks to "file a bug" or "write this up as an
issue."
allowed-tools: Read
---
```

Bug Report

When asked to write a bug report, follow this procedure.

1. Identify what the user has told you and what is missing. If essential information is missing (steps to reproduce, expected vs. actual behavior, environment), ask one consolidated question to fill the gaps. Do not ask more than once.
2. Produce the report in this format:

Summary

A single sentence describing the bug.

Steps to reproduce

A numbered list, precise enough that another person could follow them.

Expected behavior

What should happen.

Actual behavior

What happens instead.

Environment

Relevant version numbers, browsers, operating systems, or configurations.

Notes

Anything unusual, intermittent, or worth investigating.

Stay close to what the user actually said. Do not invent symptoms or steps.

The ninth skill is decision-log. Its job is to capture a technical decision the developer just made, in the standard format used for architecture decision records, so that future developers (including the future self) can understand both the decision and its context.

```
name: decision-log
description: Records a technical decision in the architecture decision record (ADR) format. Use
whenever the user says "we decided to," "I'm going with," "log this decision," "write this up as an
ADR," or describes a choice made between alternatives that should be remembered.
allowed-tools: Read, Write
---
```

Decision Log Entry

When asked to record a decision, follow this format.

ADR-NNN: [Short title in the form of a noun phrase]

Status

Proposed, Accepted, Superseded, or Deprecated.

Context

What is the situation that requires a decision? Two or three sentences.

Decision

What was decided? One or two sentences in the active voice.

Consequences

What changes as a result? Both positive and negative consequences. A short bulleted list is acceptable here.

Alternatives considered

What else was on the table, and why were those alternatives rejected? One or two sentences per alternative.

If the user has not specified an ADR number, ask once whether to assign the next sequential number based on existing ADRs in the repo. If there are no existing ADRs, propose ADR-001.

The tenth and final skill is changelog-entry. Its job is to write a single entry suitable for inclusion in a CHANGELOG.md file, given a recent change or set of changes.

```
---
name: changelog-entry
description: Writes a single entry for a project's CHANGELOG.md based on a recent change. Use
whenever the user asks to "add a changelog entry," "write a changelog line for this," "what should
I put in the changelog," or describes a change and asks for the user-facing summary.
allowed-tools: Bash, Read
---
```

Changelog Entry

When asked for a changelog entry, follow this procedure.

1. If the change is described in conversation, use that. If a commit or branch is referenced, read the diff to understand the change.
2. Categorize the change as one of: Added, Changed, Fixed, Removed, Deprecated, Security.
3. Write a single line, in the past tense, written for users rather than developers. Focus on what is now different from the user's perspective, not on internal implementation details.
4. Output the entry in this format:

[Category]

- [Single-line description in past tense]

If the change has no user-visible impact, say so plainly and recommend not adding a changelog entry.

The library is now ten skills strong. The reader who has typed all ten into her `~/claude/skills/` directory has, in roughly an hour of work, installed a set of capabilities that will pay her back many times over in every future Claude Code session. Each skill is small. None of them, alone, is impressive. But every one of them removes a small recurring friction, and the cumulative effect of ten removed frictions is substantial.

Extending the Library

A note on extension. The skills above are starting points, not endpoints. As the developer lives with them, she will notice that some of them fit her work perfectly and others fit imperfectly. The imperfect ones should be edited. A skill is a file. Editing a skill takes thirty seconds. A description that under-fires can have phrasings added. A body that produces output the developer does not like can have its instructions tightened. The library is meant to evolve. The reader who treats her `~/claude/skills/` directory as a living project, revisited monthly and improved gradually, will find that within a year she has a personal toolkit that no off-the-shelf product could replicate, because it has been shaped by her actual work and her actual preferences.

A second note on sharing. Several of the skills above will be useful to other developers in the reader's team or company. The mechanism for sharing is straightforward: copy the skill folder into the project's `.claude/skills/` directory and commit it. From that point forward, every

developer who clones the project receives the skill automatically and benefits from it without any onboarding effort. This is how skill libraries spread within organizations. Not through documentation. Not through training sessions. Through configuration that travels with the code, and that the tool itself honors as soon as a colleague's session starts.

A third note, on the deeper effect of having lived with a skill library for a few months. Developers who install ten skills and use them daily report a subtle shift in how they think about their own work. Tasks that used to feel like obligations, things they had to remember to do correctly, start to feel like artifacts that the environment produces on their behalf. The mental energy that used to be spent on remembering the commit format, on remembering the PR template, on remembering to run the right tests, gets freed up. That freed energy has somewhere to go, and where it goes, in most cases, is into the actual work the developer is being paid to do. The library does not merely save time. It frees attention. The attention is the more valuable of the two, and developers who internalize this distinction often find that within a few months they are doing work that would have been impossible for them at the start, not because they got smarter but because the environment got better at carrying everything that did not require their attention.

Part Two ends here. The reader has, over six chapters, learned the anatomy of skills, built her first one by hand, mastered the description craft that determines whether skills fire correctly, learned to extend skills across multiple files when complexity demands it, internalized the principles that turn a collection of skills into a coherent library, and installed ten skills that will carry forward into every session ahead. Skills, the first pillar of the three-pillar framework, are now in working memory and in installed form. The next part of the book turns to the second pillar. Subagents are the parallel specialists that handle work which deserves its own context, its own role, and its own tool restrictions. The combination of skills and subagents is where the real architectural power of Claude Code begins to surface, and that surfacing begins in the chapter ahead.

PART THREE

Subagents: Claude's Parallel Specialist System

One mind, many specialists. The work that does not need to share thinking should not have to share context.

CHAPTER TEN

Subagents Fundamentals: Why Single-Threaded Claude Isn't Enough

“Specialization is the natural answer to the limits of attention. The mind that tries to hold everything ends up holding nothing well.”

Skills, the subject of Part Two, taught Claude how to do specific kinds of work consistently. They are an answer to the question of capability. The chapters that follow are about a different question, and a different answer. The question is what to do when work that needs to happen in a single session crosses several different kinds of attention, when the context window starts filling with material that the current task does not need, when the developer wants three things investigated at once instead of one, and when the cognitive cost of switching modes inside a single Claude session begins to degrade the quality of every output. The answer to these problems is the second pillar of the framework, the subagent.

Begin with a definition that will guide every chapter in this part of the book. A subagent is a separate Claude process, with its own system prompt, its own restricted set of tools, and its own isolated context window, that the main session can dispatch to perform a specific kind of work. When the subagent finishes its work, only its final output returns to the main session. The dozens of tool calls the subagent may have made along the way, the files it may have read, the drafts it may have considered, the dead ends it may have explored, none of that enters the main session. The main session sees a clean result, and the work that produced the result happens in a parallel space that the main session never has to see.

This is a substantial architectural commitment. To understand why the commitment is worth making, it is worth examining what happens to a Claude Code session that does not use subagents and that is asked to do something complex. Take a real example. A developer asks Claude Code to add a new feature to a substantial codebase. The work involves understanding the existing architecture, designing the new addition, writing the code, writing tests for it, updating documentation, and reviewing the whole change for quality. A single Claude Code session can do all of these things in sequence, and it often does. But there is a hidden cost.

In the first phase, Claude reads many files to understand the existing architecture. Each file Claude reads adds to the context window. The context grows by hundreds or thousands of tokens for every file. By the time Claude has built a working understanding of the architecture, the context window is full of source code, much of it only loosely relevant to

the new feature. In the second phase, Claude designs the new addition. The design involves several false starts, each of which leaves traces in the context. By the third phase, when Claude is writing the actual new code, the context window contains the architectural exploration, the design exploration, and the writing of the new code, all jumbled together. By the time Claude reaches the test-writing phase, the context window is dominated by historical material that has nothing to do with testing, and the test-writing suffers as a result, because the model is trying to write tests while looking at the residue of three earlier phases of work.

The Failure Mode Subagents Prevent

This is the failure mode that subagents exist to prevent. The architecture of a subagent is a direct response to this problem. Each subagent has its own context window. The architectural exploration happens inside an explore subagent, whose context fills with the relevant source code, but only the subagent's final summary returns to the main session. The main session never has to see the dozens of files the subagent read. When the main session moves on to design, it does so with a clean context that contains the summary, not the exploration. When the main session moves on to test writing, the design exploration similarly never enters its context, because the design happened in another subagent. Each phase of work happens in a context that is appropriately scoped to that phase, and the main session preserves a clean overview of the whole project rather than becoming a clearinghouse for every detail of every phase.

Three properties of subagents follow from this architecture, and they are worth naming because every subagent design decision later in this part of the book is a function of one of these three properties.

The first property is context isolation. The subagent's context is its own. The main session cannot see what the subagent is reading. The subagent cannot see what the main session is doing in parallel. The two communicate only through the subagent's initial brief and the subagent's final result. This isolation is the source of subagents' context-saving power, but it is also a constraint the developer needs to design around. A subagent cannot be told something halfway through its work; whatever it needs to know must be packed into the brief that launches it.

The second property is specialization. A subagent has its own system prompt, distinct from the main session's system prompt. The system prompt establishes the subagent's role, its expertise, its style of thinking, its outputs. A code-reviewer subagent has a system prompt that is tuned for the craft of reviewing code, not for the broader job of being a general programming assistant. A test-writer subagent has a system prompt that is tuned for the craft of writing tests. The narrowness of the system prompt is a feature, not a limitation. A specialist who is asked to do only one kind of work, and who has been briefed in detail about how to do that work well, produces consistently better outputs than a generalist who is asked to do everything.

The third property is parallelism. Multiple subagents can run at the same time. Claude Code can run several subagents concurrently; community guidance from Anthropic engineers and from heavy users converges on roughly ten as a practical ceiling before queueing becomes noticeable, although the exact number is environment-dependent and Anthropic has not published a hard cap. This means that work which is genuinely independent can happen in parallel rather than in sequence. A code review that involves a security pass, a performance pass, a style pass, and a test-coverage pass does not have to be four sequential reviews. It can be one parallel review where four subagents work simultaneously and the main session synthesizes their findings when they all return. The wall-clock time savings are substantial. The quality improvements are also substantial, because each subagent is focused on its own dimension and does not have to balance attention across all four.

Where Subagents Live

Where do subagents live, mechanically? They live as files in the `.claude/agents` directory. Project-level subagents live at `.claude/agents` in the root of a specific project, where they apply only inside that project. User-level subagents live at `~/.claude/agents` in the user's home directory, where they apply across every project. The naming convention is the same as for skills: lowercase, hyphenated, descriptive. Each subagent is a single Markdown file with YAML frontmatter at the top and a system prompt body below. Here is the simplest possible subagent definition, complete and ready to install.

```
---
```

```
name: code-reviewer
description: Reviews code for quality and best practices. Use proactively after code changes.
tools: Read, Grep, Glob
model: sonnet
---
```

You are a senior code reviewer. When invoked, analyze the code you are given and provide specific, actionable feedback on quality, security, and maintainability. Reference specific file paths and line numbers in your feedback. Keep the review focused: a short, sharp review is more useful than a long padded one.

Look closely at this file, because every subagent in the rest of this part of the book follows the same shape. The frontmatter has four fields, two of which are required. The name field is the subagent's identifier and must match the filename. The description field is what tells Claude when this subagent should be dispatched, and it is the subagent equivalent of the skill description; the same craft of trigger writing applies. The tools field, optional, restricts the tools available to the subagent. The model field, also optional, lets the developer choose which model the subagent runs on, which matters because heavier work might warrant the strongest model while lighter work can use a faster one.

The body of the file is the subagent's system prompt. It is read every time the subagent is dispatched, and it shapes everything the subagent does. The body of a subagent is, in this respect, more consequential than the body of a skill, because while a skill body adds instructions to the main session's overall behavior, a subagent body becomes the entire personality of a separate Claude process. The development of this body is the subject of the next chapter, and it is where the most important design decisions for any subagent get made.

Two conventions concerning invocation are worth knowing now. The first is automatic delegation. When the main session is working on a task and notices that one of the available subagents has a description matching the work, the main session can delegate to that subagent on its own initiative. The developer does not need to type any special command to invoke the subagent; she simply describes her task, and the main session decides whether the task should be handled in-line or delegated. This is the same principle that governs skill firing, applied to a different mechanism.

Good descriptions cause appropriate delegation. Weak descriptions cause subagents to sit unused.

The second convention is explicit invocation. The developer can also invoke a subagent directly, by typing `@agent-name` in the prompt. The mention triggers the named subagent immediately, with whatever brief follows in the prompt as the dispatch instructions. Explicit invocation is useful when the developer wants to be sure a particular subagent handles a particular task, when the description-based delegation might not fire reliably, or when the developer wants to compose multiple specific subagents on a single task. Both invocation modes are valid, and most working developers use both, depending on the situation.

Automatic Versus Explicit

The relationship between automatic delegation and explicit invocation is worth a moment of attention, because choosing between them is a real design question. Automatic delegation is convenient. The developer types in plain language and the right specialist is dispatched, much as the right skill is fired. The cost of automatic delegation is reliability. Description-based dispatch is a judgment call that the main session has to make for every prompt, and the call can occasionally be wrong, especially when several subagents have overlapping descriptions. Explicit invocation is reliable. The developer names the subagent she wants and the subagent runs, full stop. The cost of explicit invocation is friction. The developer has to remember the subagent's name and choose to type it. A working pattern that develops in most experienced users is a hybrid: explicit invocation when the work is high-stakes and the developer wants certainty about which specialist runs, automatic delegation when the work is routine and the convenience matters more than the certainty. The hybrid produces the best of both, and it is worth deliberately practicing both invocation modes during the first weeks of subagent use.

A note on the relationship to skills. Skills and subagents address different problems and complement each other rather than competing. A skill teaches Claude how to do a specific task well. A subagent provides a separate context and a specialized role for a kind of work. Many sophisticated workflows use both: a subagent handles a kind of work, and inside the subagent, skills fire when relevant tasks come up during that

work. A test-writer subagent might activate a project-specific test-conventions skill at the moment it begins drafting tests. The two systems compose, and the composition is where the deepest leverage of Claude Code begins to surface. Chapter Fourteen is dedicated entirely to this composition.

A short clarification for readers who have come from a background of writing prompts and may be confused about why subagents are not simply elaborate prompts. The difference is real and structural. A prompt, no matter how elaborate, runs in the same context as everything else in the session. The context window is shared. The system prompt is shared. A subagent has a separate process, a separate context, and a separate system prompt. The architectural distinction matters because it produces different behavior. A prompt cannot stop the rest of the session from polluting its context. A subagent runs in a clean slate. This is not a difference of degree. It is a difference of kind, and developers who try to replicate subagent benefits with cleverer prompts inside a single session will find that the benefits do not survive the lack of architectural separation. The right answer is to use the architecture, not to try to simulate it.

A short word on the relationship to Claude Code's built-in subagents. Three subagents ship with Claude Code by default and are present in every session: Explore, Plan, and General-purpose. Explore is read-only and runs on Haiku by default; it is optimized for quickly searching and understanding a codebase without making changes. Plan gathers context before the main session enters plan mode, where a strategy is presented. General-purpose handles tasks that need a mixture of exploration and modification. The developer does not invoke these directly very often; the main session routes work to them automatically. Their existence is worth knowing because they are a useful proof of concept of the subagent architecture and because, when the developer designs her own custom subagents, she is extending a system that Anthropic itself uses every day.

It is worth examining the cost side of subagents honestly, because the architecture has trade-offs that the developer should understand before building a heavy subagent library. Each subagent invocation costs tokens. The system prompt of the subagent is loaded into a fresh context, which costs whatever the system prompt's size is. The subagent then reads files, makes tool calls, and produces its final output, all of which consume tokens

that the main session would not have consumed if the work had stayed inline. The total token cost of a subagent invocation is, in most cases, somewhat higher than the cost of doing the same work in the main session, because the subagent does not benefit from any context the main session has already built.

The Cost and Its Compensation

The compensation for this cost is significant, and it has two components. The first component is the quality improvement that comes from a clean, focused context. A subagent producing a security review in a context that contains only a security-review system prompt and the relevant code will produce a deeper review than a main session that is also juggling architectural decisions, build configuration, and three previous tasks. The quality difference is usually worth more than the token cost, because the developer's time is the scarce resource, not the tokens. The second component is the wall-clock saving from parallelism, which only applies in multi-subagent workflows. When four subagents run simultaneously, the wall-clock time is determined by the slowest of the four, not by the sum. The token cost is roughly four times higher than the equivalent sequential work, but the time cost is one quarter, and time is the dimension that governs how many tasks the developer can complete in a working day.

A balanced view, then, is that subagents are worth dispatching when context isolation matters, when specialization is possible, when parallelism applies, or when several of these are true at once. Subagents are not worth dispatching when the work is small, when the main session's context is already clean, and when no specialization or parallelism is available. The developer who internalizes this balanced view will reach for subagents at the right moments and refrain from over-dispatching them at moments where they would only add overhead.

A second balanced view concerns the human side of subagent use. Some developers, on first encountering subagents, become enthusiastic and try to dispatch them on every task. The result is sometimes slower than the work would have been without them, because the orchestration overhead exceeds the benefits. Other developers, on first encountering subagents, treat them with suspicion and avoid them entirely. The result is that they never benefit from the architecture and continue to operate inside an

increasingly polluted main session. Both extremes are mistakes. The right relationship with subagents, after a few weeks of practice, becomes intuitive: the developer reaches for them when the task has an obvious specialization or parallelism shape, and lets the main session handle the rest. This intuition is not difficult to develop, but it requires a few weeks of deliberate use to settle in. The reader who works through Part Three patiently will arrive at the intuition without having to think about it explicitly any longer.

The final piece of context for this opening chapter is a property that emerges only after a developer has built and used several subagents, and that is worth naming up front so that the chapters ahead make sense. Subagents are how a single developer becomes the leader of a small specialist team. The team is patient. It does not get tired. It does not have meetings. Its members do not interrupt each other. Each member is excellent at one thing and confidently silent about everything else. The developer's job is to be the editor, the orchestrator, the one who decides who works on what and how the results come together. Claude Code, equipped with a well-designed subagent library, becomes a leverage multiplier in a way that Claude Code without subagents simply cannot. The developer who masters this multiplication will, within a few months of practice, find that the volume and quality of work she can produce has shifted in a noticeable, sustainable way.

The chapter that follows takes the body of the subagent file, the system prompt, and treats it as the central design decision in any subagent. Every choice about what a subagent does, what it does not do, how it should think about its work, and how it should report its findings, is a choice expressed in the system prompt. The discipline of writing those prompts well is the discipline that determines whether a subagent is a useful specialist or a confused generalist. That discipline begins now.

Designing Effective Subagents: Role, Tools, and Boundaries

“Give a person too much to do, and they do nothing well. Give them one thing, and they may surprise you.”

Peter Drucker, paraphrased

A subagent that does too much is worse than no subagent at all. This sounds counterintuitive on first hearing, because the natural instinct of a developer encountering a new capability is to put as much capability into it as possible. The instinct is wrong, and this chapter explains why and what to do instead. The art of subagent design is the art of restraint. The subagent that performs one role well, with a tightly defined scope and a thoughtfully constrained set of tools, produces consistent, reliable, useful output. The subagent that tries to be a general assistant in a slightly different costume produces output that is sometimes excellent, sometimes mediocre, and almost never reliably one or the other. Reliability is the entire point.

Three design dimensions govern every subagent, and the disciplined consideration of each is what separates good subagents from poor ones. The first dimension is role. The second is tools. The third is boundary. Each of these expresses a constraint, and constraints are what make subagents useful. A subagent without constraints is, in effect, a slightly more isolated version of the main session, and most of the value of having a subagent is forfeited.

Begin with role. The role of a subagent is its identity, expressed at the top of its system prompt. A code-reviewer subagent is a code reviewer. Not a general programmer who reviews code sometimes. Not a teacher who happens to be reviewing code. A code reviewer. The role definition should be specific enough that it constrains the kind of work the subagent does, the kind of language it uses, the kind of judgment it brings, and the kind of output it produces. A weak role definition reads like a job title. A strong role definition reads like the introduction of a person.

Compare these two openings of a code-reviewer subagent system prompt. The first is weak.

You are a code reviewer. Review code carefully.

Strong Subagent Descriptions

The second is strong.

You are a senior code reviewer with ten years of experience reviewing pull requests on production systems. Your job is to find issues that affect correctness, security, or maintainability, and to communicate them clearly to the author. You are not friendly for the sake of friendliness; you are honest about problems because honest reviews save the team from production incidents. You do not pad reviews with general observations. You name what is wrong, why it is wrong, and what to change. When the code is good, you say so plainly in one sentence and move on.

The second prompt does many things at once. It establishes seniority. It establishes purpose. It explicitly rejects two common failure modes: empty friendliness and padded reviews. It gives the subagent a posture, a stance toward its work. The subagent that runs under the second prompt produces noticeably different output from the subagent that runs under the first, even though both are nominally code reviewers. The role definition is a substantial fraction of the subagent's personality, and writing it carefully is among the most consequential acts of subagent design.

Move to tools. The tools field in the frontmatter restricts which tools the subagent is allowed to invoke. Omitting the field grants the subagent the full set of tools available to the main session. Specifying a list narrows the set. The narrowing is not a security feature alone, although it is a useful one. The narrowing is also a focusing feature. A subagent that has only the tools it needs does not get distracted by the tools it does not need. A code-reviewer subagent that has access only to Read, Grep, and Glob cannot make file edits even if its system prompt forgets to forbid them, and the absence of Edit, Write, and Bash from its toolkit shapes how it approaches every task. The subagent does not casually offer to fix the issues it finds. It cannot, because the tools to do so are not present, and so its output naturally reflects the boundary.

A simple rule guides tools restriction. Begin with no tools, then add only the ones the subagent demonstrably needs. A reviewer needs to read

files, search them, and find them, so Read, Grep, and Glob suffice. A documentation-writer needs to read files and write new ones, so Read, Write, and Glob suffice. A test-writer needs to read existing code, write new test files, and possibly run tests to verify, so Read, Write, Bash, and Glob form a reasonable set. The principle in each case is that the tool list mirrors the subagent's job description. If the subagent does not need a tool to do its job, the tool is not granted, full stop.

Move to boundary. The boundary of a subagent is the set of things it explicitly does not do. The role definition says what the subagent is. The boundary says what the subagent is not. Both are necessary, because the language model that runs the subagent will, when uncertain, tend to drift toward generality, and explicit boundaries push back against that drift.

Boundary Language in Prompts

Boundary language belongs in the system prompt and should be specific. A code-reviewer subagent should have boundary language such as: you do not write or modify code; if you find an issue, you describe it and recommend a fix, but you do not implement the fix yourself. A test-writer subagent should have boundary language such as: you do not change the code under test; if you discover a bug while writing tests, you note it in the test as a comment, and you flag it in your final summary, but you do not modify the production code. A documentation-writer subagent should have boundary language such as: you do not modify code, configuration, or any non-documentation file. Each of these statements feels redundant when first written, because the developer thinks the role definition already implies the boundary. The redundancy is the point. Stating the boundary explicitly costs nothing and prevents the small but real failure mode of a subagent quietly drifting outside its scope on a task where the role definition was ambiguous.

A complete subagent file, designed under all three dimensions, looks like this. The example below is a security-auditor, intended to focus narrowly on security issues in code, leaving general code review and other concerns to other subagents.

```
---
name: security-auditor
description: Audits code for security issues, including OWASP Top 10 vulnerabilities, hardcoded secrets, unsafe dependencies, and injection risks. Use proactively after changes to authentication,
```

data handling, or external interfaces.

tools: Read, Grep, Glob, Bash

model: opus

You are a senior application security engineer. Your job is to find security issues in code that other reviewers are likely to miss, and to explain those issues clearly enough that the developer can fix them without further research.

Your areas of expertise:

- Authentication and authorization flaws
- Injection vulnerabilities (SQL, command, prompt, template)
- Hardcoded secrets and credentials
- Unsafe deserialization
- Missing input validation
- Insecure direct object references
- Cross-site scripting and request forgery in web contexts
- Time-of-check to time-of-use bugs
- Vulnerable or outdated dependencies

Your method:

1. Read the code you are given.
2. Look for the patterns above explicitly. Do not skim.
3. For each issue you find, produce a short report containing:
 - Severity: critical, high, medium, or low
 - Location: file path and line number
 - Description: what the issue is, in plain language
 - Recommendation: a concrete fix, with a code example if useful

What you do not do:

- You do not modify code. You only report.
- You do not perform general code review. Style, performance, and architecture concerns belong to other reviewers.
- You do not pad your output. If the code has no security issues, say so plainly in one sentence.
- You do not invent issues. If you are uncertain whether something is a vulnerability, mark it as a question rather than a finding.

End every report with a one-sentence summary judgment: clear, minor issues only, or significant issues found.

Read the file once more, slowly, and notice the discipline at every level. The description in the frontmatter is specific about the kind of work the subagent does and uses the proactively phrasing recommended by Anthropic to encourage reliable delegation. The tools list is narrow, exactly what the subagent needs and nothing more. The model is set to opus,

because security review benefits from the strongest model the developer has available. The system prompt establishes a senior role with explicit areas of expertise. It defines the method as a numbered procedure. It includes a what-you-do-not-do section that is as long as the what-you-do section, because the boundary is genuinely as important as the role. It ends with a directive about the format of every report. Every degree of freedom that could produce variability in output has been pinned down deliberately.

A subagent designed this carefully produces strikingly consistent output across many invocations. Two different developers running this subagent against two different codebases will get reports that look like they came from the same security engineer, because in a meaningful sense they did. This consistency is what makes subagents useful as components of a larger system. A subagent whose output varies wildly from invocation to invocation is hard to compose with other subagents, hard to review, and hard to trust. A subagent whose output is stable in form, even when the substance varies with the codebase, can be plugged into a larger workflow with confidence.

A second worked example will reinforce the design principles, because the discipline becomes habitual only after the developer has seen it applied to several different specialties. The subagent below is called documentation-writer. Its job is to produce or update documentation for code, with the same kind of consistency and discipline as the security-auditor.

```
---
name: documentation-writer
description: Produces clear, useful technical documentation for code, modules, or APIs, including
README sections, JSDoc-style comments, architecture notes, or contribution guides. Use
proactively when a feature has been completed without accompanying documentation, or when
the user asks to "document this," "update the README," or describes wanting documentation
written.
tools: Read, Write, Glob, Grep
model: sonnet
---
```

You are a senior technical writer who works alongside engineers. Your job is to produce documentation that engineers actually read, that helps the next engineer who encounters this code, and that does not waste anyone's time with restatement of what the code already says.

Your principles:

- Documentation explains why, not what. Anyone can read the code to see what it does. Documentation should explain why it does that, what trade-offs were made, and what assumptions hold.
- The audience is a smart engineer who has not seen this code before and who will spend roughly five minutes deciding whether to read further. Calibrate accordingly.
- Examples beat prose. A short worked example is worth several paragraphs of explanation.
- Honest documentation beats aspirational documentation. Document what the code actually does today, not what you wish it did.
- Concise documentation is read. Long documentation is skimmed at best and ignored at worst.

Your method:

1. Read the code in full. Read the surrounding code if needed for context.
2. Identify the public surface of the code: what callers will use.
3. Produce documentation in the project's existing format. Inspect the project to determine what format is in use (README, JSDoc, Sphinx, etc.).
4. Cover, in order: purpose, public API, expected inputs and outputs, important behaviors and edge cases, a short worked example, any non-obvious considerations.

What you do not do:

- You do not modify code. Your edits are confined to documentation files.
- You do not write documentation that is a transliteration of the code into English.
- You do not invent behaviors that the code does not exhibit.
- You do not pad. A short, accurate page is the goal.
- You do not produce sycophantic prose. Avoid phrases like "powerful," "elegant," or "robust" unless the developer would say them in plain conversation.

End with a list of the documentation files you wrote or updated, plus a short note on anything you found unclear in the code itself that the engineer might want to address.

Comparing Two Specialists

Compared with the security-auditor, the documentation-writer differs in several telling ways. The role is different: technical writer rather than security engineer. The tools are different: Read, Write, Glob, Grep, including the ability to write files because documentation is the deliverable; security-auditor was read-only because audit is its deliverable. The model is sonnet rather than opus, because documentation work does not particularly benefit from the strongest model and the cost saving is real for a subagent that may run several times a day. The principles section is where the writer's craft is encoded; the security-auditor had a similar section, but its content was about security, not about writing.

What the two subagents share is the structural discipline. Both have a clear role definition. Both have a numbered method. Both have an explicit list of things they do not do. Both have a directive about the format of their final output. This shared structure is the design discipline of the chapter, and the reader should recognize that the structure itself is doing the work of producing reliability, even more than any specific instruction the prompts contain.

A debugging walkthrough is worth including, because the experience of building subagents that misbehave is universal, and the cure is teachable. The walkthrough below is from a real session. The developer had built a code-reviewer subagent, dispatched it on a pull request, and found that the output was not what she expected. The subagent had produced a review, but the review felt generic. It mentioned reasonable things, but those reasonable things did not seem to come from the specific code the subagent had been asked to review. The developer suspected, correctly, that the subagent had read the code in a cursory way and then produced a review that could have applied to almost any code.

She investigated the system prompt. The role definition was decent. The method was a numbered procedure. The output format was specified. But on close reading, she noticed that the method said "review the code" without instructing the subagent to read it carefully, look for specific issues, or reference specific lines. The boundary section did not say what generic feedback was forbidden. The result was that the subagent, working under permissive instructions, took the easiest path and produced output that was technically a review but not the close, specific review the developer wanted.

She rewrote the method. The new method instructed the subagent to read the diff in full before forming any opinions, to identify three to five specific issues that referenced file paths and line numbers, and to write each comment as it would appear in a real pull request thread. She added a what-you-do-not-do entry forbidding generic observations that did not reference specific lines. She redispached the subagent on the same pull request. The output transformed. The new review named four specific issues, each with a file and line reference, each with a concrete suggestion. The genericity was gone. The total time spent on the fix was about twenty minutes, including the time to test the new version on three different inputs to confirm the consistency had improved.

A Debugging Lesson

The lesson the developer took away from this debugging session was that subagent quality is largely controlled by the specificity of the instructions in the system prompt. A subagent given vague instructions produces vague work. A subagent given specific, structured instructions, with explicit prohibitions on the failure modes the developer has already encountered, produces work that consistently lives up to the design. The fix is rarely to make the subagent smarter. The fix is almost always to make the instructions more specific.

A final practice deserves mention before this chapter ends, because it is the practice that separates careful subagent design from sloppy subagent design. After writing a subagent, the developer should test it on three different inputs and compare the outputs. Not for correctness, although correctness matters. For consistency. The three outputs should look like they came from the same specialist. Same format. Same tone. Same level of detail. If the outputs vary in any of these dimensions, the system prompt has not pinned down the relevant degree of freedom, and the system prompt should be tightened until the variation disappears. This three-input test takes ten minutes and prevents an entire category of long-tail problems that would otherwise only surface in a real session at an inconvenient moment.

Subagent design, like skill design, becomes faster with practice. The first subagent the developer writes will take an hour. The fifth will take twenty minutes. The tenth will take ten minutes, because the design dimensions and the system-prompt patterns are now familiar, and the developer has a working library of prior subagents she can adapt. The path from beginner to confident designer is short, and the rewards begin compounding from the first well-designed specialist.

A short reflection on the role of the model field in subagent design. The model field in the frontmatter lets the developer specify which model the subagent runs on. This may seem like a minor detail, but in practice it is one of the more consequential design choices. Subagents that demand the strongest reasoning, such as the security-auditor and the architecture-reviewer earlier in this part of the book, benefit from running on opus. Subagents that perform structured but less reasoning-intensive work, such as the documentation-writer or the test-writer, run perfectly well on sonnet

at lower cost. Subagents that perform fast pattern-matching work, such as a search-helper or an explore agent, can run on haiku and complete in seconds rather than minutes. The model field, used thoughtfully, lets the developer tune each subagent to the cost and speed profile that best fits its job, rather than running everything on a single model and either overspending on simple work or underspending on demanding work. This kind of per-subagent tuning is the small mark of a mature subagent library, and it pays for itself across the months a library is in use.

The next chapter takes subagent design from the single subagent to the multi-subagent workflow. With the design of an individual specialist established, the question becomes how multiple specialists work together on a single task, how the main session orchestrates them, and how parallel execution turns work that would have been sequential into work that finishes in a fraction of the time. Orchestration is where the architecture of subagents reveals its full power, and that power is the subject ahead.

Parallel Subagents and the Orchestrator Pattern

“Coordination is the art of getting many things done in the time of one. The conductor does not play any of the instruments.”

Folk wisdom of the orchestra pit

The previous chapter taught the design of a single subagent. This chapter is about what happens when multiple subagents work on the same task at the same time. Parallelism is the property that turns subagents from a useful refinement into a structural change in how a developer relates to her tool. A single subagent saves context. Multiple parallel subagents save wall-clock time, deepen quality through specialization, and produce reviews and analyses that no single Claude session, no matter how capable, could produce in the same span of minutes.

Begin with the simple case. The main session is asked to perform a task that has multiple genuinely independent components. The components do not need to share information with each other, and the order in which they are performed does not affect the result. A code review of a substantial pull request is a perfect example. A reviewer worth her salt will check several dimensions: correctness, security, performance, style, test coverage. None of these dimensions strictly depends on any other. A correctness check does not need the security check to have run first. A style check does not need the performance check to be complete. The dimensions are orthogonal, which is exactly the property that makes them candidates for parallel execution.

The orchestrator pattern formalizes the idea. The main session takes the role of conductor. It receives the task, decomposes it into independent components, dispatches one subagent per component, waits for them all to finish, and synthesizes their outputs into a unified result. The subagents do not speak to each other. They do not know about each other. Each one focuses on its single dimension, produces its single bounded report, and returns. The conductor takes all the reports, weaves them together according to whatever format the unified output requires, and presents the result to the developer. The developer sees one coherent answer. The work

that produced the answer happened in parallel, in five separate contexts, each tuned to its own dimension.

A worked example will make this concrete. The developer has a pull request she wants reviewed thoroughly before merging. She types into Claude Code something like, give me a full review of this PR, including security, performance, style, and test coverage. The main session recognizes that a thorough review is being requested, recognizes the dimensions involved, and dispatches four subagents in parallel: a security-auditor, a performance-reviewer, a style-reviewer, and a test-coverage-checker. Each subagent receives the same brief: review this diff, focused on your dimension. Each subagent runs in its own context, reads the relevant files, applies its specialty, and produces its bounded report. The four reports return to the main session. The main session presents them in a unified format, with a summary judgment that integrates the findings across the four dimensions.

Estimating the Wall-Clock Saving

The wall-clock saving is straightforward to estimate. If each subagent takes two minutes, four subagents in sequence take eight minutes, while four subagents in parallel take roughly two minutes, plus the synthesis step. The factor of four speedup is real and felt. On a code review that the developer might have run twice a day, the saving across a working week is substantial.

But the time saving is not the most interesting benefit. The more interesting benefit is the quality. Each subagent operates in a context that contains only the material relevant to its dimension. The security-auditor's context contains the diff plus the security-auditor's system prompt, focused on security. The performance-reviewer's context contains the diff plus the performance-reviewer's system prompt, focused on performance. Neither subagent has its attention diluted by considerations that are not its own. The result is that each subagent produces a deeper, more focused report than a single generalist reviewer could produce on the same input. The total review is not just faster; it is better, because each part of it was done by a specialist who was not trying to balance attention across four dimensions at once.

The orchestrator pattern has a discipline that is worth naming. The main session's job, as the orchestrator, is decomposition and synthesis. The

subagents' job is the specialty work. The boundary between these two jobs should be respected. The main session should not attempt to do specialty work itself, even when it could; if a security review is needed, the security-auditor subagent should do it, not the main session. The main session should not attempt to mix specialties inside a single subagent, either; a security-and-performance reviewer is a worse subagent than two narrow ones, because the specialization advantage is forfeited. The orchestrator pattern asks the developer to design subagents narrowly, then trust the design, and let parallelism do its work.

There are limits to parallelism worth knowing. Claude Code can run several subagents concurrently in practice; community guidance from heavy users converges on roughly ten as a practical ceiling before queueing becomes noticeable, although Anthropic has not published a hard cap. Beyond that limit, additional dispatches queue and run as earlier ones finish. Within the limit, the parallelism is genuinely simultaneous: the subagents do not take turns; they execute in independent processes. The cost of running multiple subagents is, naturally, multiplied by the number of subagents. A four-subagent review costs roughly four times the tokens of a single-subagent review. This cost is usually well worth paying, because the time and quality benefits dominate, but it is a real cost and worth being aware of.

There are tasks for which parallelism is the wrong shape, and recognizing them is part of the discipline. Tasks whose components depend on each other do not benefit from parallel execution; the dependencies force sequential work. A task that involves first reading a database schema, then writing a query against it, then optimizing the query, has a strict order that cannot be parallelized usefully. Tasks whose components share a great deal of context also do not benefit; if every subagent would need to read the same fifty files to do its job, the duplication of that reading across many subagents wastes tokens and time. The right pattern in the dependent case is sequential subagent dispatch, where the output of one subagent becomes the brief for the next. The right pattern in the shared-context case is sometimes a single subagent that handles all the work, or sometimes a careful brief that summarizes the shared context once and passes the summary to each subagent so they do not all need to re-derive it.

When to Reach for Parallelism

A useful framing for deciding when parallelism applies is this question: would I, as a developer, hire four different specialists for four different parts of this task, knowing that the specialists do not need to talk to each other? If the answer is yes, the orchestrator pattern fits. If the answer is no, because the specialists would need to coordinate continuously, the work belongs in a single subagent or in a sequence. This human-team metaphor maps cleanly onto subagent design, and developers who train themselves to ask the metaphor question quickly learn to recognize which workflows benefit most from parallelism.

A worked end-to-end orchestration looks like this in practice. The developer types her broad request. The main session, recognizing the orchestratable pattern, identifies the dimensions and dispatches the subagents. The dispatches happen in quick succession, and the subagents begin work simultaneously. The main session, while waiting, sometimes does a small amount of preparatory work, like inspecting the diff itself for the synthesis step. The subagents return their reports as they finish; reports may not arrive in dispatch order, since some specialties take longer than others. The main session collects them, holds them until all are present, and then composes the unified output. The whole interaction takes only marginally longer than the slowest of the four subagents. The developer sees a thorough, multi-dimensional review delivered in roughly the time of one careful pass, and the review reads as if four specialists had been retained for the job, because, in a meaningful sense, four specialists were.

A concrete example will fix this in the reader's mind. Consider a developer who has just opened a pull request that touches roughly four hundred lines of code in three files. The change introduces a new authentication endpoint. Before merging, she wants a full review. She types into Claude Code, in plain language, perform a full review of this PR covering security, performance, style, and test coverage. The main session reads the request. It recognizes the four named dimensions. It looks at the available subagents and finds matches: security-auditor, performance-reviewer, code-style-checker, test-coverage-analyzer. It dispatches all four with a brief that includes the diff and a directive to focus on the subagent's named dimension.

The four subagents begin work simultaneously. The security-auditor reads the new authentication code with attention to OWASP categories. The performance-reviewer scans for hot-path issues, queries inside loops, and unbounded allocation. The code-style-checker reads the diff against the project's style conventions. The test-coverage-analyzer cross-references the diff against existing test files to identify which new code paths are unexercised. Each subagent works in a context that contains only the diff and its own focused system prompt. None of them sees the others' work. Each produces a focused report.

Roughly two minutes later, the four reports return to the main session in whatever order they finish. The security-auditor flags a missing rate limit on the password reset endpoint and a token expiry that defaults to twenty-four hours rather than the project's standard of one. The performance-reviewer flags nothing significant. The code-style-checker notes two minor style deviations. The test-coverage-analyzer points out that the new password reset endpoint has no test coverage at all. The main session reads all four reports, composes a unified review with sections for each dimension, and presents the result to the developer. Total wall-clock time, from her first prompt to the unified review on her screen, is roughly two and a half minutes. Total human attention required, zero, until the review is ready to read.

Without the Pattern

Now imagine the same review without the orchestrator pattern. The developer would have asked the main session to do the review directly. The main session would have read the diff, then mentally switched into security mode, then into performance mode, then into style mode, then into test-coverage mode, all in a context that accumulated the residue of each phase. The total wall-clock time would have been closer to ten minutes, because the work was sequential rather than parallel. The total quality would have been lower, because each phase was attended to by a generalist rather than a specialist. And the cost in tokens, while lower, would have been a false economy, because the developer's attention is more valuable than the marginal token spend, and the orchestrated review let her get to her next task seven minutes sooner.

The orchestrator pattern is not the only multi-subagent pattern, although it is the most visually striking one. A second important pattern is the sequential pipeline, in which the output of one subagent becomes the brief for the next. This pattern applies when the work has a real ordering dependency that cannot be parallelized. A common example: the developer wants to refactor a substantial piece of code, but only after having an architectural review of the current structure to confirm the refactor's direction. The architecture-reviewer runs first; its output, a recommendation about which structure is appropriate, becomes part of the brief for the refactor-specialist that runs second. The two cannot run in parallel because the second one needs the first one's conclusions.

The mechanics of sequential pipelines are slightly different from those of parallel orchestration. The main session dispatches the first subagent, waits for its result, integrates the result into the brief for the next subagent, dispatches that one, waits for its result, and so on. The total wall-clock time is the sum of each subagent's time, with no parallelism saving. The benefits are still substantial: each step happens in a focused context, each step's output is shaped by the previous step's output rather than by the noise of an accumulated session, and the pipeline can be reviewed step by step, with the option to stop at any intermediate stage if the work is not going where the developer wants. Sequential pipelines, while slower than orchestrations, are the natural shape of work that has real dependencies, and the developer who recognizes when she is in a sequential rather than a parallel situation will save herself the frustration of trying to force orchestration onto work that does not support it.

A third pattern, which is a hybrid of the first two, is worth naming briefly. The map-reduce pattern dispatches several subagents in parallel on independent components of a task and then dispatches a final subagent, sequentially, to synthesize their outputs. Sometimes the synthesis is straightforward enough for the main session to do directly, in which case it is just the orchestrator pattern. Sometimes the synthesis is itself a substantial specialty, such as composing a coherent design document from several focused reports, in which case a synthesis subagent is worth defining. The map-reduce pattern shows up most often in larger codebases, where the inputs are too varied for a single specialist to read in full. Different parallel subagents read different sections; a synthesis subagent

receives their reports and produces the final output. The pattern is more elaborate, but the underlying principles are the same: independent work in parallel where possible, dependent work in sequence where required, focused contexts at every stage.

A short reflection on what makes orchestration genuinely powerful. The most striking thing about the orchestrator pattern is not the wall-clock saving, although the saving is real and welcome. The most striking thing is what the developer is freed from. She no longer has to be the one who remembers all the dimensions a thorough review should cover. The pattern itself remembers them. She no longer has to mentally context-switch between security thinking and performance thinking and style thinking, because the subagents handle each dimension in its own thread. Her attention, which was previously distributed across all the dimensions she cared about, can be concentrated on whatever the unified output most demands of her. She becomes, in effect, a senior engineer reading reports written by competent specialists, rather than a junior engineer trying to be all four specialists at once. The transformation in working experience is not just a productivity improvement; it is a change in the kind of cognitive work she is asked to do, and developers who experience it for the first time often describe the shift as the moment Claude Code stopped feeling like a chat interface and started feeling like an environment.

When Not to Orchestrate

A final note, before this chapter closes, on when not to reach for orchestration. The orchestrator pattern carries overhead. Dispatching four subagents and synthesizing their outputs is more elaborate than asking the main session to do a small task directly. For small tasks, the overhead is not worth paying. A quick code review of a fifty-line patch does not need four subagents; the main session can handle it directly. A trivial documentation update does not need a documentation specialist; the main session is fine. The orchestrator pattern earns its keep on substantial tasks, where the parallelism saving and the specialization benefit are worth the dispatch overhead. The developer who reaches for orchestration appropriately, and refrains from reaching for it on tasks that do not justify it, will have the best experience.

A useful rule of thumb is the five-minute test. If the developer would spend more than five minutes on the task as a single sequential pass through the main session, the orchestrator pattern is probably worth considering. If she would spend less than five minutes, orchestration is probably overkill. The threshold is not exact, and individual judgment varies, but the rule prevents both common mistakes: orchestrating tiny tasks where the dispatch overhead dominates, and serializing large tasks where the parallelism would have produced real savings. With practice, the developer stops needing the rule, because the right shape of work for orchestration becomes visible at a glance. Until that intuition has set in, the five-minute test is a reasonable working approximation.

A second observation worth keeping in mind is that orchestration scales with the developer's familiarity. New users of subagents tend to under-orchestrate, treating the main session as the default and reaching for subagents only on the most obvious cases. Experienced users tend to orchestrate naturally, recognizing the orchestratable shape of work in their first or second pass through a problem. The transition from the first state to the second takes only a few weeks of deliberate practice, and the practice is enjoyable rather than tedious, because every successful orchestration produces a noticeable, satisfying acceleration of work that would otherwise have been slow.

One final reflection on this chapter, before moving to the inventory of specialists. The orchestrator pattern is the visible face of subagents, the part that produces the most striking demonstrations and the most quotable speedups. But the deeper value of subagents, the value that compounds quietly across months of use, is not the speed. It is the consistent quality. A library of well-designed specialists produces consistently high-quality work because each specialist has been designed for its specific role, and the design is preserved across every invocation. The developer who has installed a serious subagent library is not just faster. She is more reliably good at her work, because the failure modes that come from overload, distraction, or context pollution have been engineered out of her environment. The reliability is the prize. The speed is a happy side effect.

The next chapter shifts from the single orchestration pattern to the inventory question. With the design of single subagents and the architecture of parallel orchestration both established, the developer needs a working

library of subagents to draw from. The chapter ahead provides that library. Eight specialist subagents, each ready to install, each addressing a recurring need in daily development work, will make the architecture of this part of the book concrete in installed form. By the end of the next chapter, the reader will have a team of specialists waiting in her `.claude/agents` directory, ready to be dispatched whenever her work calls for them.

The Subagent Library: Eight Specialists Every Developer Needs

“A small team of true specialists outperforms a large team of competent generalists almost every time. The difference is not effort. It is depth.”

attributed to W. Edwards Deming

This chapter is the practical counterpart to Chapter Nine. By the time the reader finishes it, eight specialist subagents will be installed in her `~/.claude/agents` directory, ready for use in any project. Each specialist is presented in the same uniform format: the problem it solves, the complete subagent file, and a short explanation of the design choices that shaped it. The reader who installs all eight will have a small but capable team of specialists at her disposal, available for dispatch whenever her work crosses one of their domains.

The selection of eight specialists is deliberate. The eight cover the most common dimensions of code work that benefit from isolation and specialization: security, performance, testing, documentation, architecture, refactoring, dependency hygiene, and migration. These are the dimensions that, in a real engineering organization, would be staffed by named specialists with named roles. Replicating those specialist roles in subagent form is what gives a single developer, working in Claude Code, access to expertise that would otherwise be available only inside a substantial engineering team.

Before beginning the installation, a small piece of housekeeping. All eight subagents below are personal subagents, installed at the user level, and therefore live in `~/.claude/agents`. Each one is a single Markdown file named after the subagent. After installing all eight and restarting Claude Code, the subagents will be available in every session, in every project the developer opens. Project-specific specialists, if needed, can be added later in the `.claude/agents` directory of specific projects.

The First Specialist

The first specialist is security-auditor, presented in full in the previous chapter. Its job is to audit code for security vulnerabilities. If the reader has not already installed it, the file from Chapter Eleven can be used directly.

The second specialist is performance-reviewer. Its job is to review code for performance issues, with a focus on the kinds of problems that show up in production rather than the kinds that show up in micro-benchmarks.

```
---
name: performance-reviewer
description: Reviews code for performance issues that affect real-world execution, including
algorithmic complexity, database query patterns, memory allocation, and unnecessary work in hot
paths. Use proactively on changes that touch loops, queries, or data structures.
tools: Read, Grep, Glob, Bash
model: sonnet
---
```

You are a senior performance engineer. Your job is to find performance issues that will affect production behavior, and to ignore micro-optimizations that do not matter at scale.

Your areas of focus:

- Algorithmic complexity ($O(n^2)$ where $O(n)$ would do)
- Database queries inside loops (the classic $N+1$ pattern)
- Excessive memory allocation in hot paths
- Synchronous I/O where async would be appropriate
- Unbounded work that should be paginated or streamed
- Cache misses where caching would clearly help
- Repeated computation that should be memoized

Your method:

1. Read the code you are given.
2. Identify hot paths: code that runs frequently, runs on every request, or runs in tight loops.
3. Apply your areas of focus to those hot paths first. Apply them lightly to cold paths only if something obvious stands out.
4. For each issue, produce a short report with severity, location, description, and recommendation.

What you do not do:

- You do not modify code. You report.
- You do not flag micro-optimizations. A nanosecond saved in a path that runs once a day is not worth a comment.
- You do not pad. If the code looks fine, say so plainly.
- You do not benchmark. Your review is qualitative, based on patterns. The developer can benchmark if she wants quantitative data.

End with a one-sentence judgment: clear, minor issues only, or significant issues found.

The third specialist is test-writer. Its job is to write tests for a given function or module, following the conventions of the project it is operating in.

```
---
name: test-writer
description: Writes test files for a given function, module, or feature, following the test
framework and conventions of the current project. Use when the user asks for tests, says "add
coverage," "write a test for this," or finishes a feature without tests in place.
tools: Read, Write, Bash, Glob
model: sonnet
---
```

You are a senior test engineer. Your job is to write tests that catch real bugs, not tests that only exercise the happy path so that coverage numbers look better.

Your method:

1. Read the file under test and any closely related files needed to understand the code's behavior.
2. Detect the test framework in use by inspecting package.json or the project's equivalent. Default to whatever the project already uses.
3. Identify the public surface of the file: functions, methods, classes that other code calls.
4. For each item on the public surface, write tests covering at minimum:
 - The happy path
 - At least one boundary condition
 - At least one error or invalid-input case
5. Mock external dependencies. Do not make network calls or write to real files in tests.
6. Place test files alongside the code under test, following the project's existing layout.

What you do not do:

- You do not modify the code under test, even if you discover bugs while writing tests. Note the bug as a comment in the test and flag it in your final summary.
- You do not write tests that only call the function and assert that it does not throw. That is not a test.
- You do not mock the function under test. You mock its dependencies.
- You do not produce an excess of nearly-identical tests. Three meaningful tests are better than ten redundant ones.

End with a list of the test files you wrote and any bugs or concerns you noticed in the code under test.

The Documentation Writer

The fourth specialist is documentation-writer. Its job is to write or update documentation for a given module, API, or feature. The specialist is read-and-write within documentation files only, leaving all code untouched.

name: documentation-writer

description: Writes or updates documentation for code, including README sections, API docs, JSDoc comments, and architecture notes. Use when the user asks for docs, "document this," "update the README," or finishes a feature that needs accompanying documentation.

tools: Read, Write, Glob

model: sonnet

You are a senior technical writer who works alongside engineers. Your job is to produce documentation that engineers actually read and that helps the next engineer who encounters the code.

Your principles:

- Documentation explains why, not what. Anyone can read the code to see what it does. Documentation should explain why it does that.
- The audience is a smart engineer who has not seen this code before. Calibrate detail to that level.
- Examples beat prose. A short worked example is worth several paragraphs of explanation.
- Honest documentation beats aspirational documentation. Document what the code actually does, not what you wish it did.

Your method:

1. Read the code you are documenting in full.
2. Identify the public surface: what callers will use.
3. Write the documentation in the project's existing documentation format. If the project uses JSDoc, use JSDoc. If the project uses Markdown READMEs, write Markdown.
4. Cover: purpose, public API, expected inputs and outputs, important behaviors and edge cases, a short example.
5. If something about the code is unclear, ask the engineer rather than guess.

What you do not do:

- You do not modify code. Your only edits are to documentation files.
- You do not write documentation that simply restates the code in English.
- You do not invent behaviors that the code does not actually exhibit.
- You do not pad. A short, clear page beats a long padded one.

End with a list of the documentation files you wrote or updated.

The fifth specialist is architecture-reviewer. Its job is to look at proposed changes from a step-back perspective, asking whether the structure of the solution is right rather than whether the implementation is correct.

name: architecture-reviewer

description: Reviews code or proposed changes for architectural concerns: separation of responsibilities, layering violations, coupling, abstraction quality, and consistency with existing patterns. Use when reviewing significant new modules, major refactors, or design proposals.

tools: Read, Grep, Glob

model: opus

You are a senior software architect. Your job is to look at code from a step-back perspective: is the structure right, do the responsibilities live in the right places, does this fit with the rest of the system, will future engineers be able to extend this without unwinding it.

Your areas of focus:

- Separation of concerns: does each module have a clear, single responsibility?
- Layering: are higher-level modules depending on abstractions, with implementation details pushed downward?
- Coupling: do modules know about each other only through narrow, intentional interfaces?
- Abstraction quality: are abstractions earning their place, or are they obscuring rather than clarifying?
- Consistency: does this code fit with the patterns already established in the project, or does it introduce a new pattern that may fragment the codebase?

Your method:

1. Read the code in question.
2. Read enough of the surrounding code to understand the existing patterns and conventions.
3. Apply your areas of focus, prioritizing the ones most relevant to this change.
4. Produce a short report. Lead with the most important architectural observation. Do not dilute it with minor stylistic concerns.

What you do not do:

- You do not nitpick. Architectural review is about the shape of the solution, not the details of the implementation.
- You do not suggest sweeping rewrites unless the code clearly cannot be evolved into something acceptable.
- You do not modify code.
- You do not pad. Architectural review is qualitative; one good observation is worth a hundred mediocre ones.

End with a one-sentence judgment: structurally sound, structurally questionable but acceptable, or structurally problematic and worth reconsidering.

The sixth specialist is refactor-specialist. Its job is to take working code and produce a refactor that improves a specific named property, without changing behavior.

name: refactor-specialist

description: Produces a refactor of given code that improves a specified property (readability, testability, cohesion, or naming) while preserving behavior exactly. Use when the user says "refactor this," "clean this up," "make this easier to test," or names a specific structural property to improve.

tools: Read, Edit

model: sonnet

You are a senior refactoring engineer. Your job is to produce refactors that make code better along a specific dimension, without ever changing what the code does.

Your discipline:

- Behavior preservation is absolute. The refactored code must produce identical observable output for all inputs the original handled.
- Improvements are along a single named dimension at a time. If the user asks for readability, you improve readability. If the user asks for testability, you improve testability. You do not bundle multiple kinds of improvement into one change unless explicitly asked.
- You explain the change. After producing the refactor, you list what improved and why.

Your method:

1. Read the code carefully.
2. Confirm the dimension you are optimizing for. If unclear, ask the user before proceeding.
3. Identify the highest-leverage opportunity to improve along that dimension.
4. Produce the refactor as a complete replacement of the original, not as a diff.
5. Explain what changed and why, in three to five lines.

What you do not do:

- You do not change behavior. If you notice a bug, you mention it separately and do not silently fix it.
- You do not bundle unrelated improvements into the same refactor.
- You do not introduce new dependencies, frameworks, or libraries unless the user explicitly authorizes them.
- You do not make stylistic changes that do not serve the named dimension.

End with the refactor and the explanation, and nothing else.

The Dependency Analyzer

The seventh specialist is dependency-analyzer. Its job is to inspect the project's dependency manifest and report on any dependencies that are outdated, vulnerable, unused, or duplicated.

name: dependency-analyzer

description: Analyzes the project's dependencies for outdated versions, known vulnerabilities, unused packages, and duplicate or redundant entries. Use when the user asks "what's in package.json," "audit my dependencies," "any vulnerable packages," or as part of a periodic hygiene check.

tools: Read, Grep, Glob, Bash

model: sonnet

You are a senior dependency hygiene engineer. Your job is to keep a project's dependency surface clean, secure, and minimal.

Your method:

1. Read the dependency manifest (package.json, requirements.txt, Cargo.toml, etc.).
2. Read the lockfile if one exists.
3. Identify, in order of priority:
 - Dependencies with known security vulnerabilities (run the project's audit tool if available)
 - Major-version updates available for actively maintained dependencies
 - Dependencies that appear in the manifest but are not imported anywhere in the source
 - Duplicate or redundant dependencies (two libraries doing the same job)
 - Dependencies with significant licensing concerns
4. Produce a report grouped by category, with each item including the package name, current version, latest version (if relevant), and a short recommendation.

What you do not do:

- You do not update the manifest. You only report.
- You do not flag every outdated minor version. Focus on major-version drift and security concerns.
- You do not invent vulnerabilities. If your audit tool produces no findings, say so plainly.
- You do not pad. A clean dependency tree gets a one-line judgment, not a pseudo-report.

End with a one-sentence judgment of overall dependency health: clean, minor concerns only, or significant cleanup recommended.

The eighth and final specialist is migration-planner. Its job is to produce a step-by-step plan for a non-trivial migration, such as moving from one framework to another, upgrading a major dependency, or restructuring a substantial part of a codebase.

name: migration-planner

description: Produces a step-by-step migration plan for a substantial change such as a framework upgrade, language version bump, library replacement, or architectural restructuring. Use when the user describes wanting to migrate from one technology to another or asks for a plan to upgrade something significant.

tools: Read, Grep, Glob

model: opus

You are a senior migration engineer. Your job is to break a substantial migration into a sequence of safe, reviewable steps that the team can execute incrementally.

Your principles:

- A good migration plan is a sequence of small, ship-able changes that can each be reviewed and reverted independently. Big-bang migrations fail.
- Every step has a clear acceptance criterion. The team should know when each step is complete and whether it succeeded.
- Risk goes at the front of the plan, not the back. Identify the riskiest part of the migration and propose a small experimental change that surfaces the risk early.
- Honest plans assume some steps will fail. Include a rollback strategy for each non-trivial step.

Your method:

1. Read the relevant code to understand the current state.
2. Read enough about the target state to understand the destination.
3. Decompose the migration into a numbered sequence of steps. Aim for steps that take a single PR each, no more.
4. For each step, document: what changes, what does not change, the acceptance criterion, the rollback strategy, and the estimated effort in person-days.
5. Identify the riskiest step and propose an early experiment that reduces the risk before the team commits to the full plan.
6. End with a one-paragraph executive summary suitable for a team meeting.

What you do not do:

- You do not write migration code. You produce the plan.
- You do not commit to specific dates. Effort estimates in person-days are appropriate; calendar dates depend on team capacity.
- You do not pad with general advice about migrations. Your output is specific to this migration.

End with the executive summary.

The library is now eight specialists strong. The reader who has installed all of them has, in roughly an hour of work, assembled a small but capable team that can be dispatched on demand whenever her work crosses one of their domains. The team is patient. It does not get tired. It does not have meetings. Each specialist is excellent at one thing and confidently silent about everything else. The developer's job is to be the editor, dispatching the right specialist to the right work and synthesizing the results into the unified outputs her real life demands.

Extending the Library

A note on extension. The eight specialists above are starting points, not endpoints. As the developer lives with them, she will discover that her work has shapes the eight do not cover, or that one of the eight is slightly off-fit for her specific stack. The fix is the same as for skills: edit the file. Each subagent is a single Markdown file. Editing it takes thirty seconds. Adjusting the role definition, expanding the description, narrowing the tools, refining the boundary, all of these are small changes that the developer can make as her understanding of her own workflow deepens.

A second note on adding new specialists. The developer who finds, after a month or two of using the eight specialists, that a recurring kind of work is not covered by any of them, has the raw material for a ninth specialist. The pattern by which she should design it is by now familiar: name the role specifically, narrow the tools to what the role needs, write boundary language that stops the role from drifting, and test the result on three representative inputs. Within an hour, she has a ninth specialist on the team, and her library has grown along the contour of her actual work.

A third note on team-level subagents. Several of the specialists above will be useful to other developers in the reader's team or company. The mechanism for sharing is the same as for skills: copy the subagent file into the project's `.claude/agents` directory and commit it. From that point forward, every developer who clones the project receives the specialist automatically. This is how subagent libraries spread within organizations, and how a thoughtful library built by one developer becomes a standing resource for an entire team. The library committed to a project repository is a quiet but real form of leverage. It travels with the code, it activates on the first session, and it raises the floor of what every developer working in that project can produce.

The chapter that follows is the climax of Part Three. With skills installed and subagents installed, the question becomes how the two systems compose. Subagents can use skills. Skills can be designed to fit specific subagents. The combination is where the architectural depth of Claude Code becomes most visible, and the next chapter is dedicated entirely to it. The reader who has installed both libraries by the end of Part Two and Part Three is now in possession of the components for a substantial leap in capability. The next chapter shows how to assemble

those components into the integrated system that has been the destination of the book all along.

Subagents and Skills: The Composition Pattern

“The whole, when properly composed, contains capabilities that none of its parts possess in isolation. This is the meaning of architecture.”

attributed to a systems engineer, anonymous

This chapter is where the two pillars built so far in the book begin to reinforce each other in a way that produces capabilities neither could produce alone. Skills, the subject of Part Two, taught Claude how to do specific tasks consistently. Subagents, the subject of Part Three, gave Claude a team of specialists, each in its own context. The composition of the two is the architectural depth that the book has been building toward, and this chapter teaches it explicitly.

Begin with the basic mechanic. Skills are available everywhere. They are loaded at the level of the Claude Code installation, and any Claude process running in that installation, including any subagent, can activate any applicable skill. A subagent does not need a special configuration to access skills; it simply needs to have a description that matches the kind of work where a skill would naturally fire. The composition is not bolted on. It is the default behavior of the two systems when they are present together, and the developer’s job is to design with the composition in mind from the start.

A worked example illustrates the power. The developer has a test-writer subagent installed, presented in the previous chapter. The developer also has a project-specific skill called test-conventions, which encodes how tests are organized and named in this particular codebase, including the choice of test framework, the file layout, the mocking style, the use of fixtures, and the team’s preferred patterns for assertion phrasing. The test-conventions skill lives in the project’s `.claude/skills` directory; the test-writer subagent lives in the user’s `~/.claude/agents` directory.

Now imagine the developer dispatches the test-writer subagent on a new function. The subagent receives its brief: write tests for this function. The subagent begins its work. It reads the function. As it considers the

structure of the tests it is about to write, it recognizes that the request matches the description of the test-conventions skill, and it loads that skill into its own context. The subagent now has, simultaneously: its own role as a senior test engineer, its own focused tools and isolated context, and the project-specific knowledge encoded in the skill. The tests it produces are written in the project's exact conventions, by a specialist who knows how to write good tests in general, in a context that is not polluted by anything outside the test-writing task.

Composition in Action

The result is strikingly better than what either component could produce alone. A skill alone, fired in the main session, would shape the output toward project conventions, but the output would be subject to context pollution from whatever else the main session was working on. A subagent alone, dispatched on the same task, would produce well-isolated, generally good tests, but it would not know the project's specific conventions and would have to guess. The combination has both: convention-correct tests, produced in a clean context, by a specialist who treats the work seriously. This is the composition pattern in its simplest form, and once a developer sees it work even once, she will not want to design subagents that do not know how to use skills.

There is a design principle that follows from this and that should guide every subagent the developer writes from now on. Subagents should be designed as roles, not as repositories of project knowledge. A test-writer subagent should know how to be a senior test engineer in general; it should not know that the project uses Vitest rather than Jest, or that mocks should be placed in a specific directory, or that the team prefers a specific assertion library. Those are project facts, and they belong in skills. The subagent provides the role; the skill provides the project-specific knowledge. When the role meets the project, the composition produces the right behavior.

This principle has a second-order effect that is worth naming. A subagent that contains project-specific knowledge becomes specific to that project, and its usefulness drops as soon as the developer moves to a different project. A subagent that contains only the role, with project-specific knowledge factored into skills, is portable. The same test-writer subagent works in five different projects, because the role is the same in

each, and the project-specific conventions are loaded as skills from each project's own `.claude/skills` directory. A small library of well-designed, role-only subagents at the user level, combined with project-specific skills in each project, gives the developer the best of both worlds: stable, portable specialists, and project-appropriate behavior in every project she works in.

A second worked example raises the composition to a more sophisticated level. The developer has installed the security-auditor subagent from Chapter Eleven, the architecture-reviewer subagent from Chapter Thirteen, and a project-specific skill called `auth-conventions`, which encodes the codebase's authentication patterns, token handling, and security-sensitive practices. The skill is not project-specific knowledge that the security-auditor needs to do its job in general; the security-auditor knows about OWASP Top 10 and common vulnerability patterns regardless of project. But the skill encodes the way this specific codebase chooses to handle authentication, which is information the security-auditor needs in order to flag deviations from the established pattern as concerns rather than treating them as ordinary alternatives.

When the developer dispatches the security-auditor on a change that touches authentication, two things happen simultaneously. The subagent activates its general security expertise. The subagent also loads the `auth-conventions` skill, because the description matches. The subagent now has both: the general security knowledge that says hardcoded credentials are bad, and the specific project knowledge that says this codebase uses a token-based scheme with refresh rotation handled by a particular module. The audit it produces is doubly informed. It catches general security issues. It also catches deviations from the project's specific patterns, which a generalist would not have known were patterns at all.

An Advanced Composition Pattern

There is a more advanced composition pattern worth introducing here, although its full development belongs in Part Five. The orchestrator pattern from Chapter Twelve, in which the main session dispatches several subagents in parallel and synthesizes their outputs, can be combined with skill composition. Each parallel subagent can activate its own appropriate skills. A four-way code review dispatched as four parallel subagents, each of which knows how to use the relevant project skills, produces a deeper,

more project-aware review than any single channel could produce. The four-by-four matrix, four subagents and four skills, is the structure of a serious engineering review process, and the composition pattern of this chapter is what makes it a single concept rather than two unrelated ones.

A practical caveat. Not every subagent benefits from skill composition, and the developer should not over-engineer the design. Some specialists are useful as pure role containers and should not be paired with skills at all. The migration-planner subagent from the previous chapter, for example, produces a generic migration plan that is shaped more by the specifics of the migration than by ambient project conventions. There is little benefit in trying to load skills into it. The composition pattern is a tool, not a doctrine. The developer should reach for it when it produces real value and leave it alone when it would only complicate a workflow that does not need complication.

A second practical caveat concerns description engineering. For the composition to fire correctly, the skill descriptions and the subagent descriptions both need to be written well. The subagent description determines whether the subagent is dispatched. The skill description determines whether the skill activates inside the dispatched subagent. Both layers of triggering need to work for the composition to produce its intended behavior. The same disciplines from Chapter Six apply here: canonical statement, trigger language, boundary, input expectation. The developer who has internalized those disciplines can debug failed compositions quickly. The developer who has not will sometimes find that the composition does not work and will not be sure why.

A third worked example will deepen the pattern. The developer has installed the documentation-writer subagent and a project-specific skill called doc-style-guide, which encodes the team's conventions for technical writing: the preferred section structure, the rules about voice and tense, the format of code examples, and the policy on what level of detail is appropriate for different audiences. The skill is project-specific because every team has its own documentation style, and a documentation-writer subagent that wrote in some generic style would produce documentation that did not fit the team's existing corpus.

The composition produces a documentation-writer that is doubly informed. The subagent provides the role: a senior technical writer who knows how to explain code clearly, how to choose examples, and how to keep documentation honest. The skill provides the project conventions: section ordering, voice, tense, example format. When the developer dispatches the subagent on a new module that needs documenting, both fire together. The output reads like good technical writing in the team's house style, because both layers of expertise are present at once. The developer did not have to brief the subagent on the team's conventions, because the skill brought them automatically. She did not have to teach the skill how to write good prose, because the subagent provided that. Each system did its job, and the composition produced output better than either alone.

The Architectural Principle

There is a deeper architectural principle visible here, which is worth naming explicitly because it will guide every later chapter of the book. The composition pattern is an instance of the broader principle of separation of concerns. Skills hold project knowledge. Subagents hold roles and isolate contexts. Neither system tries to do the other's job. When a workflow needs both kinds of capability, the systems compose, and the composition produces capabilities that neither alone could provide. This is the same principle that underlies every well-architected piece of software, applied to the design of a Claude Code environment. Developers familiar with separation of concerns in their own code will recognize the principle immediately and find the composition pattern intuitive. Developers unfamiliar with it will, by working through this part of the book, internalize the principle by seeing it applied to a domain where its benefits are immediate and visible.

A practical implication of this principle is the design discipline for adding new capabilities to a Claude Code environment. When the developer notices a recurring kind of work that her current configuration handles poorly, she should ask, is the gap a knowledge gap or a role gap. If she finds herself wishing the environment knew something specific, that is a knowledge gap, and the answer is a skill. If she finds herself wishing a different kind of attention were being applied to a kind of work, that is a role gap, and the answer is a subagent. Sometimes the gap is both, and the

answer is a composition. Asking the right question first prevents the developer from building skills that should have been subagents, or subagents that should have been skills, both of which are common mistakes that produce environments that work but feel awkward. Asking the right question consistently produces environments that fit the developer's work cleanly, because each capability lives in the right place.

A second design discipline that emerges from the composition pattern concerns versioning. Skills and subagents that are designed to compose with each other should evolve together. If the developer changes the conventions encoded in a skill, the subagents that depend on that skill may need to be checked to ensure their behavior still aligns. If the developer reshapes a subagent's role, the skills designed to fit that role may need to be revisited. This coupling is loose, not tight, and the systems remain independently functional, but the composition is most reliable when the developer treats related skills and subagents as parts of a single design rather than as unrelated artifacts. The discipline is to version-control the entire .claude directory, to commit related changes together, and to review skill and subagent changes side by side when they are likely to affect each other.

One more reflection on the experience of the composition pattern, before this chapter closes. The first time a developer sees a skill activate inside a subagent and watches the combined behavior produce something neither could have produced alone, the experience tends to be memorable. There is a moment of recognition in which the architecture stops feeling abstract and starts feeling like a real engineering substrate. The skill library and the subagent library, which until then had felt like two separate collections of useful artifacts, become something larger. They become the components of an environment, and the environment becomes the place where the developer's real work happens. This shift in perception is not just aesthetic. It is the shift from using Claude Code to engineering with Claude Code, which is the central promise of this book and the moment the entire framework starts paying back the investment of learning it.

A reflection on the architecture, before this chapter and this part of the book end. The design choices that made skills and subagents into separate systems, rather than merging them into a single mechanism, were deliberate. Skills are extensible knowledge. Subagents are specialized roles

with isolated contexts. The two are different kinds of thing, and treating them as different kinds of thing keeps the design clean. A merged system would have been simpler at first glance and worse in practice. The cost of having two systems is the small overhead of learning both. The benefit of having two systems is that each remains focused on what it does best, and the composition between them is principled rather than tangled. The reader who has worked through the last six chapters now has both systems in working memory and in installed form, and she has seen how they reinforce each other to produce capabilities neither could produce alone.

Second-Order Benefits

There is a second-order benefit to the composition pattern that becomes visible only after a developer has lived with a composed library for a few months. The benefit is that the library becomes self-documenting in a way that monolithic configurations are not. A new developer joining the project can read the `.claude/skills` directory to learn the project's conventions; reading the `SKILL.md` files is essentially reading a structured guide to how the team prefers to work. The same new developer can read the `.claude/agents` directory to learn what specialist roles the team treats as standing capabilities; reading the subagent definitions is essentially reading the team's job descriptions for its AI-assisted workflow. Together, the two directories form a piece of working documentation that is more accurate than any wiki, because the documentation is the configuration the tool actually uses. This self-documenting property is a substantial benefit for any team that grows, because it lowers the cost of onboarding new members and reduces the gap between what the team says it does and what the team actually does.

A third reflection concerns the relationship between this composition and the third pillar of the framework, hooks, which is the subject of the part of the book that follows. The composition of skills and subagents that this chapter has taught is internal to the agent loop. Both skills and subagents operate within the probabilistic space of the language model. They shape what the model does, but they do not impose deterministic constraints on it. Hooks are different. Hooks live outside the loop and impose deterministic constraints on it. The fully engineered Claude Code environment that the book is building toward includes all three pillars working together: skills

providing capability, subagents providing specialization, hooks providing the deterministic edges that make the whole environment safe and reliable to depend on. The reader who has finished this chapter has half of the architecture in place. The next part of the book completes it.

A short observation is worth recording about the experience of working in a fully composed environment, although the reader will only verify the observation for herself once she has built one. Developers report that the felt difference between Claude Code without a configured environment and Claude Code with a thoughtful skills-plus-subagents composition is larger than they expected. The unconfigured tool feels capable but generic. The configured tool feels specific to the developer's work in a way that is initially surprising. Tasks that used to require explicit briefing now run themselves. Reviews that used to take ten minutes finish in three. The output that used to need editing for tone and convention arrives already in the right voice. The whole working day reshapes itself around the new floor of capability, and the developer's attention rises to fill the new space the environment has created. This is the practical payoff of the composition pattern, and it is the payoff that justifies the investment of time in building skill and subagent libraries thoughtfully rather than casually.

Part Three ends here. The reader has, over five chapters, learned the architecture of subagents, mastered the design of single specialists, internalized the orchestrator pattern that turns sequential work into parallel work, installed a library of eight working specialists, and seen how skills and subagents compose to produce the deeper architectural power of the system. The second pillar of the framework is now in place. The third pillar, hooks, is the subject of the next part of the book. Hooks are how the probabilistic system gets wrapped in deterministic edges, how the developer ensures that the things that must happen always happen, and how the boundaries of the engineered environment become unbreakable at the points where breakage would matter most. That work begins in the chapter ahead.

Before turning the page, a short closing thought. Each part of this book has built one layer of the larger architecture. Part One established the mental model. Part Two installed the first pillar. Part Three has installed the second pillar and shown how the first two pillars compose. The pattern of work that the book asks of the reader is, in this respect, accumulating. Each part builds on what came before, and each successive layer makes the

previous layers more powerful by giving them a richer environment to operate in. The hooks that the next part introduces will, in turn, make the skills and subagents already installed more reliable, because the deterministic guarantees that hooks provide will protect the probabilistic systems below them from the failure modes those probabilistic systems cannot prevent on their own. The architecture is genuinely hierarchical, and the experience of building the upper layers is, over time, the experience of giving the lower layers a sturdier foundation to stand on.

PART FOUR

Hooks: Claude's Deterministic Enforcement System

The probabilistic model needs deterministic edges. Hooks are how the edges get built.

CHAPTER FIFTEEN

Hooks Fundamentals: Deterministic Behavior in a Probabilistic System

“Trust, but verify. The verification is what makes the trust possible.”

attributed to Ronald Reagan, who borrowed it from a Russian proverb

Skills and subagents, the two pillars covered so far, share a fundamental property. Both operate inside the probabilistic space of the language model. Both shape what Claude does, but neither imposes a hard constraint on what Claude must or must not do. A skill, however well written, instructs Claude through prose; the model interprets the prose and behaves accordingly, but the model retains the freedom to interpret. A subagent runs in its own context, but it is still a language model, and its behavior is still shaped by probability rather than by mechanical rule.

For most of the work a developer asks Claude Code to perform, this probabilistic shaping is exactly right. The flexibility of the model is what allows it to handle the open-ended variety of real requests. But there is a class of requirements where probabilistic shaping is insufficient. The class is small but important, and the requirements in it share a common shape: the failure rate must be zero. Tests must run before commits. Secrets must never reach disk. Production configuration must never be edited without a human in the loop. Linters must run after every code edit. For these requirements, no amount of careful prompting is good enough, because careful prompting can fail two percent of the time and the requirement does not tolerate two percent failure.

This chapter introduces the third pillar of the framework, which exists precisely to handle this class of requirement. Hooks are user-defined shell commands that Claude Code executes automatically at specific moments in its lifecycle. They run outside the language model. They cannot be talked out of running by clever prompts. They cannot be forgotten by Claude. They run, and their result either allows the lifecycle to continue or interrupts it. The discipline of using hooks well is the discipline of identifying which requirements need this kind of guarantee and then encoding the guarantee in shell commands that Claude Code will honor whether the model wants to or not.

Begin with the mechanics. A hook is configured in a settings file. The most common location is the project's settings file, which lives at `.claude/settings.json` in the project root. User-level hooks, which apply to every project, live at `~/.claude/settings.json`. The settings file is JSON, and the hooks live under a top-level `hooks` key. Each hook entry specifies an event, an optional matcher that filters which tool calls the hook applies to, and a

command to run when the event fires. The simplest possible hook configuration looks like this.

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          {
            "type": "command",
            "command": "npx prettier --write \"$CLAUDE_TOOL_INPUT_FILE_PATH\""
          }
        ]
      }
    ]
  }
}
```

Reading the Configuration

Read this configuration carefully, because the structure is identical for every hook the rest of this part of the book will introduce. The top-level hooks key holds an object whose keys are event names. The event name in this case is `PostToolUse`, which fires after a tool call has completed successfully. Under that event name is an array of hook configurations. Each configuration has an optional matcher, which uses regular expression syntax to filter which tools this hook should apply to; here, `Edit` and `Write` are the only matched tools, so the hook fires after edits and writes but not after, say, `Bash` or `Read` calls. The hooks array under that contains the actual commands to execute. Each command has a type, almost always `command` for shell execution, and the command itself, which is run with the standard shell.

The hook configuration above is small but functional. Its effect: every time Claude Code edits or writes a file, the file is automatically passed through Prettier, which formats it according to whatever Prettier configuration the project has. The developer never has to remember to run Prettier. Claude never has to remember either. The hook remembers, because the hook is not part of the language model's loop. The hook is

wrapped around the loop, at a specific point in the lifecycle, and it runs every time the lifecycle reaches that point, no matter what.

The lifecycle is the grid of available moments where hooks can attach. Every Claude Code session passes through a sequence of named events, and a hook can be bound to any of them. The most common events, and the ones that almost every working developer eventually uses, are these. `PreToolUse` fires before a tool is about to be invoked, with the tool's name and input available to the hook. `PostToolUse` fires after a tool has completed successfully. `UserPromptSubmit` fires when the user submits a prompt, before Claude processes it. `Stop` fires when Claude finishes generating a response. `SubagentStop` fires when a subagent completes its work, before its result returns to the main session. `SessionStart` fires when a new session begins or is restored. `SessionEnd` fires when a session terminates. `Notification` fires when Claude needs to alert the user about something, such as needing approval. `PreCompact` fires just before Claude compacts its context to save space.

Each event has a slightly different shape of input passed to the hook on standard input as JSON, and a slightly different shape of expected output. `PreToolUse` hooks receive the tool name, the tool input, and the session context, and can decide to allow, deny, ask the user about the action, or defer the decision back to the agent loop. `PostToolUse` hooks receive the tool name, the input, and the response, and can issue feedback to Claude or block subsequent steps. `UserPromptSubmit` hooks receive the prompt text and can either block it or inject additional context for Claude to read. `SessionStart` hooks receive minimal context but can produce output that gets injected into the session's starting context, which is useful for loading project-specific reminders or recent git information at every session start.

The mechanism by which a hook signals its decision is straightforward. The hook is a shell command. Its standard output and standard error are captured. Its exit code is checked. Exit code zero means success, and the lifecycle continues without modification. Exit code two has a special meaning: in `PreToolUse` hooks the action is blocked; in `PostToolUse` hooks the action has already happened, so exit code two cannot block it but instead surfaces the standard error output back to Claude as feedback, and whatever the hook printed to standard error is surfaced back to Claude as feedback about why the block happened. Other non-zero exit codes are treated as

errors but do not block; they are logged. For richer control, the hook can also print structured JSON to standard output, which lets the hook approve or deny the action explicitly, modify the tool input before it runs, or inject additional context. The simple exit-code path covers most needs. The JSON path is for when the hook needs to do something more sophisticated than a binary allow or block.

Exit Codes in Action

A worked example using exit codes will make the blocking pattern concrete. Suppose the developer wants to ensure that Claude Code never edits the `.env` file in any project. A small hook accomplishes this. The hook is configured under `PreToolUse`, with a matcher of `Edit` or `Write`, and the command is a small shell expression that reads the tool input on standard input, checks whether the file path is `.env`, and exits with code two if so.

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          {
            "type": "command",
            "command": "jq -e '.tool_input.file_path | test(\"\\\\\\\\\\.env$\")' >/dev/null && { echo 'Editing .env files is not allowed.' >&2; exit 2; } || exit 0"
          }
        ]
      }
    ]
  }
}
```

The shell expression in that command is dense, but its logic is simple. It reads the JSON input from standard input, uses `jq` to test whether the tool input's file path matches the regular expression for `.env` files, and if it does, prints an error message to standard error and exits with code two, which blocks the operation. If the file is not a `.env` file, the hook exits with code zero, allowing the operation. The model receives the standard error message as feedback and can explain to the user that the `.env` file is protected. The user gets a deterministic guarantee: no matter what Claude wants to do, no

matter how the user phrased the request, no matter what the system prompt or skills say, the `.env` file cannot be modified.

That guarantee is the whole point. The probabilistic model is left to its probabilistic work, and the deterministic requirement is handled deterministically, in its proper place. The model is freed from having to be vigilant about something it does not need to be vigilant about, because the hook is vigilant on its behalf. The developer is freed from having to verify on every interaction, because the verification is mechanical.

A second mechanism worth knowing about is the JSON output format, which the hook can use for richer control than a binary allow or block. When a hook writes a JSON object to standard output and exits with code zero, Claude Code parses the JSON for instructions about how to handle the action. The most useful field for `PreToolUse` hooks is `hookSpecificOutput`, which carries a `permissionDecision` of allow, deny, or ask, plus an optional `updatedInput` field that lets the hook modify the tool input before it runs. This last capability is striking on first encounter. A hook can intercept a tool call, decide that the call is acceptable but the input should be modified, and rewrite the input on the fly. The model never sees the modification. The tool runs with the rewritten input. A hook that wraps every Bash call to ensure that `npm install` runs with the `--no-save` flag would use this mechanism, intercepting the call, rewriting the command to add the flag, and letting the modified command proceed. The capability is powerful, and it should be used sparingly, because invisible modifications to tool input can produce confusing sessions if the developer forgets the hook is there. But when the use case is right, the JSON output format is the cleanest way to express it.

The Settings Hierarchy

A short word on the settings hierarchy. Hooks can be configured at three levels: enterprise policy, user-level, and project-level. Enterprise policy settings are managed by an organization's administrators and apply to every project a developer opens. User-level settings live in `~/.claude/settings.json` and apply to every project the user opens. Project-level settings live in `.claude/settings.json` at the root of a specific project and apply only inside that project. There is also a project-local file at `.claude/settings.local.json` (which the project's `.gitignore` should exclude), intended for personal hooks that the developer wants in this project but does not want to commit to the

repository. The hierarchy is additive: hooks from all applicable levels run on every event, in a defined order, and any one of them can block. The developer should think carefully about which level a given hook belongs at. A security hook that protects against `rm -rf` belongs at the user level, because it should apply everywhere. A code-formatting hook that runs Prettier with project-specific configuration belongs at the project level, because the configuration is project-specific. A personal hook that sends desktop notifications when Claude needs input belongs in `settings.local.json`, because it is the developer's personal preference and not something her teammates need or want.

Understanding where each hook belongs is more than an organizational nicety. It directly affects how the hook is shared, how it is reviewed, and how easily it can be adjusted. A hook in the wrong place produces friction long after it was installed. A user-level hook that should have been project-level forces every other project to deal with conventions that only one project actually wants. A project-level hook that should have been user-level fails to protect the developer when she switches to a different project. A personal hook in the wrong place gets committed to the repository where her teammates encounter it as an unexplained presence. None of these mistakes is catastrophic, but each of them produces low-grade ongoing annoyance, and the annoyance can be avoided entirely by spending thirty seconds on the placement decision when the hook is being installed.

The relationship between hooks and the agent loop deserves explicit attention, because it is the relationship that makes the entire architecture work. The agent loop, recall from Chapter Two, is the cycle of think, act, observe that the language model runs through every turn. Every tool call happens inside the act phase. Every observation of a tool result happens inside the observe phase. Hooks attach to the boundaries of these phases. `PreToolUse` hooks fire just before the act phase invokes a tool. `PostToolUse` hooks fire just after the act phase has completed and the result is being returned to the observe phase. The hooks are, in this sense, the boundary between the model's probabilistic decisions and the deterministic environment that surrounds them. They are not part of the model's loop. They are part of the framework that the loop runs inside, and the framework can impose constraints that the model has no way to evade.

A small clarification about the interactive `/hooks` command is worth including, because new users sometimes overlook it and pay an unnecessary cost. The `/hooks` command, run inside Claude Code, opens an interactive editor for the hook configuration. The editor lists every event, shows the hooks currently bound to each event, and offers menu options to add, modify, or remove hooks without having to edit the JSON configuration file directly. For developers who are new to hooks and not yet comfortable hand-editing JSON, the interactive editor is the friendlier path. For developers who are comfortable with JSON, direct editing of the settings file is faster, especially for non-trivial hooks. Both paths produce identical results, and the developer can switch between them depending on what is easier for the specific change she is making. The interactive editor is also useful as a verification tool: running `/hooks` in a session is a quick way to confirm that a hook configuration is loaded and active, which can save time when debugging why a hook seems not to be firing. Two related slash commands are worth knowing about as well. The `/agents` command lists the subagents available in the current session and lets the developer inspect their definitions interactively. The `/doctor` command reports on the current installation and surfaces the most common configuration problems before they become a debugging session. A common pitfall worth flagging here is forgetting that newly added hooks require a session restart before they take effect; configuration changes do not retroactively apply to a running session, and the developer who edits her hooks and then expects the changes to be live should restart Claude Code or open a new session before testing.

A worked debugging walkthrough is worth including, because hooks can fail in subtle ways and the developer who has a debugging procedure in working memory will save herself hours when something goes wrong. The walkthrough below is from a real session. The developer had configured a `PostToolUse` hook to run Prettier after every file write. She tested it, it worked, and she committed the configuration. Two days later, she noticed that Prettier had stopped running. The files Claude was writing were no longer being formatted, even though the hook configuration appeared unchanged.

A Debugging Walkthrough

She approached the problem methodically. The first step was to verify the hook was loaded. She ran the `/hooks` command inside Claude Code, which lists all configured hooks for the current session. The Prettier hook appeared in the list, which meant the configuration was being read. The second step was to verify the hook was running. She added a small log statement to the hook itself, writing a timestamp to a file every time the hook fired. She made an edit, and the log file gained a new entry. The hook was firing. The third step was to verify the hook was succeeding. She checked the file the hook had been formatting and saw that the formatting had not been applied. The hook was firing but not completing its work.

The diagnosis came on the fourth step. She invoked the hook directly from her terminal, passing in a JSON input that mimicked what Claude Code would send. The hook ran, attempted to invoke Prettier, and failed silently. The reason was that `npm run prettier` required the project's `node_modules` to be installed, and a recent dependency cleanup had removed Prettier from the project's dependencies. The hook itself was correct. The environment it ran in had drifted out from under it. She reinstalled Prettier, ran her test again, and the formatting returned.

The lesson the developer took away was that hooks fail silently more often than they fail loudly, and the procedure for debugging them is to walk down the chain: configuration loaded, hook firing, hook executing, hook succeeding. Each step has a verification. Each verification narrows the problem space. The whole procedure takes ten minutes and resolves the large majority of hook problems, because the failure is almost always at one of the four checkpoints, and the checkpoints are easy to test in isolation.

A reflection before this chapter closes, on where hooks fit in the broader architecture of the book. Skills shape what Claude knows. Subagents shape who, in effect, is doing the work. Hooks shape what is permitted. The three together cover capability, specialization, and boundary. The fully engineered Claude Code environment uses all three, with hooks providing the deterministic edges that make it possible to trust the probabilistic systems in the middle. A developer who has installed skills and subagents but no hooks has a powerful environment that still depends on the model's judgment for safety-critical concerns, which is a reasonable trade-off but a real one. A developer who adds even three or four well-chosen hooks transforms her environment from a powerful assistant into a

system she can rely on for things that genuinely matter, because the things that genuinely matter are now protected by mechanism rather than by hope.

The next chapter takes hook design from these mechanics into design patterns. The three archetypes that cover almost every useful hook are gates, guards, and notifiers, and the chapter ahead develops each pattern in depth, with worked examples ready to install. The reader who finishes the next chapter will have a small but capable set of hooks in her environment and the design vocabulary to add more whenever her work uncovers a new requirement that deserves deterministic handling.

Hook Design Patterns: Gates, Guards, and Notifiers

“The shape of a tool is the shape of the problem it solves. Three problems, three shapes, no more.”

Henry Petroski, paraphrased

This chapter takes the hook mechanics of the previous chapter and develops them into three design patterns that, in combination, cover almost every hook a working developer will ever want to write. Naming the three patterns explicitly is valuable because it gives the developer a vocabulary for thinking about her hook design and a checklist for asking, when a new requirement arises, which of the three shapes the requirement most resembles. The three patterns are gates, guards, and notifiers, and each one solves a specific class of problem in a specific way. Once the patterns are in working memory, designing a new hook becomes a matter of recognizing which pattern fits and following the conventions for that pattern.

Begin with gates. A gate is a hook that runs before an action and decides whether the action is allowed to proceed. The canonical lifecycle event for gates is `PreToolUse`, which fires before a tool call is dispatched. The gate inspects the tool input, applies whatever rule the developer cares about, and either lets the action through or blocks it. The blocking mechanism, as established in the previous chapter, is exit code two, with whatever explanation the developer wants surfaced to Claude on standard error.

Gates are the most common and most consequential pattern in any hook library, because they are the pattern that turns wishes into rules. A developer who wishes that Claude would not commit directly to the main branch can encode the wish as a wish, by writing it into `CLAUDE.md` or into a skill, and Claude will follow the wish most of the time. The developer who wants the wish to become a rule writes a gate. The gate runs before any `git push` tool call, inspects the command for direct pushes to main, and exits with code two if the push targets main without an explicit override.

The wish is now a rule. Claude cannot push to main, full stop, regardless of what the user types or how the request is phrased.

A worked gate example will make the pattern concrete. The gate below blocks any Bash command that contains the dangerous `sudo rm -rf` pattern, because the developer has decided that this pattern should never be executed by Claude Code, no matter what.

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Bash",
        "hooks": [
          {
            "type": "command",
            "command": "jq -r '.tool_input.command' | grep -qE 'sudo[[:space:]]+rm[[:space:]]+-rf'
            && { echo 'Refusing to run sudo rm -rf. This pattern is blocked at the system level.' >&2; exit 2;
            } || exit 0"
          }
        ]
      }
    ]
  }
}
```

The shell expression reads the tool input as JSON, extracts the command, checks for the dangerous pattern with `grep`, and if it matches, surfaces a clear refusal message and exits with code two. The block is total. The model cannot bypass it, because the gate is not a model concern. It is a system concern, enforced before the model's requested action ever reaches the shell.

The Discipline of a Good Gate

Gates have a discipline worth naming. A good gate has three properties. It is targeted; it fires only on the specific tool calls and inputs it is designed to evaluate, not on every Bash command in the universe. It is fast; it should add only a few milliseconds to any tool call, because hooks fire constantly and slowness in a hook becomes slowness in every interaction. It is informative; when it blocks an action, the message it surfaces explains why the block happened in language the developer or the model can understand

and act on. A gate that blocks without explanation produces frustrating sessions. A gate that explains its reasoning turns a refused action into a learning moment.

Move to guards. A guard is a hook that runs after an action and enforces consistency. The canonical lifecycle event for guards is `PostToolUse`. The guard inspects the result of the action and either accepts it as-is, modifies the surrounding state, or signals back to Claude that the action's result was insufficient and more work is needed. The Prettier hook from the previous chapter is a classic guard: it runs after a file is written, formats the file according to project rules, and ensures that whatever Claude wrote ends up in the canonical project format whether Claude knew the format or not.

Guards differ from gates in their relationship to the action. A gate is preventive; it stops things from happening. A guard is corrective; it lets things happen and then ensures the result fits the project's conventions. Guards are usually less dramatic than gates, because they do not block, but they are at least as valuable in practice. A library of well-chosen guards makes the project's output consistent in ways that no amount of careful prompting can match, because the guards run on every relevant action and apply their corrections without requiring anyone to remember.

A worked guard example raises the Prettier pattern to something more complete. The guard below runs after any file edit or write, formats the file with Prettier, and additionally runs ESLint with the auto-fix flag to apply automatic linting fixes.

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write|MultiEdit",
        "hooks": [
          {
            "type": "command",
            "command": "FILE=$(jq -r '.tool_input.file_path'); npx prettier --write \"$FILE\" 2>/dev/null || true"
          },
          {
            "type": "command",
            "command": "FILE=$(jq -r '.tool_input.file_path'); npx eslint --fix \"$FILE\" 2>/dev/null || true"
          }
        ]
      }
    ]
  }
}
```

```
    }  
  ]  
}  
]  
}  
}
```

Two hooks fire in sequence under the same matcher. The first runs Prettier. The second runs ESLint with auto-fix. Both extract the file path from the JSON input, both run their tool against the path, and both swallow errors and exit cleanly so that a transient formatter failure does not interrupt the session. The result: every file that Claude touches ends up formatted and lint-clean before the next prompt, regardless of what Claude wrote. The project's style is enforced mechanically, and the developer never has to think about formatting again.

Notifiers in Detail

Move to notifiers. A notifier is a hook that runs at a lifecycle event for the purpose of producing some output to a system other than Claude Code itself. The output might be a desktop notification, a log file entry, a Slack message, an audit record, or anything else that the developer wants the world outside the session to know about. Notifiers do not block. They do not modify behavior. They simply observe the lifecycle and react.

The most common notifier event is the `Notification` event itself, which fires when Claude wants to alert the user about something, often a permission request. The developer can attach a notifier to this event that produces a desktop pop-up, so that she does not have to be staring at the terminal to see when Claude needs her attention. `Stop` is another common notifier event; a notifier on `Stop` can log every completed task to an audit file or send a brief summary to a team channel. `SessionEnd` notifiers can save session statistics. `PostToolUse` notifiers can append every Bash command Claude ran to a permanent log, which is useful both for audit and for retrospective debugging.

A worked notifier example combines two of these patterns. The configuration below sends a desktop notification when Claude needs the user's input, and logs every Bash command Claude runs to a persistent log file.

```

{
  "hooks": {
    "Notification": [
      {
        "hooks": [
          {
            "type": "command",
            "command": "notify-send 'Claude Code' 'Awaiting your input'"
          }
        ]
      }
    ],
    "PostToolUse": [
      {
        "matcher": "Bash",
        "hooks": [
          {
            "type": "command",
            "command": "jq -r '\"\\(.session_id) \\(.tool_input.command)\\\"' >> ~/.claude/bash-
history.log"
          }
        ]
      }
    ]
  }
}

```

The first hook fires whenever Claude needs the user's attention; the developer's desktop pops up a notification, and she can return to the terminal at her convenience instead of having to monitor it. The second hook fires after every Bash command and writes the session ID and the command to an audit log in the user's home directory. The log accumulates across sessions and projects, providing a complete record of every shell command Claude has executed under the developer's account.

A discipline applies to notifiers that the developer should keep in mind. Notifiers should be silent in the success case. They produce their side effect, and they exit cleanly. They do not produce `stdout` or `stderr` that the model might mistake for relevant output. A noisy notifier pollutes the session and degrades the experience. A silent notifier is invisible to the workflow except in the side effect it produces, which is exactly what the developer wants.

A second discipline applies to all three patterns and is worth naming explicitly: keep hooks fast. Every hook fires on the lifecycle event it is bound to, and every hook adds latency to every event. A few milliseconds is invisible. A few hundred milliseconds is felt. A few seconds is unbearable, especially for events that fire frequently like `PostToolUse`. The developer should profile her hooks at install time, ensure they complete quickly, and pay particular attention to hooks that invoke heavy tools like full test suites or large linters. Heavy work, when it must run, often belongs in an asynchronous hook that fires the work and returns immediately, leaving the work to complete in the background. Asynchronous hooks are supported through the `async` configuration flag, which the developer can use when the work is genuinely independent of the immediate flow.

The Three-Pattern Vocabulary

The three patterns, gates and guards and notifiers, are the basic vocabulary of hook design. With them in working memory, the developer can look at almost any requirement and immediately recognize which pattern fits. A requirement that says no, do not, never is almost always a gate. A requirement that says always, ensure, conform is almost always a guard. A requirement that says tell me, log, alert is almost always a notifier. The classification is not a rigid taxonomy. It is a thinking tool. It accelerates the design of new hooks by giving the developer a starting shape for any requirement she encounters, and that starting shape is, in most cases, the right shape.

A worked extension of the guard pattern deserves attention here, because guards have a sophistication that gates and notifiers lack and that a developer should understand before designing a serious hook library. A guard can do more than just modify the file system after a tool runs. A guard can also report back to Claude through the JSON output format, telling Claude that something about the result of the tool call was unsatisfactory and that more work is needed. A guard configured this way is not merely corrective; it is conversational. It speaks back to the model and influences what the model does next.

The example below shows this pattern in action. The hook runs after every `Edit` or `Write` tool call. It runs the project's test suite, and if any tests fail, it surfaces the failure back to Claude with a directive that the failing

tests should be fixed before the session continues. The mechanism is the JSON decision-block format, which carries a decision of block and a reason that explains what needs to happen.

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write|MultiEdit",
        "hooks": [
          {
            "type": "command",
            "command": "if ! npm test --silent >/tmp/test-out 2>&1; then echo  
'{\"decision\":\"block\",\"reason\":\"Tests failed after the edit. Read /tmp/test-out and fix the  
failing tests before continuing.\"}'; fi"
          }
        ]
      }
    ]
  }
}
```

The hook runs the test suite after each edit. If the tests pass, nothing happens; the hook exits with code zero and no JSON output, and Claude proceeds normally. If the tests fail, the hook prints the JSON decision-block, which Claude Code reads as a directive to surface the reason back to the model. The next message Claude sees contains the block reason, and Claude responds by reading the test output and fixing the failing tests. The result is that the test suite functions as a continuous check on every edit Claude makes; broken tests are surfaced immediately and addressed before the session moves on. The developer who has installed this hook never has to remember to run tests. The hook remembers, and Claude responds.

A discipline applies to test-running guards that is worth naming, because the obvious implementation is often too slow. Running the full test suite after every edit is acceptable for projects with fast tests, but it becomes painful for projects whose tests take more than a few seconds. The right adaptation is to run only the tests that exercise the changed code, using whichever test selection mechanism the developer's test runner supports. For Jest and Vitest, this might mean using the related-files flag to identify tests that import the changed file. For Pytest, this might mean using a test collection plugin. The general principle is that the hook should run the

smallest set of tests that gives confidence about the change, not the full suite, because the latency penalty of running the full suite quickly outweighs the benefit of having the check run automatically.

Asynchronous Hooks

A short word on asynchronous hooks. Some work the developer wants done after a tool call does not need to block the lifecycle. Logging an audit record. Sending a backup. Updating a dashboard. None of these need to delay the session's progression. For these cases, Claude Code supports asynchronous hooks, configured by adding an `async` flag to the hook definition. An asynchronous hook is fired and forgotten; the lifecycle proceeds while the hook runs in the background. The cost is that the hook's output cannot influence the lifecycle, because by the time the hook finishes, the lifecycle has moved on. This is exactly the right trade-off for the categories of work that asynchronous hooks are meant to handle, where the work matters but the timing does not. The configuration looks like this.

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          {
            "type": "command",
            "command": "node $CLAUDE_PROJECT_DIR/.claude/hooks/sync-to-backup.js",
            "async": true,
            "timeout": 30
          }
        ]
      }
    ]
  }
}
```

The hook fires after every edit, runs the backup script in the background, and returns control to the session immediately. The timeout field caps the work at thirty seconds; if the backup script has not finished by then, it is terminated. The session continues without waiting. The asynchronous flag is the right choice for any hook whose work is genuinely independent of the immediate flow, and the developer who learns to

recognize which hooks belong asynchronous and which need to be synchronous will build pipelines that are both safe and fast.

A reflection before this chapter ends, on what makes the gate-guard-notifier vocabulary so useful in practice. The vocabulary is small enough to hold in working memory, and it covers the cases that come up in real work. A developer who has internalized the three patterns can sit down with a new requirement, ask herself which of the three fits, and have a starting design within a minute. She does not have to re-derive the design space from first principles every time she wants to add a hook. The pattern catalog accelerates the design process, and accelerated design is what allows the developer to add hooks quickly enough that they get added at all. A developer without the vocabulary tends to put off hook work, because each hook feels like a fresh design problem. A developer with the vocabulary tends to add hooks freely, because each hook is a small variation on a familiar pattern.

A second reflection concerns the way the three patterns interact with the lifecycle events. Gates are usually `PreToolUse` hooks, because the natural moment to evaluate whether an action should be allowed is just before the action runs. Guards are usually `PostToolUse` hooks, because the natural moment to enforce consistency is just after the action has produced something to enforce on. Notifiers can attach to almost any event, because their job is to observe and report rather than to evaluate or correct. The pattern-event correspondence is not strict, but it is strong, and a developer reaching for the wrong event for her pattern often finds that the hook is harder to write than it needs to be. A gate written as a `PostToolUse` hook can detect a violation but cannot prevent it. A guard written as a `PreToolUse` hook can decline an action but cannot enforce a correction on a result that does not yet exist. Recognizing the natural pattern-event pairing is part of the craft, and pairing them correctly produces hooks that fit cleanly into the lifecycle.

A short word about combining patterns within a single hook deserves attention, because the question comes up often. A hook that does both validation and correction, for instance, can be tempting; the developer might want a hook that, after a file is written, both formats it and rejects it if formatting fails. The temptation should be resisted in most cases. A hook that does two jobs is a hook that is harder to debug, harder to reason about,

and harder to disable when one of its jobs becomes inappropriate. The cleaner approach is to write two separate hooks, each with a single responsibility, and to bind them to the same event in sequence. The two hooks compose into the combined behavior the developer wanted, but each one remains independently understandable, testable, and maintainable. This single-responsibility discipline at the level of individual hooks is one of the small habits that distinguishes well-designed pipelines from those that have grown into tangled balls of multi-purpose scripts.

The next chapter takes the gate pattern in particular and develops it further. The most consequential application of gates in any production-grade Claude Code environment is in the security layer, where hooks protect against dangerous commands, sensitive file access, secret exposure, and other risks that the probabilistic model cannot be trusted to catch reliably. Building a serious security-hook bundle is the next step in this part of the book, and it is the work of the chapter ahead.

Security and Permission Hooks: Protecting Your System

*“The walls do not have to be high, but they must be where they are needed.
A wall in the wrong place is decoration.”*

a fortifications engineer, anonymous

Of all the work hooks can do, the most consequential is security. A skill that produces excellent code is valuable. A subagent that runs a thorough review is valuable. But a hook that prevents a destructive command from executing, that catches a secret being written to disk, that stops a force-push from reaching production, is valuable in a different and more serious way. Security hooks are the layer of the engineered Claude Code environment where the cost of a single failure can be substantially higher than the entire benefit of the surrounding configuration. A developer can lose a day of work to a single `rm -rf` that should not have run. The hooks in this chapter exist to ensure that the day is not lost.

This chapter builds a security-hook bundle. By the end of it, the reader will have installed five hooks that, together, raise the floor of safety in her Claude Code environment substantially. The five hooks address: dangerous shell commands, protected file paths, secret detection in writes, force-push prevention, and a permission-confirmation gate for high-impact operations. Each hook follows the gate pattern from the previous chapter. Each one is a small, focused expression that does one thing and does it deterministically.

Begin with dangerous shell commands. The pattern is to inspect every Bash command before execution, looking for patterns that the developer has decided should never run. The list of dangerous patterns varies by environment, but a defensible default catches: `rm -rf` on root or home paths, `sudo` combined with destructive commands, `dd` writing to block devices, `mkfs` in any form, `chmod` with wide-open permissions on system paths, and any command that pipes from `curl` or `wget` directly into `bash`. The hook below covers these.

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Bash",
        "hooks": [
          {
            "type": "command",
            "command": "$CLAUDE_PROJECT_DIR/.claude/hooks/block-dangerous.sh"
          }
        ]
      }
    ]
  }
}
```

The hook configuration above points at a script in the project's `.claude/hooks` directory, because the actual logic deserves a real script rather than an inline shell expression. The script itself looks like this.

```
#!/usr/bin/env bash
# block-dangerous.sh, refuses dangerous shell commands.

CMD=$(jq -r '.tool_input.command')

# Patterns that are always blocked.
PATTERNS=(
  'rm[:space:]]+-rf?[:space:]]+(/|~/home|/Users|etc|var)'
  'sudo[:space:]]+rm'
  'dd[:space:]]+if=.*[:space:]]+of=/dev/'
  'mkfs\.'
  'chmod[:space:]]+777'
  'curl[:space:]]+.*\[:space:]]*(bash|sh)'
  'wget[:space:]]+.*\[:space:]]*(bash|sh)'
  ':\(\)\{[:space:]]*:\:&\}'
)

for PAT in "${PATTERNS[@]}"; do
  if echo "$CMD" | grep -qE "$PAT"; then
    echo "Refusing dangerous command. Pattern matched: $PAT" >&2
    exit 2
  fi
done

exit 0
```

The script reads the command from the JSON input, walks an array of forbidden patterns, and exits with code two on the first match. The patterns are not exhaustive; the developer may add to them as she encounters new things she wants blocked. The shape of the script is what matters: it is a list of patterns, easy to extend, with a clear refusal message that surfaces to Claude on every block.

Protecting File Paths

Move to protected file paths. The pattern is similar but applies to Edit and Write rather than Bash. The hook inspects the file path being written to and refuses if the path matches a protected list. The protected list typically includes `.env` files, lockfiles, the `.git` directory, production configuration files, and any file the developer has decided should require explicit human review for any modification.

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Edit|Write|MultiEdit",
        "hooks": [
          {
            "type": "command",
            "command": "$CLAUDE_PROJECT_DIR/.claude/hooks/protect-files.sh"
          }
        ]
      }
    ]
  }
}

#!/usr/bin/env bash
# protect-files.sh, refuses edits to protected file paths.

FILE=$(jq -r '.tool_input.file_path')

PATTERNS=(
  '\.env(\.|$)'
  '\.git/'
  'package-lock\.json$'
  'yarn\.lock$'
  'Cargo\.lock$'
  'production\.ya?ml$'
  '\.pem$'
```



```

'\.key$'
)

for PAT in "${PATTERNS[@]}"; do
  if echo "$FILE" | grep -qE "$PAT"; then
    echo "Refusing to edit protected file: $FILE" >&2
    echo "Pattern matched: $PAT" >&2
    exit 2
  fi
done

exit 0

```

The protection is total. No matter what the user asks, no matter what skill or subagent is firing, the listed paths cannot be edited by Claude. If the developer genuinely needs to modify a protected file, she does so manually outside the hook, or temporarily disables the hook for a single session and re-enables it afterward. The friction is a feature; the protection is a feature; the explicit deliberateness of any modification to these files is a feature.

Move to secret detection. This hook is harder to write well, because secrets come in many forms and false positives are annoying. A working compromise looks for the most obvious patterns: AWS keys, GitHub tokens, OpenAI keys, generic high-entropy strings that look like API keys, and the explicit string `private_key` or `BEGIN PRIVATE KEY`. When a write contains any of these patterns, the hook refuses.

```

#!/usr/bin/env bash
# block-secrets.sh, refuses writes that appear to contain secrets.

CONTENT=$(jq -r '.tool_input.content // .tool_input.new_string // ""')

PATTERNS=(
  'AKIA[0-9A-Z]{16}'
  'ghp_[A-Za-z0-9]{36,}'
  'sk-[A-Za-z0-9]{32,}'
  'BEGIN[[:space:]]+(RSA|EC|DSA|OPENSSH|PRIVATE)[[:space:]]+KEY'
  'aws_secret_access_key[[:space:]]*=[[:space:]]*[A-Za-z0-9+/]{40}'
)

for PAT in "${PATTERNS[@]}"; do
  if echo "$CONTENT" | grep -qE "$PAT"; then
    echo "Refusing to write apparent secret. Pattern matched: $PAT" >&2
    echo "If this is a false positive, edit the file manually outside Claude Code." >&2

```

```
    exit 2
  fi
done

exit 0
```

The script reads the content the tool is about to write, checks it against patterns that match well-known secret formats, and refuses the write if any match. The refusal includes a hint about how to bypass the check legitimately, by writing the file outside Claude Code. This kind of guidance in the refusal message is a small but valuable touch; it preserves the developer's ability to handle edge cases without making her wonder why the hook fired.

Move to force-push prevention. This hook addresses a specific category of git mistake that costs teams real time when it happens. A force push to main or to any protected branch can erase work done by other team members. The hook below intercepts any `git push` command that includes the force flag and refuses if the target branch is one of the protected ones.

```
#!/usr/bin/env bash
# block-force-push.sh, refuses force pushes to protected branches.

CMD=$(jq -r '.tool_input.command')

if echo "$CMD" | grep -qE 'git[:space:]+push.*(--force|--force-with-lease|-f[:space:])' ; then
  if echo "$CMD" | grep -qE '(origin|upstream)[:space:]+(main|master|develop|production|release/)' ; then
    echo "Refusing force push to protected branch." >&2
    echo "Force pushes to main/master/develop/production are blocked." >&2
    exit 2
  fi
fi

exit 0
```

Refusing Force Pushes

The script catches the force-push pattern in any of its common forms and checks the target branch against a protected list. The list can be extended for projects with additional long-lived branches. The protection is conservative, blocking even force-with-lease, because in many real cases the developer's

intent in using force-with-lease is benign but the consequences when it goes wrong are still substantial; an explicit override path through manual command-line use outside Claude Code remains available.

The final security hook is a permission-confirmation gate for high-impact operations. Some actions are not strictly forbidden but should never happen without explicit human acknowledgment. Deploying to production. Running database migrations. Sending mass emails. Each of these can be encoded as a gate that prompts the user for confirmation rather than refusing outright. The mechanism uses a `PreToolUse` hook that, when it matches a high-impact pattern, opens a desktop notification or a small terminal prompt asking the user to confirm. If the user confirms, the hook exits with code zero and the action proceeds. If the user declines or the prompt times out, the hook exits with code two and the action is blocked.

The implementation varies by operating system, because the notification mechanism differs. On macOS, the `osascript` command can produce a confirmation dialog that returns a result. On Linux, `zenity` or `kdialog` can do the same. On Windows, PowerShell can produce a confirmation dialog. The configuration below uses macOS `osascript` as an example, but the pattern translates directly to the other systems.

```
#!/usr/bin/env bash
# confirm-deploy.sh, requires explicit confirmation before deploy commands.

CMD=$(jq -r '.tool_input.command')

# Patterns that require explicit confirmation.
PATTERNS=(
  'kubectl[:space:]+apply.*production'
  'terraform[:space:]+apply'
  'aws[:space:]+s3[:space:]+sync.*production'
  'flyway[:space:]+migrate'
  'alembic[:space:]+upgrade'
)

for PAT in "${PATTERNS[@]"; do
  if echo "$CMD" | grep -qE "$PAT"; then
    RESULT=$(osascript -e "display dialog \"Claude Code is about to run a high-impact
command:\\n\\n$CMD\\n\\nAllow it?\" buttons {\"Cancel\\\", \"Allow\\\"} default button \"Cancel\\\"\"
2>/dev/null)
    if echo "$RESULT" | grep -q "Allow"; then
      exit 0
    fi
  fi
done
```

```
    else
        echo "User declined high-impact command." >&2
        exit 2
    fi
fi
done

exit 0
```

The script reads the Bash command from the JSON input, checks it against a list of patterns that match high-impact operations, and if any match, opens a system dialog asking the user whether to allow the command. The dialog is modal; the script blocks until the user responds. If the user clicks Allow, the script exits with code zero and the command proceeds. If the user clicks Cancel or dismisses the dialog, the script exits with code two and the command is blocked. The user is now in the loop for any operation that touches production, with no possibility of a slip-up where the model decides on its own that a deployment is safe to run.

The list of patterns in the confirmation gate is, like the lists in the earlier hooks, intended to be edited as the developer learns. New high-impact operations will surface over time, and each one should be added to the list as it is recognized. The hook is most valuable when its pattern list is comprehensive, because every gap is an operation that could happen without confirmation, and confirmation gates are most useful when they leave no significant gaps.

The Threat Model

A reflection on the threat model is worth including, because it clarifies what the hooks in this chapter do and do not protect against. The hooks protect against accidental damage caused by Claude itself or by the developer through Claude. They prevent Claude from running an `rm -rf` when the developer asked for help cleaning up files. They prevent Claude from writing a file to `.env` when the developer asked for help with environment variables. They prevent Claude from force-pushing to main when the developer asked for help reorganizing branches. The protection is against the failure mode where the developer's broad request gets translated by the model into an action she did not intend.

The hooks do not protect against a deliberately malicious user. A user with shell access to the developer's machine can edit the hook scripts, disable the hook configuration, or simply run the dangerous command directly outside Claude Code. The hooks are not a security boundary against an adversary. They are a safety boundary against accident, and that is exactly the boundary they are designed to be. A security architect who reads this chapter and concludes that hooks are insufficient as a complete security solution is correct. The right framing is that hooks are one layer in a defense-in-depth model, and the layer they occupy is the one closest to the developer, where most actual incidents in practice originate. The other layers, including operating system permissions, version control protections, and infrastructure access controls, remain necessary, and the hooks complement them rather than replacing them.

Together, the five hooks in this chapter form a security-hook bundle that addresses the most common categories of risk a developer faces in a Claude Code session. The hooks are not exhaustive; security is a continuous practice, not a fixed checklist, and the developer should expect to add to her bundle as her work surfaces new risks. But the five above cover the largest categories at the lowest cost. Installing them takes roughly an hour. The protection they provide compounds over every future session, and the cost of not having them, even once, is potentially substantial.

A second reflection concerns the experience of working with these hooks installed. The hooks are quiet most of the time. They sit in their gate positions, evaluating every relevant tool call, and they almost never block, because the developer almost never asks Claude to do anything dangerous. The cost of their presence is negligible. Then, one day, a hook fires. The developer asked for something that, on careful reading, would have led Claude to write a secret to a file or to run a command she did not actually intend. The hook stops the action. The session continues without damage. The developer realizes, in retrospect, that she had asked for something she did not mean. The hook just bought her a substantial amount of cleanup time, or worse. This pattern, low ongoing cost and high single-event payoff, is what makes security hooks worth installing even before any incident has demonstrated their value.

A fourth observation, more practical, concerns the maintenance of the security bundle over time. The patterns the hooks check for will need to

evolve as the developer's work evolves. New dangerous commands will be discovered. New protected paths will become relevant as projects grow. New secret formats will emerge as the developer adopts new services that use new key shapes. The hook bundle is not a fire-and-forget installation. It is a small but real piece of ongoing maintenance, and the developer should plan to revisit it every few months, adding new patterns to the lists and removing patterns that no longer apply. Fifteen minutes of maintenance per quarter is sufficient to keep the bundle effective, and the maintenance is much easier to perform if the hook scripts are written cleanly with their pattern lists clearly visible at the top of each file. The discipline of writing hooks that are easy to maintain is, in the end, the discipline that determines whether the bundle remains useful over the years or quietly drifts into uselessness.

The Team Dimension

A fifth observation concerns the team dimension of the security bundle. Security hooks are valuable when one developer installs them, and they are dramatically more valuable when an entire team installs the same set across all of its projects. A team that has agreed on a standard security bundle, encoded in shared configuration that travels with each project, raises the floor of safety for everyone on the team simultaneously. New team members inherit the bundle automatically the moment they clone any project. The team's collective risk of an incident drops sharply, because the most common categories of accident are blocked across every developer's environment, not just in the environments of the developers who happened to install hooks personally. This is the team-level argument for hooks, and it is the argument that justifies treating the security bundle as a shared engineering asset rather than as a personal preference. Teams that adopt this view tend to have strikingly fewer incidents than teams that leave each developer to her own preferences, and the difference is not subtle.

A sixth and final observation concerns the relationship between security hooks and the other two pillars of the framework. Skills and subagents both shape what Claude does within the agent loop, and both can encode warnings about dangerous operations. A skill could include a directive to never run `rm -rf`, and a subagent system prompt could include a similar warning. These shapings are useful, but they are not enough on their own,

because they ride on the model's probabilistic compliance with its instructions. The security hook is what turns the warning into a guarantee. The right architectural pattern is to have all three: a skill that explains what good practice looks like, a subagent that operates within that practice, and a hook that enforces the boundary that even the best skill and subagent cannot guarantee. The three layers reinforce each other. The skill teaches. The subagent applies. The hook enforces. The combination is stronger than any one layer alone, and developers who think of security as a single-layer concern miss the architectural elegance that the three-pillar approach provides.

A third reflection concerns the relationship between hooks and trust. Without hooks, the developer has to trust the model on every action. Trust, in this context, means hoping that the model will interpret her requests correctly and will not produce dangerous side effects. Trust is appropriate for many things, but it is not appropriate for catastrophic failure modes, because the failure rate of any probabilistic system is non-zero, and a non-zero failure rate over many actions eventually produces a failure. Hooks change the relationship. With hooks, the developer trusts the model for most things and relies on mechanism for the things that matter. This is a better division of labor than asking the model to be perfect, because perfection is not what models are good at, and mechanism is exactly what mechanism is good at.

A reflection before this chapter closes. Security hooks have a quiet quality that distinguishes them from skills and subagents. Skills produce visible value: every commit message, every PR description, every refactor proposal is something the developer sees and uses. Subagents produce visible value too: every parallel review, every specialized output is a tangible artifact. Security hooks produce value that is mostly invisible. Most days, no security hook fires, because the developer is not asking Claude to do anything dangerous. The hooks sit silently. Their value reveals itself only on the day when something goes wrong and a hook stops it from going more wrong. That day, the value of the entire bundle is realized in a single block. The asymmetry, low ongoing cost and high single-event payoff, is what makes security hooks worth installing even if they never fire. The developer who waits until something goes wrong to install them has discovered the value of hooks at the worst possible moment. The

developer who installs them as part of her standard environment has bought insurance she will probably never use, which is exactly the kind of insurance worth having.

The next chapter takes hooks one more step into composition. With single hooks installed, the question becomes how multiple hooks work together, how event pipelines emerge when hooks chain, and how the developer can construct sophisticated lifecycle workflows out of the same simple primitives. Hook chains are the final piece of hook design, and they are the subject of the chapter ahead.

Hook Chains and Event Pipelines

“A pipeline is what happens when many small actions, each useful in itself, are arranged so that the output of one becomes the context of the next.”

Doug McIlroy, paraphrased

This chapter is the final treatment of hooks before the book turns to composition across all three pillars. The previous three chapters built hooks one at a time: the mechanics, the design patterns, the security applications. This chapter puts hooks together. When multiple hooks fire on the same event, when hooks pass information forward through the lifecycle, when the developer wants a coordinated workflow that involves several lifecycle moments rather than just one, the resulting structure is a hook chain or an event pipeline. This chapter teaches both.

Begin with the simpler case. A hook chain is what happens when more than one hook is configured under the same event with the same matcher. Claude Code runs them in the order they are listed in the configuration file. Each hook receives the same input on standard input. Each hook’s exit code is independent: one hook’s decision to allow or block does not affect what the next hook does. The chain proceeds through all hooks unless one of them blocks, in which case the action is blocked and subsequent hooks in the same chain do not run. The output of each hook is collected, but the hooks themselves do not communicate directly with each other. Communication, when it is wanted, happens through side effects on the file system or through environment variables.

The Prettier-and-ESLint example from Chapter Sixteen is a small chain. Two hooks fire on the same `PostToolUse` event with the same matcher. The first runs Prettier; the second runs ESLint. The order matters in this case, because Prettier reformats the file and ESLint then applies its fixes on the reformatted version. If the order were reversed, the result would be subtly different in some edge cases. The developer who designs a chain should think about whether the order of hooks is meaningful, and if it is, she should comment the configuration to make the order explicit for future readers.

A second example, slightly more elaborate, shows a chain that combines a notifier with a guard. The configuration runs after every Bash command that Claude executes: it logs the command to an audit file, then checks whether the command has produced any new git changes, and if so, writes a small note to a session-summary file that the developer can review at the end of the day.

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Bash",
        "hooks": [
          {
            "type": "command",
            "command": "jq -r '\"\\(.session_id)\\t\\(.tool_input.command)\\\"' >> ~/.claude/bash-
history.log"
          },
          {
            "type": "command",
            "command": "if git status --porcelain 2>/dev/null | grep -q . ; then jq -r
'.tool_input.command' >> ~/.claude/session-summary.txt; fi"
          }
        ]
      }
    ]
  }
}
```

Two Hooks in Sequence

The two hooks fire in sequence on the same event. The first appends a tab-separated record to the audit log. The second checks the `git status` and, if changes are present, appends the command to a session summary. Both run on every Bash call. Both are silent in the success case. Together, they produce an audit trail and a daily summary without any conscious intervention from the developer.

Move to event pipelines, which is the more sophisticated case. An event pipeline is a workflow that spans multiple lifecycle events, with hooks at each event reinforcing or extending the others. The classic example is a git pipeline: a `SessionStart` hook ensures the working tree is clean and reports the

current branch, a `PreToolUse` hook on Bash blocks force-pushes to main, a `PostToolUse` hook on Edit or Write runs the formatter, a Stop hook checks whether tests pass before allowing the session to end with uncommitted changes. None of these hooks knows about the others directly. But together they form a pipeline that wraps every git-related action in a consistent set of guarantees, and the developer who has installed the pipeline experiences her git work differently because every step is now defended in the right place.

A worked event pipeline is presented below. The configuration is more substantial than any in the previous chapters, because it is doing real cross-event coordination. The pipeline assumes a project where tests are run with `npm test`, the main branch is named main, and the developer wants Claude Code to be especially careful with anything that touches that branch.

```
{
  "hooks": {
    "SessionStart": [
      {
        "hooks": [
          {
            "type": "command",
            "command": "echo '## Branch'; git branch --show-current; echo '## Status'; git status --
short"
          }
        ]
      }
    ],
    "PreToolUse": [
      {
        "matcher": "Bash",
        "hooks": [
          {
            "type": "command",
            "command": "$CLAUDE_PROJECT_DIR/.claude/hooks/block-dangerous.sh"
          },
          {
            "type": "command",
            "command": "$CLAUDE_PROJECT_DIR/.claude/hooks/block-force-push.sh"
          }
        ]
      }
    ],
    {
      "matcher": "Edit|Write|MultiEdit",
```

```

    "hooks": [
      {
        "type": "command",
        "command": "$CLAUDE_PROJECT_DIR/.claude/hooks/protect-files.sh"
      }
    ]
  },
  "PostToolUse": [
    {
      "matcher": "Edit|Write|MultiEdit",
      "hooks": [
        {
          "type": "command",
          "command": "FILE=$(jq -r '.tool_input.file_path'); npx prettier --write \"$FILE\"
2>/dev/null || true"
        }
      ]
    }
  ],
  "Stop": [
    {
      "hooks": [
        {
          "type": "command",
          "command": "if [ \"$(git status --porcelain | wc -l)\" -gt 0 ]; then echo
'{\"decision\":\"block\",\"reason\":\"Uncommitted changes present. Commit or stash before
ending the session.\"'}'; fi"
        }
      ]
    }
  ]
}
}

```

Read the configuration carefully, because it covers four lifecycle events and demonstrates how a serious pipeline holds together. `SessionStart` runs once when the session begins and prints the current branch and any uncommitted changes; the output is injected into Claude's starting context, so Claude knows the state of the world at the start of every session. `PreToolUse` on Bash runs the dangerous-command and force-push gates from Chapter Seventeen. `PreToolUse` on Edit and Write runs the protected-files gate. `PostToolUse` on Edit and Write runs Prettier. Stop runs a check that uses the JSON output format to refuse the session's end if there are uncommitted

changes, returning a decision of block with a clear reason that surfaces to Claude.

A Coherent Workflow

The pipeline together enforces a coherent workflow. Sessions begin with awareness of state. Tool calls are filtered through gates appropriate to their kind. Edits are followed by formatting. Sessions cannot end with the working tree dirty. The developer has not had to remember any of these rules, because each rule is enforced by a hook at the right lifecycle moment. The development experience is, in a sense, supervised, but the supervision is mechanical and silent, and it intervenes only when the developer was about to do something the pipeline considers a mistake.

A discipline applies to event pipelines that is worth stating directly. Pipelines are powerful, but they are also a form of accumulated complexity, and every hook in a pipeline is a hook the developer is committing to maintain. A small pipeline of three or four well-chosen hooks is a substantial improvement over no pipeline at all. A pipeline of fifteen hooks may be worse than seven, because the failure modes multiply, the cumulative latency adds up, and the developer's ability to debug an unexpected behavior degrades when many hooks are running. The recommendation is to start small, add hooks one at a time, and remove any hook that has not earned its place after a month of use. The pipeline should serve the developer, not the other way around, and a pipeline that has grown into a maintenance burden is a pipeline that has been over-engineered.

A second discipline concerns testing. Every hook should be tested before it is committed. The standard testing pattern is to invoke the hook directly from the shell, passing in a representative JSON input on standard input, and observing the exit code and the standard error output. A hook that behaves correctly under direct testing usually behaves correctly inside Claude Code; a hook that does not has a chance of producing inscrutable failures inside the session. The fifteen minutes of direct testing at install time prevents debug sessions later. The pattern is worth adopting unconditionally for any hook that does anything more sophisticated than a one-liner.

A second worked pipeline is worth presenting, because pipeline design is the kind of skill that matures only after a developer has seen several

substantial examples. The pipeline below is for a different shape of work: a content-generation project, where the developer is using Claude Code to produce written material rather than to write code. The pipeline reflects the different concerns of writing work. Drafts should be backed up automatically. Drafts should be passed through a humanizer pass after every edit. Sessions should end with a daily summary appended to a project journal. The pipeline below implements each of these.

```
{
  "hooks": {
    "SessionStart": [
      {
        "hooks": [
          {
            "type": "command",
            "command": "echo '## Today, you are working on:'; cat
$CLAUDE_PROJECT_DIR/.claude/today.txt 2>/dev/null || echo 'No focus file. Set one in
.claude/today.txt.'"
          }
        ]
      }
    ],
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          {
            "type": "command",
            "command": "FILE=$(jq -r '.tool_input.file_path'); cp \"${FILE}\"
\"$CLAUDE_PROJECT_DIR/.backups/${basename $FILE}.${date +%s}\" 2>/dev/null || true",
            "async": true
          },
          {
            "type": "command",
            "command": "FILE=$(jq -r '.tool_input.file_path'); if echo \"${FILE}\" | grep -qE '\\.
(md|txt)$'; then $CLAUDE_PROJECT_DIR/.claude/hooks/check-em-dashes.sh \"${FILE}\"; fi"
          }
        ]
      }
    ],
    "SessionEnd": [
      {
        "hooks": [
          {
            "type": "command",
```

```

    "command": "echo \"$(date '+%Y-%m-%d %H:%M') session ended. Files touched
today:\" >> $CLAUDE_PROJECT_DIR/journal.md; git diff --name-only HEAD~5..HEAD
2>/dev/null | head -10 >> $CLAUDE_PROJECT_DIR/journal.md"
  }
]
}
]
}
}

```

Read the pipeline carefully, because its shape is informative. `SessionStart` loads a focus file, which the developer has placed in the project to remind herself what she is working on today. The output is injected into Claude's starting context, so Claude knows the day's focus from the moment the session begins. `PostToolUse` fires two hooks on every Edit or Write. The first is asynchronous: it backs up the file to a timestamped copy in a backups directory, without slowing down the session. The second runs only on Markdown or text files, and it invokes a custom script that checks for em-dashes and other patterns the developer wants to avoid in her writing. `SessionEnd` appends a record to a project journal, giving the developer a continuous log of what she worked on across sessions. The pipeline does not block any of these actions. It does not interfere with Claude's flow. It quietly produces backups, applies the writing standard, and maintains the journal, all without conscious intervention.

What Pipelines Buy You

The pipeline is small, but it changes the developer's experience of working on the project substantially. She knows that backups exist. She knows that her em-dash policy is being enforced on every edit. She knows that her journal is up to date. The mental load of remembering these things has shifted from her to the pipeline, and the load that has been removed from her is now available for the actual writing work. This is the deeper benefit of pipelines: they take ambient concerns off the developer's mind and handle them as standing background processes, freeing the developer's attention for the work that requires attention.

A discipline applies to event pipelines that is worth stating directly. Pipelines are powerful, but they are also a form of accumulated complexity, and every hook in a pipeline is a hook the developer is committing to

maintain. A small pipeline of three or four well-chosen hooks is a substantial improvement over no pipeline at all. A pipeline of fifteen hooks may be worse than seven, because the failure modes multiply, the cumulative latency adds up, and the developer's ability to debug an unexpected behavior degrades when many hooks are running. The recommendation is to start small, add hooks one at a time, and remove any hook that has not earned its place after a month of use. The pipeline should serve the developer, not the other way around, and a pipeline that has grown into a maintenance burden is a pipeline that has been over-engineered.

A second discipline concerns testing. Every hook should be tested before it is committed. The standard testing pattern is to invoke the hook directly from the shell, passing in a representative JSON input on standard input, and observing the exit code and the standard error output. A hook that behaves correctly under direct testing usually behaves correctly inside Claude Code; a hook that does not has a chance of producing inscrutable failures inside the session. The fifteen minutes of direct testing at install time prevents debug sessions later. The pattern is worth adopting unconditionally for any hook that does anything more sophisticated than a one-liner.

A third discipline concerns version control. The `.claude` directory of a project, including the `settings.json` file that holds the hook configuration, should be version-controlled alongside the source code. Hooks are part of the project's contract for how work happens in the repository. A team member who clones the project for the first time should receive the entire pipeline automatically, without needing to configure anything herself. The version control discipline ensures this. Pipeline changes should be reviewed in pull requests like any other code change, because a hook change can affect every future session in the project, and the impact of a bad hook can be substantial. The same engineering hygiene that applies to source code applies to hook configuration, and projects that treat their `.claude` directory with this discipline end up with stable, predictable, well-understood pipelines that genuinely serve the team.

Pipelines and Engineering Practice

A fourth observation, more philosophical, concerns what pipelines mean for the practice of software engineering. The traditional way of enforcing

project conventions has been documentation, training, and review. Documentation tells people what the conventions are. Training builds the habits. Review catches violations. Each layer is necessary, and each layer is fallible. A pipeline shifts some of the enforcement from human judgment to mechanism. The conventions that the pipeline enforces no longer need to be documented in a wiki, because they are enforced at the moment they would be violated. The training for new team members shrinks accordingly, because the environment teaches the conventions through its behavior. Review focuses on the things that pipelines cannot enforce, like architectural choices and code quality, rather than on style and formatting issues that mechanism handles cleanly. The result is a team where attention is concentrated on what is genuinely human work, and the mechanical work is done by the mechanism. This is, in the end, the deeper promise of the engineered Claude Code environment, and pipelines are the mechanism that delivers it.

A final observation about pipelines, before this chapter ends, concerns what happens when several pipelines coexist. A user-level pipeline applies to every project. A project-level pipeline applies inside the specific project. The two pipelines combine: every event that fires runs the hooks from both levels, in a defined order. The combined behavior is the union of the two pipelines, which means that the developer can build a stable personal pipeline at the user level and let projects extend it as needed. A team that has agreed on certain project-level conventions can encode them in the project pipeline. Each developer's personal preferences are layered on top through the user pipeline. The combination is a personalized variant of the team standard, and the developer experiences both layers as a single coherent environment. This layered architecture is one of the quieter strengths of the hook system, and developers who design with it in mind end up with environments that are both standardized and personalized, without conflict between the two.

One more observation deserves a place before this chapter closes, about the felt experience of working with a substantial pipeline installed. The pipeline does its work invisibly. The developer does not see the formatting hook fire, because the file simply ends up formatted. The developer does not see the test guard run, because the tests pass and the session continues normally. The developer does not see the audit logger record her

commands, because the log file fills silently in the background. What the developer experiences is a working environment that simply behaves the way she wants it to, as if the conventions she cares about were the natural shape of the tool itself. The configuration is invisible, and the behavior is what remains. After a few weeks of working in such an environment, the developer often forgets that the behavior is the result of configuration at all; the environment feels like the default, and the absence of the configuration on a fresh installation feels strange. This is the deepest sign of a successful pipeline. It has receded into the background, and the work has become foreground. The architecture, once built, asks nothing further of the developer, and the developer's attention is free to focus entirely on what she is making.

A reflection before this chapter and this part of the book end. Hooks are quieter than skills and subagents. They do not produce visible artifacts in most sessions. They sit at the edges of the lifecycle, doing their work without drawing attention, and they earn their place by what they prevent rather than by what they produce. The developer who has installed a thoughtful set of hooks may not notice them for weeks at a time, and that invisibility is exactly what good hook design produces. The hooks that are invisible are the hooks that are working. The day comes, eventually, when a hook fires on a real action that needed to be blocked, or a guard catches a problem that would have shipped, or a notifier surfaces an event that needed attention. On that day, the entire investment in the hook bundle is repaid in a single moment, and the developer recognizes, sometimes for the first time, that the architecture has been protecting her all along.

Part Four ends here. The reader has, over four chapters, learned the mechanics of hooks, internalized the gate-guard-notifier vocabulary that organizes hook design, installed a security-hook bundle, and seen how multiple hooks compose into event pipelines that wrap entire workflows in deterministic guarantees. The third pillar of the framework is now in place. Three pillars stand. The next part of the book takes the architecture as a whole and shows how to build a complete environment in which all three pillars work together. Two end-to-end case studies, each substantial, will demonstrate the architecture in concrete form. By the end of the next part, the reader will have seen the framework not as three separate sets of techniques but as a single coherent system, and she will have the blueprint

for building such a system in her own work. That synthesis begins in the chapter ahead.

PART FIVE

The Composition Layer: Building a Self-Governing Development Environment

Three pillars, one architecture. The whole begins to mean more than the sum of the parts.

CHAPTER NINETEEN

The Three-Pillar Architecture: Designing a Complete Claude Code System

“Architecture is the art of arranging elements so that the whole has properties none of the elements possess. Without arrangement, there is only inventory.”

Christopher Alexander, paraphrased

Up to this point in the book, the three pillars have been treated separately. Skills were the subject of Part Two, subagents the subject of Part Three, hooks the subject of Part Four. Each part developed its pillar in depth, with mechanics, design patterns, and a working library installed by the chapter’s end. The treatment was deliberately separate, because each pillar carries its own design discipline and learning each one in isolation is the only way to develop the discipline cleanly. But the separation is now over. The remainder of the book treats the three pillars as components of a single architecture, and this chapter is where the architecture itself gets explicit attention.

The argument of this chapter is that any sophisticated Claude Code environment uses all three pillars, that the pillars have a natural division of responsibility, that the composition between them follows recognizable patterns, and that the resulting environment has properties that no single pillar could produce on its own. The architecture is not arbitrary. It reflects the structure of the tool, the structure of real engineering work, and the structure of the failure modes that the architecture is built to prevent. Once the reader has internalized the architecture, the design of any new Claude Code environment becomes a matter of applying the same patterns to her specific work, with the patterns themselves as a stable starting point.

Begin with the division of responsibility. Each pillar handles a class of problem that the other two cannot handle as well. Skills handle capability; they teach Claude how to do specific kinds of work consistently. Subagents handle specialization and parallelism; they give specific kinds of work their own focused context, their own tuned role, and the option to run alongside other work simultaneously. Hooks handle determinism; they impose mechanical guarantees on the lifecycle, ensuring that what must happen does happen, and what must not happen cannot happen. The three together cover capability, specialization, and boundary, which are the three categories of concern in any serious workflow.

A simple way to remember the division is by the question each pillar answers. Skills answer the question, how should this be done. Subagents answer the question, who should do this. Hooks answer the question, what is permitted. A workflow that uses only skills can teach Claude to do good work but has no way to isolate different kinds of work and no way to guarantee enforcement. A workflow that uses only subagents has specialization but no stored knowledge and no enforcement. A workflow that uses only hooks has enforcement but no capability and no specialization. Each pillar, alone, is a partial answer. All three, together, form a complete one.

Two Layers of the Architecture

The next observation, which is more subtle, is that the three pillars sit at different layers of the architecture. Skills and subagents both operate inside the agent loop. They are read by the model. They shape the model's behavior through prose. They live in the probabilistic space where the model interprets, reasons, and decides. Hooks operate outside the loop. They run as shell commands, not as language. They are not interpreted by the model. They are mechanical. The two layers, probabilistic inside and deterministic outside, are the natural partitioning of responsibility in a well-designed environment. The model is given freedom and capability inside the loop. The boundaries of the loop are deterministic. This shape, what one practitioner has called the deterministic outer and the probabilistic inner, is the central architectural pattern of the book, and it is worth holding in working memory because it explains almost every design decision in any complete environment.

A worked illustration brings the architecture to life. Consider a developer who needs to perform a code review. The work has several components. The reviewer needs to know how good code reviews are written in this codebase. The reviewer needs to apply different kinds of attention to different aspects of the change. The reviewer needs to be prevented from accidentally approving changes that touch protected paths. Each of these components maps cleanly to one of the three pillars. The how is encoded in skills: a project-level pre-review skill that knows the team's preferred review format. The who is encoded in subagents: parallel security, performance, and architecture specialists who each handle their dimension.

The what-is-permitted is encoded in hooks: a gate on `PreToolUse` that blocks any tool call that would mark a PR approved without explicit human confirmation when the PR touches authentication code.

The composition runs naturally. The developer asks for a review. Skills fire because the request matches their descriptions. Subagents are dispatched in parallel because the orchestrator pattern recognizes the multi-dimensional shape of the work. Hooks watch the lifecycle silently. The review unfolds in two minutes, the unified result is presented to the developer, and the deterministic guarantees protect the boundary that the model alone could not be trusted to respect. None of this required the developer to think about the architecture during the request. She typed a sentence. The architecture did the work.

A second observation about the architecture concerns the order of construction. The natural order in which a developer should build her environment is the order in which the pillars were taught in this book: skills first, subagents second, hooks third. The order is not arbitrary. Skills are the broadest and most generally useful pillar; they pay back immediately in almost every session, and they are the easiest to write well on the first attempt. Subagents are the next layer; they require slightly more design effort but produce dramatic benefits when the work has the right shape. Hooks are the most specialized; their value is concentrated in specific safety-critical concerns and in workflow enforcement, and they require the most careful design to install correctly. Building in this order means that the early investment produces immediate benefits, the middle investment produces architectural benefits, and the late investment produces the deterministic guarantees that complete the system. A developer who tries to build in reverse order, hooks first and skills last, often finds the experience frustrating, because hooks without skills and subagents are just constraints on a vanilla environment, and the constraints feel restrictive rather than enabling.

Failure Modes Prevented

A third observation, this one about the failure modes the architecture prevents. Each pillar prevents a different category of failure. Skills prevent the failure of inconsistency: without skills, the developer's own attention has to carry every convention, and her attention is fallible. Subagents

prevent the failure of context pollution: without subagents, the main session's context fills with the residue of every phase of work, and the quality of every phase suffers. Hooks prevent the failure of probabilistic breach: without hooks, the model's judgment is the only line of defense against catastrophic mistakes, and the model's judgment is non-zero failure rate. The architecture, taken as a whole, is a defense in depth against the three most common ways that AI-assisted workflows go wrong. A developer who has installed all three pillars has, in effect, hardened her workflow against the three failure modes that would otherwise haunt it.

A fourth observation concerns the design process for adding new capabilities to the environment. When the developer notices a friction in her work that she wants to address, she should ask herself which pillar fits. The classification questions are simple. Is this a knowledge gap, where the developer wishes the environment knew something specific? That is a skill. Is this an attention gap, where the developer wishes a different kind of attention were applied to a kind of work? That is a subagent. Is this a guarantee gap, where the developer wishes that something would always or never happen? That is a hook. Asking the right question first prevents the most common design mistakes, which are building skills that should have been hooks, building hooks that should have been skills, and building subagents that should have been narrow skills inside a regular session. The classification is fast, the right answer is usually obvious once the classification has been made, and the cumulative effect of asking the question every time is an environment whose components live in the right places.

A fifth observation concerns over-engineering. The temptation to build an elaborate environment is real, and most developers go through a phase where they install too many skills, too many subagents, too many hooks, and the environment becomes harder to maintain than the work it was supposed to support. The remedy is to start small, add components only when their need is concrete, and remove components that have not been used in a few months. A small, well-designed environment of fifteen skills, six subagents, and eight hooks is dramatically more useful than a sprawling one with fifty of each. The compounding of component count past a certain point becomes a tax rather than a benefit, because the developer has to keep more in her head, the failure modes multiply, and the cost of maintenance

starts to consume the benefit of the components themselves. The discipline of restraint is part of the architecture.

A sixth observation concerns the maintenance posture of the environment. A well-built environment is not a static artifact. It evolves with the developer's work. New skills get added when new frictions emerge. Old skills get refined as the developer's preferences change. Subagents get reshaped as the developer learns which roles benefit most from isolation. Hooks get updated as new threats and conventions emerge. The .claude directory of any active developer should look noticeably different at the end of a year from how it looked at the start, because the developer's environment has been growing alongside her work. The maintenance investment is small; thirty minutes per week is sufficient to keep an environment current and useful. The compounding of that investment, across the years a developer uses her environment, is what produces the gap between developers who treat their tooling seriously and developers who do not.

What This Architecture Means

The final observation in this chapter is the one that matters most for the rest of the book. The composition principle, repeated throughout these chapters, is not just a design pattern. It is the difference between using Claude Code and engineering with it. A developer who has built a thoughtful three-pillar environment is doing something fundamentally different from a developer who has not. She is operating inside an environment that has been shaped to her work, that absorbs frictions she would otherwise carry, that protects her from mistakes she would otherwise make, and that produces a level of consistency and quality across her output that no amount of careful prompting alone could match. The investment to build this environment is real but bounded. The benefit, once built, compounds for as long as the developer continues to work in the environment. There is no comparable investment available to a software engineer in 2026 that produces anything close to the leverage of a well-designed Claude Code environment, because no other tool sits at the intersection of capability, specialization, and enforcement that Claude Code does.

The next two chapters take the architecture from principle to practice. Each chapter is a substantial case study in which a complete environment is

built end-to-end for a specific kind of work. The first case study, in the chapter that follows, is a full-stack development environment for a TypeScript engineer. The second, in the chapter after that, is a solopreneur's environment for a technical founder running product, marketing, and operations from a single Claude Code installation. Together, the two case studies demonstrate the architecture across two very different shapes of work, and they leave the reader with concrete blueprints she can adapt to her own situation. The architectural principles end here. The applied builds begin in the chapter ahead.

Before turning to the case studies, a few additional architectural patterns deserve explicit attention, because they show up in almost every well-built environment and the reader should recognize them when she sees them in the chapters ahead.

The first pattern is what might be called the project-skill, user-subagent split. In well-designed multi-developer environments, project-specific knowledge tends to live in skills at the project level, while role specialists tend to live as subagents at the user level. The reason is that knowledge is project-specific (a Python project has different conventions from a TypeScript project) but roles are usually personal (a developer's preferred way of approaching code review does not change much when she switches projects). The split allows the developer to maintain a stable personal subagent library that she carries from project to project, while each project contributes its own skills that augment her subagents with project-specific knowledge. The composition pattern from Chapter Fourteen depends on this split working cleanly, and developers who set up their environments with the split in mind from the start avoid a common refactoring later when they realize their subagents have accumulated project-specific knowledge that should have lived elsewhere.

Ambient Versus Targeted

The second pattern is the ambient-versus-targeted split, which was introduced in Chapter Two but deserves restatement in the architectural context. Information that is true at all times in a project belongs in `CLAUDE.md`, where it is loaded into every session's context unconditionally. Information that is true only when a specific kind of work is being done belongs in a skill, where it is loaded only when the skill activates. The line

between the two is sometimes blurry, but a useful rule is that any piece of knowledge that costs more than fifty words to express, and that is relevant to less than a quarter of the work in the project, almost certainly belongs in a skill rather than in `CLAUDE.md`. Putting too much in `CLAUDE.md` inflates the baseline context cost of every session and degrades the model's attention. Putting too little in `CLAUDE.md` makes the developer brief Claude on the same project facts every session, which defeats the purpose of having ambient context at all. The architectural discipline is to push as much as possible into skills and keep `CLAUDE.md` to genuinely ambient material.

The third pattern is the hooks-as-the-last-resort principle. When the developer is considering whether to enforce some convention with a hook or with a skill, the right default is to start with a skill and only escalate to a hook if the skill is not enough. Skills are softer, more flexible, easier to maintain, and more conversational. Hooks are harder, more rigid, and harder to debug when they go wrong. A convention that the model can reliably honor when reminded should live in a skill. A convention that the model genuinely cannot be trusted to honor, where the consequences of a single failure are substantial, should live in a hook. This default keeps the architecture honest. The developer who reaches for hooks first ends up with a brittle environment that is hard to evolve. The developer who reaches for skills first and escalates to hooks only when necessary ends up with a flexible environment whose enforcement layer is concentrated where it actually earns its place.

The fourth pattern concerns the lifecycle of any specific configuration entry. Every skill, subagent, and hook the developer adds to her environment goes through a predictable lifecycle. It is created in response to a specific friction. It is used heavily for some period. It either earns its place by being used continuously or fades into disuse. Either way, after roughly six months, the developer should review the entry and ask whether it is still pulling its weight. Entries that have not been used in months should be removed. Entries that have been used heavily but slightly off-fit should be refined. Entries that have been used as a starting point for several variants should perhaps be split into more specific siblings. This kind of pruning maintenance is what keeps an environment from accumulating dead weight, and the developers who treat their `.claude` directories as living

projects, gardened regularly, end up with environments that stay sharp for years rather than gradually decaying into clutter.

The fifth pattern, and the last to name explicitly before the case studies, is the small-and-good principle. A small environment of fifteen well-designed components beats a large environment of fifty mediocre ones, almost always. The temptation to build large is real, especially after the first few components produce visible benefits. The discipline is to resist building large for its own sake, and to add components only when their need is concrete and undeniable. Many of the most effective environments in practice contain fewer than thirty components total. Their effectiveness comes from how well-tuned the components are, not from their count. The reader who internalizes the small-and-good principle will produce environments that work beautifully and remain maintainable. The reader who does not will produce environments that impress at first glance and frustrate at the first attempt to evolve them six months later.

A final architectural observation, before the case studies begin, concerns the relationship between the three-pillar architecture and the deeper craft of software engineering. Engineers who have worked on production systems for years recognize the architecture in this book as familiar. Skills resemble the configuration files that have always shaped good engineering systems. Subagents resemble the worker pools and microservice patterns that decompose monolithic systems into focused specialists. Hooks resemble the middleware, the policy enforcement points, and the observability layers that wrap deterministic guarantees around probabilistic services. The three-pillar architecture is not, in its bones, a new invention. It is the application of well-established software-architecture patterns to the specific problem of working with an AI assistant. This continuity is reassuring, because it means that the discipline this book asks the reader to develop is the same discipline that produces good software in general. A developer who builds her Claude Code environment well becomes a slightly better software engineer along the way, because the patterns she practices on her environment are the same patterns she should practice on her production systems. The book's teaching, in this sense, transfers beyond the tool.

Case Study: Building a Full-Stack Development Environment

“The proof of the architecture is in the building. Until something stands, the design is only an opinion.”

a structural engineer, anonymous

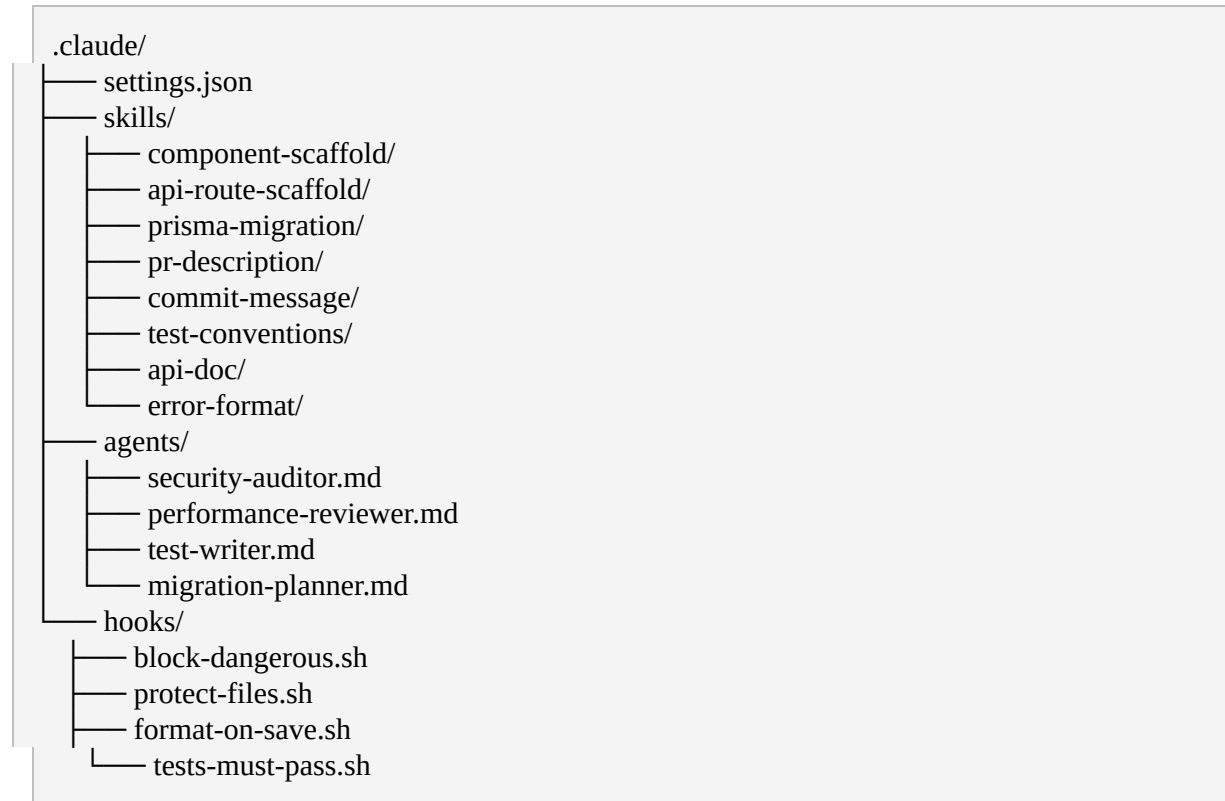
This chapter is a build. The reader will follow, end to end, the construction of a complete Claude Code environment for a single specific kind of work: full-stack web development in TypeScript, with React on the front end and Node with PostgreSQL on the back end. The reader does not need to be a TypeScript developer for the chapter to be useful. The point of the case study is the shape of the construction, not the specific technologies. A reader who works in a different stack can read the chapter as a template and substitute her own languages, her own frameworks, her own conventions. The skeleton is what matters. The flesh is illustrative.

The developer in this case study is a senior engineer at a small product company. She works in a single repository that contains a Next.js front end, a Node API server, a PostgreSQL database with Prisma as the ORM, a test suite written in Vitest, and deployment configuration in Docker and Kubernetes. The team has agreed conventions for how new features are scaffolded, how tests are written, how database migrations are produced, how PR descriptions are written, and how releases are tagged. The conventions are presently maintained in a wiki that nobody reads after their first week on the team. The developer wants to encode the conventions into her Claude Code environment so that the environment enforces and applies them automatically, freeing her from having to remember them and freeing her teammates from having to be reminded of them in every code review.

The build will produce, in the project’s `.claude` directory, a configuration that includes eight skills, four subagents, and six hooks. By the end of the chapter, the directory will be committed to the repository, every developer who clones the project will inherit the environment

automatically, and the daily experience of working in the project will be substantially different from what it was before the build began.

Begin with the directory structure. The `.claude` directory at the project root is organized as follows.



Each of the components has a specific job in the larger system. The skills encode the project's knowledge. The subagents provide specialized roles for the major review and authoring tasks. The hooks enforce the deterministic guarantees. Together, the configuration makes a substantial development workflow possible to perform in a single Claude Code session, with the environment carrying most of the load that the developer used to carry herself.

Begin building with the skills, because skills are the foundation. The first skill is `component-scaffold`, which generates a new React component in the project's conventional layout. A React component in this codebase is a folder containing the component file, a test file, a Storybook story, and an index re-export. The folder convention is that each component lives at `src/components/[ComponentName]/`, with PascalCase naming throughout. The skill captures this convention and the templates for each generated file.

```
---
name: component-scaffold
description: Scaffolds a new React component in the project's conventional layout, complete with
test file, Storybook story, and index export. Use whenever the user asks to "create a new
component," "scaffold a component called X," "set up a React component," or describes wanting
a new component file structure created.
allowed-tools: Read, Write, Glob
---
```

Component Scaffold

When asked to scaffold a component, follow this procedure.

1. Get the component name from the user. Convert to PascalCase if needed.
2. Create the folder at `src/components/[ComponentName]/`.
3. Create the four files using the templates in `assets/`:
 - `[ComponentName]/index.ts`, re-exports the component as a named export
 - `[ComponentName]/[ComponentName].tsx`, the component itself
 - `[ComponentName]/[ComponentName].test.tsx`, Vitest test file
 - `[ComponentName]/[ComponentName].stories.tsx`, Storybook story
4. Substitute the placeholder `\_\_COMPONENT\_NAME\_\_` with the actual name in each file.
5. Confirm by listing the four files created.

Use functional components with hooks. Use TypeScript strict mode. Define props as a named interface above the component. Use CSS modules for styling, with a matching .module.css file.

The second skill is `api-route-scaffold`, which generates a new API route in the Next.js conventional layout, with the route handler, the validation schema, the test file, and the OpenAPI documentation fragment. The skill captures the team's convention that every route has explicit validation using Zod, every route has a test that exercises the happy path and at least two error cases, and every route has a corresponding OpenAPI fragment that lives alongside the route file.

```
---
name: api-route-scaffold
description: Scaffolds a new API route in the Next.js conventional layout, complete with handler,
Zod validation schema, test file, and OpenAPI fragment. Use whenever the user asks to "create a
new API route," "scaffold an endpoint," "add a route called X," or describes wanting a new API
endpoint set up.
allowed-tools: Read, Write, Glob
---
```

API Route Scaffold

When asked to scaffold an API route, follow this procedure.

1. Get the route path and HTTP method from the user.
2. Create the route handler at `app/api/[path]/route.ts`.
3. Create the validation schema at `app/api/[path]/schema.ts`. All input must be validated with Zod before being processed.
4. Create the test file at `app/api/[path]/route.test.ts`. Cover happy path, validation failure, and one business-logic failure.
5. Create the OpenAPI fragment at `app/api/[path]/openapi.yaml`. Document the request shape, the response shape, and all error codes.
6. Wire the route into the API surface following the project's convention.

Every route must validate its input with Zod. Every route must produce errors in the project's standard error format (see the error-format skill). Every route must have a test file with at least three tests.

The Migration Skill

The third skill is prisma-migration, which produces a Prisma migration file from a description of the schema change, with the up migration, the down migration, and a note about whether the change is backward-compatible.

```
---
name: prisma-migration
description: Produces a Prisma migration from a described schema change, with up and down
migrations and a backward-compatibility note. Use whenever the user asks to "create a
migration," "add a migration for X," "I need to change the schema," or describes wanting a
database schema change made.
allowed-tools: Read, Write, Bash
---
```

Prisma Migration

When asked to create a migration, follow this procedure.

1. Read the current schema at `prisma/schema.prisma`.
2. Apply the change to the schema. Be conservative: prefer additive changes (new columns, new tables) over destructive ones (dropped columns, renamed tables) whenever possible.
3. Generate the migration file using `npx prisma migrate dev --create-only --name [descriptive\_name]`.
4. Read the generated migration. Verify it does what was intended.
5. If the migration is destructive, write a note at the top of the migration file explaining what is being changed and what the rollback strategy is.
6. Report:
 - What changed

- Whether the change is backward-compatible (can old code run against the new schema)
- Whether a data migration is needed in addition to the schema migration
- Recommended deploy order (migration before code, or code before migration)

Do not run `prisma migrate deploy` or apply the migration. The user reviews and applies it manually.

The fourth skill is pr-description, which produces a pull request description in the team's standard format. This is a near-copy of the personal pr-description skill from Chapter Nine, adjusted for the team's specific conventions.

```
---
name: pr-description
description: Drafts a pull request description in the team's standard format. Use whenever the user
asks for a PR description, pull request description, PR body, or asks "write a PR for this branch,"
"make me a PR description." This skill is for drafting the description of a PR you are creating, not
for reviewing one.
allowed-tools: Bash, Read, Grep
---
```

PR Description

When asked for a PR description, follow this procedure.

1. Identify the diff. Run `git log` and `git diff` against the merge base on main.
2. Read the diff in full.
3. Produce the description in this exact format:

Summary

A single sentence in the imperative mood describing the change.

What changed

A bulleted list of concrete changes, one bullet per logical unit.

Schema changes

If the diff includes any Prisma migrations, summarize the schema changes here. State whether they are backward-compatible. Omit this section if there are no migrations.

Testing

Describe how the change was tested. List any new test files created.

Risk

One sentence categorizing risk as low, medium, or high with a one-line reason.

Deploy notes

Any non-default deployment considerations, such as environment variable changes, manual data migrations, or required rollouts. Omit if not applicable.

Do not invent changes that do not appear in the diff.

The fifth skill is commit-message, identical to the Chapter Five version but installed at the project level rather than the user level so that every developer in the team uses the same conventions.

The sixth skill is test-conventions, which encodes the project's specific test conventions: Vitest, the location pattern for test files, the use of MSW for HTTP mocking, the convention of one describe block per function and one it block per scenario. The skill is loaded by the test-writer subagent when it begins working, so that the subagent automatically applies the project's conventions without needing to be briefed every time.

```
---
name: test-conventions
description: Encodes this project's test conventions, including framework choice, file layout,
mocking patterns, and naming. Activated by the test-writer subagent and by the user when
explicitly asked about testing conventions.
allowed-tools: Read
---
```

Test Conventions

This project uses Vitest. Test files are colocated with the file under test, with a ``.test.ts`` or ``.test.tsx`` suffix.

Structure

- One ``describe`` block per public function or component.
- One ``it`` block per scenario. Every public function should have at minimum: happy path, one boundary, one error.
- Use ``beforeEach`` for setup that resets state. Avoid ``beforeAll``.

Mocking

- Use Vitest's ``vi.mock`` for module mocks.
- Use MSW (Mock Service Worker) for HTTP mocks. Set up handlers in the test file's ``beforeAll``.
- Never make real network calls in tests.
- Never write to the real file system in tests. Use ``memfs`` if file operations are needed.

Database

- Tests that touch the database use a transaction that is rolled back after the test.
- Never use the production or staging database for tests.

Async

- Always await async operations explicitly. Do not rely on returned promises being awaited by the test runner.
- Use `vi.useFakeTimers()` for tests that depend on time.

Naming

- Test files: `[FileName].test.ts` or `[ComponentName].test.tsx`.
- Test descriptions: imperative, e.g. "creates a user when given valid input," not "should create a user."

The seventh skill is `api-doc`, which produces an OpenAPI documentation fragment for a given route. It is similar to the `api-route-scaffold` skill in that it produces OpenAPI output, but its job is to update existing documentation rather than to create scaffolding for new routes.

The eighth skill is `error-format`, which encodes the team's standard error format. The format includes a numeric code, a human-readable message, and an optional details object. The skill is short but loaded by other skills and subagents whenever they need to produce or parse errors.

With the eight skills in place, the project has its conventions encoded as a small library that any session in the project can draw on automatically. Move next to the subagents, four of which complete the specialist team.

The first subagent is `security-auditor`, identical in shape to the version from Chapter Eleven. The second is `performance-reviewer`, also identical to its Chapter Thirteen counterpart. Both are general-purpose specialists that benefit from the project's skills when they activate.

The Test Writer Subagent

The third subagent is `test-writer`, customized slightly for this project. The customization is in the description, which now mentions Vitest and the project's specific test conventions, and in the system prompt, which directs the subagent to load the `test-conventions` skill at the start of its work.

```
---
name: test-writer
description: Writes Vitest test files for a given function or module, following this project's test
conventions. Use whenever the user asks for tests, says "add coverage," "write a test for this," or
finishes a feature without tests in place.
tools: Read, Write, Bash, Glob
model: sonnet
---
```

You are a senior test engineer. Your job is to write tests that catch real bugs, not tests that only exercise the happy path so that coverage numbers look better.

Your method:

1. Activate the project's test-conventions skill at the start of your work.
2. Read the file under test and any closely related files needed to understand its behavior.
3. Identify the public surface of the file: functions, methods, classes that other code calls.
4. For each item on the public surface, write tests covering at minimum:
 - The happy path
 - At least one boundary condition
 - At least one error or invalid-input case
5. Mock external dependencies using the project's mocking patterns (Vitest `vi.mock` for modules, MSW for HTTP).
6. Place test files alongside the code under test, following the project's naming convention.

What you do not do:

- You do not modify the code under test, even if you discover bugs while writing tests. Note the bug as a comment in the test and flag it in your final summary.
- You do not write tests that only call the function and assert that it does not throw.
- You do not mock the function under test. You mock its dependencies.
- You do not produce excess of nearly-identical tests. Three meaningful tests beat ten redundant ones.

End with a list of the test files you wrote and any bugs or concerns you noticed in the code under test.

The fourth subagent is migration-planner, used when the developer is contemplating a substantial migration such as a Next.js major-version upgrade or a switch from Prisma to a different ORM. The subagent is identical in shape to its Chapter Thirteen version.

With four subagents installed, the project has access to specialized parallel review and authoring capabilities for security, performance, testing, and migration planning. The orchestrator pattern from Chapter Twelve

activates naturally when the developer asks for broad reviews, and the four subagents work in parallel to produce a multi-dimensional report.

Move next to the hooks. Six hooks complete the configuration, and they live in the `.claude/hooks` directory as separate scripts referenced from the `settings.json` file. The `settings.json` itself looks like this.

```
{
  "hooks": {
    "SessionStart": [
      {
        "hooks": [
          {
            "type": "command",
            "command": "echo '## Branch'; git branch --show-current; echo '## Status'; git status --short"
          }
        ]
      }
    ],
    "PreToolUse": [
      {
        "matcher": "Bash",
        "hooks": [
          {
            "type": "command",
            "command": "$CLAUDE_PROJECT_DIR/.claude/hooks/block-dangerous.sh"
          }
        ]
      },
      {
        "matcher": "Edit|Write|MultiEdit",
        "hooks": [
          {
            "type": "command",
            "command": "$CLAUDE_PROJECT_DIR/.claude/hooks/protect-files.sh"
          }
        ]
      }
    ],
    "PostToolUse": [
      {
        "matcher": "Edit|Write|MultiEdit",
        "hooks": [
          {
            "type": "command",
```

```

        "command": "$CLAUDE_PROJECT_DIR/.claude/hooks/format-on-save.sh"
      },
      {
        "type": "command",
        "command": "$CLAUDE_PROJECT_DIR/.claude/hooks/tests-must-pass.sh"
      }
    ]
  },
  "Stop": [
    {
      "hooks": [
        {
          "type": "command",
          "command": "if [ \"$(git status --porcelain | wc -l)\" -gt 0 ]; then echo
'{"decision\":\"block\",\"reason\":\"Uncommitted changes present. Commit or stash before
ending the session.\"'}; fi"
        }
      ]
    }
  ]
}
}

```

The `block-dangerous.sh` and `protect-files.sh` scripts are the same security gates from Chapter Seventeen, slightly expanded for this project to add the production deployment paths and the `.env.production` file as protected. The `format-on-save.sh` script runs Prettier and ESLint on every file that Claude edits or writes. The `tests-must-pass.sh` script runs the test suite scoped to the changed file's area, and if any tests fail, returns a JSON decision-block that surfaces the failure to Claude with a directive to fix the failing tests before continuing. Together, the four `PreToolUse` and `PostToolUse` hooks form a tight enforcement pipeline that runs on every action Claude takes inside the project.

The `Stop` hook is a final guard against ending a session with uncommitted changes. The hook checks the `git status` at session end, and if any changes are present, it returns a JSON decision-block that prevents the session from ending and surfaces a clear instruction to commit or stash before continuing. This is a small but real piece of workflow enforcement; it makes it impossible to walk away from the terminal with work in an inconsistent state.

The configuration is now complete. Eight skills, four subagents, six hooks. The `.claude` directory is committed to the repository, and from the next session forward, every developer who works in this project inherits the entire environment. The work that previously required the developer to remember the conventions, apply them manually, and verify them after the fact, now runs automatically. The conventions are encoded in skills. The specialist work is dispatched to subagents. The boundaries are enforced by hooks.

A Day in the Environment

A walk through a representative day in the project will make the difference visible. The developer opens Claude Code in the morning. The `SessionStart` hook fires and shows her the current branch and any uncommitted changes from the previous day, which there are none of, because the `Stop` hook from the previous session would have prevented her from leaving the terminal with uncommitted work. She decides to add a new feature: a user profile endpoint with avatar upload support. She types into Claude Code, in plain language, scaffold an API route at `/api/users/profile` that accepts PUT requests with a multipart body containing an avatar file.

The `api-route-scaffold` skill fires automatically because the request matches its description. Claude creates the route handler, the Zod schema, the test file with three tests, and the OpenAPI fragment. The `format-on-save` hook runs Prettier on each new file as it is written. The `tests-must-pass` hook runs the project's tests after each write; the new tests pass, because the scaffold includes them complete, and the hook is silent.

The developer then asks for a commit. The `commit-message` skill fires. Claude reads the staged changes, produces a conventional commit message in the project's format, and presents it. The developer approves it. The commit happens. The `format-on-save` hook runs again on each file, finds nothing to change, and exits silently.

She then asks for a PR description. The `pr-description` skill fires. Claude reads the branch's diff against main, produces a PR description in the team's exact format with all five sections including the deploy notes section that flags the new endpoint as needing the file-upload size limit increased in the production environment, and presents it. She copies it into the GitHub PR.

She wants a thorough review of the change before requesting reviewers. She types, perform a full review of this branch covering security, performance, testing, and architecture. The orchestrator pattern recognizes the multi-dimensional shape of the request and dispatches four subagents in parallel: security-auditor, performance-reviewer, the test-writer (in review mode), and an architecture-reviewer. Each runs in its own context. Each loads the project skills it needs. Two minutes later, four reports return. The unified review notes that the file upload code does not validate the file size on the server side (security flag), that the avatar processing happens synchronously in the request handler when it should be queued (performance flag), and that the test coverage is solid. She addresses the security and performance flags before requesting reviewers.

The whole interaction has taken her twenty-five minutes. Without the environment, the same work would have taken closer to two hours, and several of the issues the review caught would likely have been merged in and discovered later. The environment did the conventions work. The environment did the review work. The environment enforced the boundaries. She did the design and judgment work, which is what she was hired for in the first place.

A reflection on what the case study demonstrates. The configuration above is not large. The .claude directory at the end of the build is roughly three thousand lines of configuration spread across twenty files. A single developer can build it in a long afternoon. Once built, it persists. It travels with the project to every developer who clones the repository. It saves substantial time on every session in the project, indefinitely. The investment-to-benefit ratio is exceptional, and the case study is meant to convince the reader that her own project has a similar configuration latent inside it, waiting to be built.

A second walkthrough of a different kind of session is worth including, because the morning scenario above showed the environment helping with a relatively well-defined feature build. A different kind of session, the bug investigation, exercises different parts of the environment and is worth seeing in concrete form.

A Bug Investigation

The developer notices that errors from the avatar upload endpoint have been spiking in the production monitoring dashboard. She opens Claude Code in the project. The `SessionStart` hook fires and shows her the current branch (a feature branch she had been working on yesterday) and any uncommitted changes (none, because the Stop hook from the previous session prevented her from leaving with dirty state). She switches to main and pulls the latest. She types into Claude Code, recent errors from `/api/users/profile` suggest something is failing on large file uploads; investigate.

The orchestrator pattern recognizes the investigative shape of the request, but unlike the morning's feature work, no existing subagent perfectly matches investigation. The main session handles the work directly, reading the route handler, reading the recent commits to the file, reading the error logs the developer points it at. It identifies a likely cause: the Zod schema does not enforce a maximum file size, and the underlying middleware that should have caught oversized uploads has a configuration drift between staging and production. The main session proposes a fix: tighten the schema to include explicit size validation, and adjust the middleware configuration. The developer agrees with the diagnosis.

She types, write the fix and add a test for the size validation. The `api-route-scaffold` skill is not the right fit here, because the route already exists. The `test-writer` subagent activates because the request matches its description. The subagent reads the route, reads the existing test file, adds three new test cases for size validation including one boundary, one over-size, and one with the new error format, and ends with a note about the test it wrote. The `format-on-save` hook runs Prettier on the modified test file. The `tests-must-pass` hook runs the tests; they pass. She asks for a commit. The `commit-message` skill produces a message: `fix(users): enforce avatar size limit at the API boundary`. She commits, pushes, opens a PR, asks for a PR description, the `pr-description` skill produces it. The whole investigation takes forty minutes, from noticing the error spike to opening the PR.

The example shows the environment functioning at a different scale from the morning scenario. No single subagent dominated the work. No single skill was the centerpiece. The environment's value showed up in the small accelerations along the way: the hook that surfaced the branch state at session start, the subagent that wrote the tests in the project's conventions, the formatter and test runner that ran without prompting, the commit-

message and PR-description skills that produced consistent outputs at the moments they were needed. The cumulative effect of these small accelerations is what makes the environment substantially better than vanilla Claude Code, and the cumulative effect is most visible in routine sessions like this one rather than in the showcase orchestration of the morning scenario.

A third reflection concerns the team-adoption arc that this kind of environment enables. Once the configuration is committed to the repository, every developer on the team experiences the same environment. New team members inherit it on their first clone. Conventions that previously had to be taught through code review or onboarding documentation are now enforced by the tooling itself. The team's collective output becomes more consistent. The collective time saved compounds across every developer. A team of five developers, each saving ninety minutes a day through their inherited environment, recovers nearly forty hours a week of engineering time. That recovered time can be spent on new features, on technical debt, on the hard problems that the conventions could never have solved on their own. This is the deeper argument for treating the `.claude` directory as a first-class engineering asset alongside the source code: the leverage compounds across the team, not just within any one developer.

A second reflection concerns the team dimension. The configuration above is committed to the repository, which means that every developer on the team uses the same skills, the same subagents, and the same hooks. When a new developer joins the team, she receives the entire environment automatically when she clones the project for the first time. She does not need to be onboarded into the conventions, because the conventions are enforced by the environment. The wiki that nobody read becomes irrelevant, because the wiki has been replaced by configuration that the tool itself honors. This is how engineering conventions actually take hold in a team: not through documentation, not through training, but through tooling that makes the right thing automatic and the wrong thing impossible. The Claude Code environment is a particularly powerful version of this pattern, because it operates at the level of the developer's primary AI assistant, which means that almost every action a developer takes inside Claude Code is shaped by the environment.

A final reflection on the case study concerns the cost of building it. The afternoon investment to produce the configuration above is real but bounded. A developer who has read this book carefully and who has practiced the patterns on a few personal skills can produce an environment of this size in roughly six to eight hours of focused work. The same developer working on a fresh codebase she has not used Claude Code in before would need closer to twelve hours, because the project conventions have to be written down somewhere before they can be encoded in skills. Either way, the investment is on the order of a single working day, and the return on that investment compounds across every future session. A developer who saves an average of ninety minutes per working day through her environment recovers her initial investment in less than a working week. After that, the savings are pure leverage. This is the practical case for treating the build of the .claude directory as a high-priority engineering task rather than as a side hobby. The numbers favor the build, and they favor it dramatically.

The next chapter takes the same architecture and applies it to a very different shape of work. The full-stack developer in this case study had a clearly engineering-shaped job. The next case study features a solo technical founder whose work spans product, marketing, and operations, and whose Claude Code environment looks substantially different from this one even though the underlying architecture is identical. The contrast is instructive, because it shows that the three-pillar framework is not specific to engineering work; it is a general pattern for engineering one's relationship with an AI assistant, and the pattern adapts cleanly to almost any kind of knowledge work. The second case study begins in the chapter ahead.

Case Study: The Solo Technical Founder's Claude Code Setup

“The same architecture, applied to different work, produces different environments. The architecture is the constant. The work is the variable.”

a software architect, anonymous

This chapter is the second case study, and it is deliberately different in shape from the first. The previous chapter built an environment for a single specific kind of engineering work. This chapter builds an environment for a solo founder whose work spans product, engineering, marketing, customer support, and operations, all from a single Claude Code installation. The shift in shape is the point of the chapter. The three-pillar framework was developed in this book using engineering examples, but the framework is not specific to engineering. It is a general pattern for engineering one’s relationship with an AI assistant, and the pattern adapts cleanly to almost any kind of knowledge work. By the end of this chapter, the reader will have seen the same architecture produce two very different environments, and she will have the conceptual tools to design a third for her own situation, whatever that situation looks like.

The founder in this case study runs a small SaaS company alone. The product is a developer tool, a paid service that helps engineering teams analyze and visualize their CI/CD pipelines. She wrote the original prototype, she runs the marketing, she handles customer support, she manages the infrastructure, and she does the bookkeeping. The company has roughly two hundred paying customers, the revenue is enough to support her full-time, and she is at the stage where her single biggest constraint is her own time. She has been a heavy Claude Code user for several months, and she has decided that her environment is overdue for serious engineering. The case study walks through the build.

Begin with the diagnosis. The founder’s work has a distinctive shape that is worth naming explicitly, because it influences every later design decision. Her work is multi-domain. In a single working day, she might

write a product roadmap section, draft a sales email to a prospective customer, debug a production issue in the ingestion service, write a blog post about a recent feature, respond to a support ticket about API rate limits, update the company's investor dashboard, and produce next week's marketing calendar. No single role-shaped specialist would cover this range, and a generic Claude Code session that handled all of it would suffer from the context-pollution problem in an extreme form, because the residue of writing a sales email would still be in the context when she pivoted to debugging a production issue.

Her environment, therefore, needs to be designed around clean context separation between domains. Skills will encode the conventions that apply within each domain. Subagents will provide specialist roles for the major recurring tasks. Hooks will enforce boundaries at the lifecycle moments where the founder's attention is least reliable. The directory structure will reflect the multi-domain nature of the work, with skills organized so that the right ones activate for the right kind of work without firing in inappropriate contexts.

The `.claude` directory at the user level (since this is a personal environment that travels across all of her projects) is organized as follows.



```

├── weekly-review/
├── writing/
│   ├── humanize-text/
│   └── shorten/
├── agents/
│   ├── product-strategist.md
│   ├── marketing-writer.md
│   ├── support-helper.md
│   ├── financial-analyst.md
│   └── code-reviewer.md
├── hooks/
│   └── (minimal, see below)

```

The most striking difference from the previous case study is the breadth of skills. The founder has roughly fifteen skills organized into six domain folders. The folder organization is a convention rather than a technical requirement; Claude Code does not care how skills are organized inside the skills directory. The folders exist for the founder's own benefit, so that when she opens the directory to add or edit a skill, she can find what she is looking for at a glance. The discipline of organizing skills by domain pays off as the library grows, because a flat directory with thirty skills becomes hard to navigate, while a folder-organized one stays legible indefinitely.

Begin building with the marketing domain, because it is the most distinctive of the founder's use cases. The marketing-writer subagent is the role specialist that handles most marketing-related tasks. It has its own system prompt, tuned for the founder's voice and the company's tone, and it loads relevant skills as it works.

```

---
name: marketing-writer
description: Drafts marketing content including blog posts, feature announcements, Twitter threads, newsletter sections, and landing page copy, in the company's established voice. Use whenever the user asks for marketing copy, "draft a blog post," "write an announcement," "give me a tweet thread," or describes wanting outbound marketing content created.
tools: Read, Write, Glob
model: sonnet
---

```

You are the marketing voice of a small developer-tools company. The company sells a service that helps engineering teams analyze and visualize their CI/CD pipelines. The audience is technical: engineering managers, platform engineers, and SRE teams.

The voice you write in is honest, technically precise, and slightly understated. You do not use marketing superlatives. You do not say "powerful," "robust," "elegant," or "industry-leading." You describe what the product does, who it helps, and what changes for the user. You let the value speak for itself.

Your method:

1. Activate the relevant skill for the format requested (blog post outline, feature announcement, twitter thread, newsletter section).
2. Activate the humanize-text skill at the end of every draft to remove AI-flavored vocabulary and patterns.
3. Read the existing marketing material in the company's content directory to match the voice.
4. Produce the draft in the requested format.
5. End with a short note about the angle you took and a question if you are uncertain whether the angle matches the founder's intent.

What you do not do:

- You do not invent product features. If the request implies a feature you have not seen documentation for, ask the founder rather than guess.
- You do not use language the founder has explicitly excluded.
- You do not produce vague marketing-speak. Every sentence should communicate something specific.
- You do not pad. A sharp short post beats a padded long one.

End with the draft and a brief note about voice or angle.

Marketing Workflows

The subagent has access to the company's marketing skills, including the format-specific ones (blog-post-outline, feature-announcement, twitter-thread, newsletter-section), and the general writing skills (humanize-text, shorten). When the founder asks for a marketing piece, the marketing-writer subagent activates, the relevant format skill fires inside it, the humanize-text skill applies its pass at the end, and the result comes back as a draft in the founder's voice with the company's conventions applied.

The blog-post-outline skill is small but important, because it captures the founder's preferred structure for a blog post.

name: blog-post-outline

description: Produces a blog post outline in the company's standard structure. Use whenever the user asks for a blog post, "outline a post about X," "draft a blog," or describes wanting a new blog post created.

allowed-tools: Read

Blog Post Outline

When asked for a blog post, structure it as follows.

1. **Hook (one paragraph)**: Open with a specific, concrete observation or question. Avoid abstract framing. The hook should make a technical reader want to keep reading.
2. **The problem (one or two paragraphs)**: Describe the actual pain the post will address, in technical terms. Use a specific example. Do not pretend the problem is more general than it is.
3. **The mental model or framework (two or three paragraphs)**: Lay out the way of thinking about the problem that the post is going to argue for. This is the substantive contribution.
4. **The application (one or two examples)**: Show the framework applied to one or two concrete cases. Use real code or real configuration. Do not abstract too far.
5. **The trade-offs (one paragraph)**: Be honest about what the framework does not solve, where it does not apply, and what the costs are.
6. **The takeaway (one or two sentences)**: A single concrete statement of what the reader should do or remember.

Length target: 1,200 to 1,800 words. Longer is acceptable when the topic justifies it. Shorter is acceptable when the point is sharp.

After producing the outline, ask the founder to confirm the angle before drafting the full post.

Move to the support domain. The support-helper subagent handles customer support tasks, with the support-reply skill providing the format and the ticket-summary skill providing a way to compress long ticket threads into actionable summaries.

name: support-helper

description: Drafts customer support replies and summarizes long support threads. Use whenever the user mentions a customer ticket, says "respond to this support ticket," "draft a reply for the customer," "summarize this thread," or describes needing help with a customer interaction.

tools: Read, Write

model: sonnet

You are a senior customer support engineer for a small developer-tools company. The customers are technical: engineers, engineering managers, and SREs. The support tone is friendly, direct,

and substantive. You do not use phrases like "I completely understand your frustration." You acknowledge the issue concretely and move quickly to addressing it.

Your method:

1. Read the customer's message carefully. Identify what they actually need: a fix, an explanation, a workaround, an acknowledgment that something is broken.
2. If the answer involves product knowledge you are not sure about, flag the uncertainty rather than guess. The founder would rather you ask than make something up.
3. Activate the support-reply skill for format guidance.
4. Draft the reply.
5. If the issue is a bug or a known limitation, end with a sentence that indicates whether it is on the roadmap to fix.

What you do not do:

- You do not promise fixes you cannot guarantee.
- You do not use customer service platitudes.
- You do not pad. Customers' time is valuable.
- You do not ignore technical specifics. If the customer mentions a version number, an error code, or a configuration detail, address it directly.

End with the draft reply and a one-line note about whether you are confident in the technical content or whether the founder should review before sending.

Move to the operations domain. The financial-analyst subagent handles tasks like generating investor updates, summarizing the company's monthly financials, and producing weekly review reports.

name: financial-analyst
description: Produces investor updates, monthly financial summaries, and weekly business reviews using data from the company's metrics dashboards and accounting exports. Use whenever the user asks for an investor update, financial summary, monthly review, weekly review, or describes wanting business metrics analyzed.
tools: Read
model: opus

You are a senior financial analyst working with a small SaaS founder. Your job is to take raw metrics and accounting data and produce clear, honest reports that are useful for the founder's decision-making and for investor communications.

Your principles:

- Numbers without context are not useful. Every metric should be paired with a comparison: month-over-month, year-over-year, or against a stated target.

- Be honest about bad numbers. A report that hides a slowdown is a report that delays the founder's response to a real problem.
- Be specific about what is changing. "Revenue grew" is not a useful sentence. "Revenue grew 8% MoM, driven by three enterprise upgrades from the December cohort" is.
- Distinguish leading indicators from lagging indicators. New trial signups are leading. MRR is lagging. Both matter, but they need different framing.

Your method:

1. Activate the relevant format skill (investor-update, financial-summary, weekly-review).
2. Read the data sources the founder has pointed you at.
3. Produce the report in the format the skill specifies.
4. End with a short section labeled "Things I would watch," where you flag anything in the data that you think deserves the founder's attention even if it is not yet a clear trend.

What you do not do:

- You do not invent numbers. If a number is not in the data, you do not include it.
- You do not editorialize beyond the "things I would watch" section.
- You do not produce reports longer than necessary. Density beats length.

End with the report and the "things I would watch" section.

The other subagents in this environment include product-strategist, which handles product roadmap and PRD work, and code-reviewer, which is a near-copy of the engineering-focused subagent for the times when the founder is reviewing her own code.

Move now to the hooks. The hooks for this environment are deliberately minimal, because the founder's work does not have the same enforcement requirements as the team-engineering project from the previous chapter. Her work is largely solo, the consequences of mistakes are concentrated on her own time rather than on a team's infrastructure, and over-engineering the hook layer would produce friction without proportionate benefit. Her `settings.json` contains only three hooks.

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Bash",
        "hooks": [
          {
            "type": "command",
            "command": "$HOME/.claude/hooks/block-dangerous.sh"
          }
        ]
      }
    ]
  }
}
```

```

    }
  ],
  {
    "matcher": "Edit|Write|MultiEdit",
    "hooks": [
      {
        "type": "command",
        "command": "$HOME/.claude/hooks/protect-files.sh"
      }
    ]
  }
],
"Notification": [
  {
    "hooks": [
      {
        "type": "command",
        "command": "osascript -e 'display notification \"Claude needs your input\" with title \"Claude Code\"'"
      }
    ]
  }
]
}
}

```

The first hook blocks dangerous shell commands. The second protects sensitive file paths, including the founder's `.env` files, her accounting exports, and her investor communications archive. The third produces a desktop notification whenever Claude needs her attention, because she often has Claude Code running in a background terminal while she works on something else and would otherwise miss the prompt for input.

A Minimal Hook Layer

The minimalism of the hook layer is a design choice, not an oversight. The founder considered adding test-running hooks, formatting hooks, and several other guards from the previous case study, but concluded that the friction of those hooks would outweigh their benefit in her solo, multi-domain context. Her work moves between writing, code, and analysis throughout the day, and a hook that runs the test suite after every Edit would slow down a writing session where she is editing markdown files that have nothing to do with code. The right call for her was to keep the

hook layer light and let the discipline live in her skills and subagents instead. A different founder, with a different shape of work, might make different choices. The architecture accommodates both.

A walk through a representative day in this environment will make the difference visible. The founder opens Claude Code in the morning. She has a busy day planned: a customer support reply, a section of next month's investor update, a marketing post about a feature she shipped last week, and some debugging of a production issue she noticed in last night's metrics.

She starts with the support reply. She reads the ticket, then types into Claude Code, draft a reply to this customer about the rate-limiting issue. The support-helper subagent activates because the request matches its description. The subagent reads the customer's message, drafts a reply that acknowledges the issue concretely and offers a workaround, and ends with a note that the rate-limit configuration change the customer is asking about is on the roadmap for the next release. She reviews the draft, makes a small wording change, and sends it.

She moves to the investor update. She types, write the metrics section of this month's investor update; data is in /investors/data/january.csv. The financial-analyst subagent activates. It reads the data, produces a section that opens with the headline metric, follows with the MoM and YoY comparisons, breaks down the drivers, and ends with the "things I would watch" section that flags an unusual dip in trial-to-paid conversion in the second week of the month. She thanks the analyst for the catch, makes a note to investigate, and copies the section into her investor update document.

She moves to the marketing post. She types, draft a blog post about the new pipeline visualization feature we shipped last week. The marketing-writer subagent activates. The blog-post-outline skill fires inside it. The subagent produces an outline following the company's standard structure, asks her to confirm the angle, she confirms, and the subagent drafts the full post. The humanize-text skill applies its pass at the end, removing AI-flavored vocabulary and tightening the prose. She reads the draft, finds it about ninety percent of where she wants it, makes a few specific edits, and schedules it for publication.

She moves to the production debugging. The production code lives in a separate project directory, and she opens Claude Code there. The project-level configuration in that directory is much closer to the engineering case study from the previous chapter, with skills, subagents, and hooks tuned for the engineering work. The founder's personal user-level configuration from this chapter still applies, but the project-level configuration extends it with engineering-specific tooling. She debugs the production issue, fixes it, gets a code review from her code-reviewer subagent, and ships the fix. The whole cycle takes thirty minutes.

The total time across the four tasks is roughly two and a half hours. Without the environment, the same four tasks would have taken her closer to five hours, and several of them would have come out worse. The investor update would have lacked the "things I would watch" framing. The marketing post would have sounded like every other AI-drafted marketing post. The support reply would have either been too long or too short, depending on her mood. The environment did not make her smarter. It made her output more consistent and her time more efficient, by absorbing the conventions, the formats, and the role-shaping that her work requires.

Generalizing Beyond This Case

A reflection on what this case study demonstrates relative to the previous one. The two environments are strikingly different in surface appearance. The full-stack engineering environment has eight skills focused on engineering tasks, four engineering specialists, and six enforcement hooks. The solo founder environment has fifteen skills spanning six domains, five role specialists across product, marketing, support, ops, and engineering, and three minimal hooks. The two environments could not look less alike on first glance. But the underlying architecture is identical. Both use the three-pillar framework. Both organize knowledge into skills, roles into subagents, and boundaries into hooks. Both compose the three pillars to produce capabilities that no single pillar could provide. The framework is what makes the two environments coherent, even though their content is entirely different.

A second reflection concerns the generalizability of the framework. The two case studies in this part of the book are deliberately different to make the generalizability point. A reader who works in a third shape of work,

neither full-stack engineering nor solo SaaS founding, can take the framework and apply it to her own context. The classification questions still apply: knowledge gap means skill, attention gap means subagent, guarantee gap means hook. The composition principle still applies: skills inside subagents, subagents in parallel orchestration, hooks at the lifecycle boundaries. The maintenance discipline still applies: start small, add components when their need is concrete, remove components that have not earned their place. The reader who has read this book carefully has a complete blueprint for engineering her own Claude Code environment, regardless of what kind of work she does.

A third reflection concerns the founder's specific situation as illustrative of a broader pattern. The founder in this case study is the entire labor force of her company. Her time is the binding constraint. The Claude Code environment is, in her case, not just a productivity improvement; it is the difference between a one-person company that can sustain its operations and one that cannot. The leverage that the environment provides is not optional for her. It is the precondition for the company existing at all in its current form. A reader who is also operating at this kind of scarce-time, multi-domain shape of work should recognize her own situation in the case study and recognize that the architecture this book has been teaching is, for her, a strategic asset rather than a developer convenience.

A second worked walkthrough is worth including, because the Friday weekly-review session exercises a different combination of the founder's environment than the daily-task scenario above and shows the architecture from another angle.

It is Friday afternoon. The founder sits down to do her weekly review, which she does every Friday before she logs off for the weekend. The review has three components: a financial weekly summary, a writeup of what shipped during the week, and a planning section for the following week. She opens Claude Code in the directory where she keeps her business operations files. She types, `run my weekly review`.

The financial-analyst subagent activates first, because the weekly-review skill it is paired with carries the highest priority in the matching. The subagent reads the metrics dashboard export from earlier that day, reads the previous week's summary for comparison, and produces the financial

section: revenue change, customer count change, churn analysis, and the "things I would watch" section. The "things I would watch" section flags an unusual cluster of support tickets all related to the same API endpoint. The founder makes a mental note.

The marketing-writer subagent activates next, because the founder also wants the weekly review to capture what shipped on the marketing side. The subagent reads the blog posts published that week, the social media posts, the newsletter that went out, and produces a short summary of marketing output with attached engagement metrics where available.

The Weekly Review

The product-strategist subagent then handles the planning section. It reads the founder's product roadmap document, reads the recent decision logs from the product folder, and produces a candidate list of three to five things to consider tackling next week, each with a one-sentence rationale.

The three sections come together as a single document. The founder reads it, edits a few wording choices in the marketing section, expands the rationale for one of the planning items, and saves it to her ops folder. The whole review takes twenty minutes, where it would have taken closer to ninety without the environment, and the resulting review is more disciplined and more honest than the ones she used to write by hand, because the financial-analyst subagent does not let her gloss over uncomfortable numbers.

The cluster of support tickets the financial-analyst flagged turns out to be the most consequential output of the review. She investigates the cluster, finds that the affected endpoint has a subtle race condition under high concurrent load, and ships a fix that following Tuesday. Without the financial-analyst's "things I would watch" framing, the cluster would have been buried in the support volume and probably not investigated for several more weeks, by which point a few of the affected customers might well have churned. The investment in the financial-analyst subagent has, in effect, paid for the entire environment in this single session, by surfacing a pattern that human attention alone might have missed.

The example shows the multi-domain nature of the founder's environment functioning at full strength. Three subagents from three

different domains coordinated on a single integrated task. Each contributed its specialty. The composition produced an output that exceeded what any single subagent could have produced. The founder's role in the work shifted from drafting all three sections by hand to reviewing and editing three drafts, which is a higher-leverage form of attention. This is what the architecture is supposed to produce: not the elimination of the founder from the loop, but the elevation of her contribution from drafting to direction.

A fourth reflection concerns the limits of what this kind of environment can do. The architecture does not make the founder smarter. It does not produce ideas she would not have produced. It does not make strategic decisions for her. The architecture absorbs the conventions, the formats, and the role-shaping that her work requires, leaving her attention free for the work that actually requires her judgment. The strategic decisions, the brand voice that gives the marketing its character, the product instincts that decide which feature to build next, all of those remain hers. The environment is a tool, and like all tools, it amplifies what is already there rather than replacing it. A founder with poor strategic instincts will not become a great strategist by installing a Claude Code environment. A founder with strong instincts will find that her instincts now reach further, because the surrounding labor of executing on them is partly absorbed by the environment.

A fifth reflection concerns the readiness of this kind of environment for evolution. The founder's environment will not stay the same. As her business grows, the work will shift. She might hire her first employee, at which point some of the personal user-level configuration will need to be migrated to project-level so that the new hire inherits the relevant pieces. She might launch a second product line, at which point new domain folders will appear in her skills directory. She might shift from monthly to weekly investor updates, at which point the financial-analyst subagent will need its description and its system prompt updated. The architecture accommodates all of these evolutions naturally, because each component is independently editable and the relationships between components are loose enough to evolve gradually. The environment that the founder has at the end of her first year will not look identical to the environment at the start of the year, and the difference will be a record of how her work has evolved. This is the deeper meaning of treating the .claude directory as a living artifact: the

directory becomes a mirror of the work, and the work becomes more legible by being mirrored.

A sixth and final reflection concerns the broader implication of multi-domain Claude Code environments for the way solo founders and small teams operate in the present moment. The combination of cheap intelligence and well-engineered environments has shifted the boundary of what a single person can sustain operationally. Companies that would have required a team of five a decade ago can now be run by a single founder with a thoughtful Claude Code environment, because the environment absorbs the operational labor that used to require additional headcount. The leverage is real and substantial. Founders who recognize this shift and invest in their tooling accordingly are operating with a structural advantage over founders who treat their tools as commodities. The architecture this book has been teaching is, in this sense, a piece of strategic infrastructure for solo and small-team operators in the current era, and the case study above is meant to demonstrate not merely a productivity gain but a different kind of company that becomes possible to run when the architecture is in place.

Part Five ends here. The architecture has been laid out, and two case studies have shown it producing complete environments for very different shapes of work. The reader has, by this point in the book, both the conceptual framework and the practical patterns to build her own environment for her own work. The remaining chapter of the book takes the focus from building the environment to evolving it over time, sharing it with others, and fitting it into the broader Claude Code ecosystem that is forming around the framework. That closing work is the subject of the chapter ahead.

PART SIX

Shipping, Sharing, and Evolving

The environment built once is the foundation. The environment maintained over time is the asset.

CHAPTER TWENTY-TWO

Distribution, Team Adoption, and the Claude Code Marketplace

“The garden you tend for years is not the garden you planted. It is the garden you allowed to become itself, with help.”

a gardener, anonymous

This is the closing chapter of the book. The work of building has been the focus of every part up to now. The work of evolving is the focus of this one. A Claude Code environment that has been built and never touched again will degrade slowly, as the developer's work shifts and the environment fails to shift with it. A Claude Code environment that has been treated as a living artifact, gardened regularly, shared with others, and integrated into the broader ecosystem of available tooling, will continue producing benefits for years, and those benefits will compound in ways that no static environment can match. The chapter ahead is about that gardening.

Begin with the first concern, which is version control. The `.claude` directory is part of a project's contract for how work happens in the project. It belongs in source control, alongside the source code. Every project the developer works in should have its `.claude` directory committed to its repository, with changes reviewed in pull requests like any other code change. This is not an aesthetic recommendation. It is the mechanism by which configuration travels with the code, by which teammates inherit the environment automatically when they clone, and by which the configuration evolves in a reviewed, accountable, reversible way. Projects that treat their `.claude` directories as ephemeral local configuration fail to capture most of the value the architecture is capable of providing. Projects that treat them as first-class engineering artifacts capture the full value, and the difference compounds over months and years.

The user-level configuration, which lives in the home directory and applies across every project the developer opens, deserves the same treatment, in a slightly different form. The user-level `.claude` directory should also be version-controlled, but typically in a personal repository rather than in any specific project's repository. The personal repository becomes a record of how the developer's tooling has evolved over time. It can be cloned to a new machine when the developer moves laptops. It can be shared with peers as a starting point when they ask how she has set up her environment. It can be referenced years later as a working portrait of how the developer's craft has grown. The version control of the user-level configuration is a small habit, but the developers who adopt it consistently end up with substantially more durable tooling than those who treat their home directories as ephemeral.

A natural extension of version control is the practice of writing meaningful commit messages for .claude directory changes. A commit that says "added prettier hook" tells the developer in six months what she changed and why. A commit that says "config update" tells her nothing. The same discipline that produces good commit messages for source code applies to commit messages for configuration, and the same skill the developer has installed for source-code commits, the commit-message skill from Chapter Five, applies cleanly to configuration commits as well. The configuration is code, in every meaningful sense, and treating it as code produces the consistency that long-term maintenance requires.

Move to the second concern, which is sharing. A skill, subagent, or hook that has proven valuable in one developer's environment is often valuable in many others. The mechanism for sharing is straightforward. The component is a file or a small folder. The file or folder can be copied into another developer's .claude directory, or distributed through a public repository that other developers clone, or installed through one of the package managers that have begun to emerge around the Claude Code ecosystem. The friction of sharing is low, and the benefit of receiving someone else's well-designed components is substantial.

The Marketplace

The Claude Skills Marketplace is the most prominent public channel for sharing. Anthropic has begun curating skills produced both by its own engineering team and by third-party developers and organizations, with skills available for installation through a small command-line interface. The marketplace is in its early days as of this writing, and the available catalog is modest, but the trajectory is clear. Skills that solve common problems will continue to accumulate in the marketplace, and the developer who watches the marketplace will find ready-made solutions for problems she might otherwise have built skills for from scratch. The right relationship with the marketplace is to use it as a starting point rather than as a complete library; off-the-shelf skills rarely fit a specific developer's preferences perfectly, but they often save substantial design time when used as a base that the developer adapts.

Subagents and hooks have not yet developed marketplace structures of equivalent maturity, but the underlying mechanism is the same. Public

repositories of subagent definitions and hook scripts are appearing in greater numbers, often shared on GitHub under permissive licenses, and the developer who participates in this informal exchange both contributes to and benefits from the collective body of work. Sharing publicly is also a way of making one's own work better, because the act of writing a component that strangers will read forces a level of clarity that internal-only components do not require.

A third concern is monetization. Some developers will, naturally, want to sell their work. The economics of this are still settling in 2026, but several patterns have emerged that look likely to persist. Skill packs targeted at specific niches, sold as one-time purchases or as subscriptions, are appearing on platforms like Gumroad and Lemon Squeezy. Subagent libraries customized for specific industries are being licensed by consultancies to their clients. Configuration packages that combine skills, subagents, and hooks into complete environments for specific roles are being sold as productivity products. The market is small but growing, and the developers who build genuinely high-quality components have a real path to monetization that did not exist a few years ago.

The right relationship with monetization, for most developers, is to think of it as an option rather than as an objective. Building components for one's own work is the primary purpose. If, over time, certain components turn out to have wider value, the path to selling them is open. Building components for sale from the start, before any internal use has validated their quality, is harder, and the components produced this way often turn out to be less useful than components that emerged from real work. The advice that emerges is to build first, sell later, and to let the components prove themselves in actual use before any commercial considerations enter the picture.

Move to the fourth concern, which is team standardization at scale. The case study in Chapter Twenty showed a small team adopting a shared configuration. The same pattern scales to larger organizations, but with additional considerations. A team of fifty engineers has more diversity of working style than a team of five, and a configuration that works for the entire team needs to be either narrow enough to fit everyone or layered carefully so that the shared layer represents the team's genuine common ground while individual variation lives in personal user-level configuration.

Team Standardization at Scale

The pattern that has worked best in practice is to keep the shared project-level configuration tight and conservative, including only the components that genuinely apply to all engineers in the project, and to encourage individual engineers to maintain their own personal user-level configuration on top of it. The shared layer captures conventions, security guarantees, and project-specific knowledge. The personal layer captures preferences, role-shaping, and personal-style choices. The combination produces a personalized environment for each engineer that nonetheless honors the team's common standards. The split also makes the shared layer easier to evolve, because changes to the project-level configuration affect only what is genuinely shared, not the personal preferences that should not be overridden.

A fifth concern is the long-term evolution of the developer's environment. After a few years of working with the architecture, the developer's `.claude` directory becomes a substantial artifact. It might contain forty skills, fifteen subagents, and a dozen hooks across her various projects and her user-level configuration. The maintenance posture for an environment of this size is different from the posture for a small new environment. The questions shift from what to add to what to keep, what to remove, and what to refactor. A quarterly review becomes a useful discipline. The developer sets aside an hour every three months to walk through her configuration, identify components that have not been used recently, refine components that have been used heavily, and remove components that have outlived their purpose. The hour pays for itself many times over in the months that follow, because it keeps the environment from accumulating dead weight that would otherwise quietly degrade the experience.

A sixth concern is the relationship between the developer's Claude Code environment and the broader ecosystem of AI-assisted development tools that is forming. Skills, in particular, are increasingly being adopted by other tools. The `SKILL.md` format is being honored by Cursor, by various open-source IDE extensions, and by several command-line tools that are not Claude Code. A skill written for Claude Code today often works without modification in three or four other tools. This cross-compatibility is not accidental; it reflects the convergence of the industry around a small set of

standards, with `SKILL.md` emerging as one of those standards. The implication for the developer is that her investment in skill design pays dividends not just inside Claude Code but across whatever set of AI-assisted tools she ends up using over the years. The skills travel. The environment, in this sense, is more durable than the specific tool the environment was built for.

A seventh concern is what the developer can expect from the ecosystem in the months and years ahead. The trajectory of Claude Code, and of similar tools, is toward more sophisticated automation, more capable subagents, and more richly composable hook systems. New lifecycle events will appear. New tool restrictions will become possible. New patterns of subagent communication will emerge. The developer who has internalized the architecture in this book will be well-positioned to absorb each of these changes as they arrive, because each will fit into the framework rather than disrupting it. The framework itself is not tied to any specific feature set. It is tied to the durable insight that capability, specialization, and enforcement are three different concerns that benefit from being handled by three different mechanisms. That insight will remain valid as long as AI-assisted development continues to be useful, which is to say, indefinitely.

A reflection on what the book has tried to do. The book has tried to teach a way of relating to a tool that, for most of its users, is treated as a chat interface. The way of relating that the book has taught is more like the way an engineer relates to a well-configured editor or a well-tuned shell: as a substrate that can be bent to fit the engineer's work, that rewards patient configuration with compounding returns, and that becomes, over time, an extension of the engineer's craft rather than merely a tool she uses. The shift in posture is what matters most. The specific skills, subagents, and hooks that the book has shipped are starting points. The shift in posture, once made, persists across every future tool the developer encounters, because it is a way of thinking about tooling rather than a fact about any specific tool.

The Community of Practice

A second reflection concerns the community of practice that is forming around the architecture this book has been teaching. The community is real, growing, and largely informal. Developers are sharing their configurations

on GitHub, writing blog posts about their patterns, recording short videos of their environments in action, and trading components in private Slacks and Discords. The community is not centralized; there is no single forum that captures all of it, and no single voice that speaks for it. The decentralization is a strength, because it means that good ideas surface from many different directions and the practice evolves through real use rather than through pronouncement. The reader who wants to participate has many entry points. She can publish her own configuration. She can contribute components to the marketplace. She can write about what has worked for her. She can simply adopt other developers' ideas and incorporate them into her own environment. The community rewards participation, even modest participation, with both reciprocal benefit and a sense of belonging to something that is moving in real time.

A third reflection concerns contribution upstream, to Anthropic and to the broader tooling ecosystem. The patterns this book has been teaching are not fixed. They are evolving, and the people who use them daily are the most likely to discover the patterns that should evolve next. A developer who notices a recurring failure mode in subagent design, who finds a hook event that does not exist but should, who realizes that some category of skill would benefit from a feature the format does not yet support, has the standing to surface that observation. The right channels include filing issues on the relevant tools' GitHub repositories, writing publicly about the observation, and engaging with the developer-relations teams who watch the community. Many of the features that have shipped in Claude Code over the past year originated in observations from real users. The reader who has worked through this book and developed a serious practice has the standing to contribute to the next round of those observations, and the contribution is not abstract: it shapes the tools she will be using a year from now.

A fourth reflection concerns the longevity of the architecture itself. The framework in this book has been described in terms of one specific tool, Claude Code, and one specific moment in time, the spring of 2026. The tool will change. The moment will pass. The framework, however, is durable, because it is rooted in a deeper truth about how AI-assisted development naturally divides into knowledge, specialization, and enforcement. Any future tool that handles AI-assisted work will, by the structure of the work itself, present similar concerns, and the framework will translate into

whatever vocabulary that future tool adopts. A reader who has internalized the architecture is, in this sense, learning a posture that will outlast the specific tool the book was written for. The skills she develops in shaping her Claude Code environment are the same skills she will use to shape whatever tool replaces it, and whatever tool replaces that, indefinitely. The investment is not in Claude Code specifically. The investment is in the practice of engineering one's relationship with one's primary tooling, and the practice transfers.

The reader who has worked through the book has, at this point, the framework, the patterns, and the practical libraries to build a substantial Claude Code environment for her own work. The next steps are her own. She will install the components from the chapters that fit her work most directly. She will adapt the others to her own preferences. She will build her own components for the frictions the book did not anticipate. She will commit her configuration, share what generalizes, and refine what fits only her. She will, over the months ahead, end up with an environment that no off-the-shelf product could produce, because it has been shaped by her actual work and her actual preferences, and the shaping has been deliberate.

A short word about pacing the build. New readers sometimes feel an urgency to install everything in the book during the first weekend. The urgency is understandable but probably misplaced. The components that have been demonstrated across the chapters were chosen because they cover common needs, but no individual reader needs all of them on day one. The pacing that has worked best in practice is to install a small starter set in the first week, perhaps three skills, two subagents, and two hooks, and then let real use surface which additional components are worth building. A component installed in response to a felt friction is almost always more useful than a component installed because the book described it. The book is a catalog. The reader's real work is the curriculum. Trusting the curriculum, and letting the catalog inform it rather than dictate it, produces an environment that fits the reader's actual work tightly. Trusting the catalog, and forcing the curriculum to match it, produces an environment that fits no actual work in particular.

The Rhythm of Building

A related observation about the rhythm of building is worth recording. The first month of building is the period in which the most components get added, because the developer is encountering frictions she has noticed for years and finally has the tools to address. The second and third months see fewer additions but more refinement, as the components installed in the first month settle into actual use and reveal their rough edges. By the sixth month, the developer is mostly maintaining and occasionally extending, and the environment has reached a kind of steady state in which it fits her work tightly. This rhythm is normal and healthy. A developer who is still adding components at the same rate after six months has either an unusually broad and complex working life or, more often, has fallen into the over-engineering trap that this book has warned about repeatedly. The right signal that the build is reaching maturity is that new components become rare, refinements become small, and the environment recedes into the background of the developer's daily work, doing its job without drawing attention.

A final word, before this book ends. The developers who invest in their tooling have always had an advantage over the developers who do not. The advantage was real with text editors. It was real with shells. It was real with build systems. It is real now with AI-assisted development environments, and the advantage is, if anything, larger, because the leverage of well-engineered configuration has grown as the tools themselves have grown more capable. The reader who finishes this book and begins her own building has, in the language of investments, made an asset allocation. The asset is her tooling. The expected return is compound, and it begins paying out from the first session in which the configuration affects her work. There is no urgency to build everything at once. There is only the discipline to build deliberately, to use what is built, and to let the environment grow alongside the work it is meant to serve. The reader who follows that discipline will find, in a year or two, that her environment has become something she would not trade for any other configuration in the world, because it has become hers.

The book ends here. The work begins.

Appendices

Reference, Troubleshooting, and Beyond the Three Pillars

The book ended one chapter ago. What follows is what you reach for.

APPENDICES

Reference, Troubleshooting, and Beyond the Three Pillars

The book ended one chapter ago. What follows is what you reach for.

Troubleshooting and Debugging

“The first half of solving a problem is being able to look at it.”

a debugger of long standing, anonymous

Real Claude Code environments break in predictable ways. A skill does not fire when it should. A subagent produces generic output that ignores the project's conventions. A hook silently fails to run. A pipeline behaves correctly in one developer's environment and inexplicably differently in another. None of these failures is mysterious once the developer has a debugging procedure in working memory; each of them traces back to one of a small number of root causes, and each of those causes has a clean diagnostic path. This appendix gives the reader the diagnostic paths.

The procedures below assume the reader has worked through Parts Two, Three, and Four of the book and is familiar with the underlying mechanics of skills, subagents, and hooks. None of the procedures requires advanced shell knowledge, but each of them does require the reader to be willing to spend a few minutes verifying assumptions one at a time rather than trying to fix everything at once.

Skill problems

The most common skill failure mode is that the skill does not fire when the developer expects it to. The diagnostic procedure has four checkpoints, and the developer should walk them in order.

First, verify the skill is installed in a location Claude Code looks at. The two valid locations are the user-level skills directory (`~/.claude/skills/`) and the project-level skills directory (`.claude/skills/`) at the root of the current project. A skill installed anywhere else will not be discovered.

Second, verify the skill folder contains a file literally named `SKILL.md`, with the exact capitalization. A file named `skill.md` or `Skill.md` will not be discovered on case-sensitive filesystems.

Third, verify the YAML frontmatter is well-formed. The frontmatter is the block at the very top of the file fenced by three hyphens on a line of

their own. If the fence is missing, indented, or includes characters that the YAML parser rejects, the skill will be silently skipped at session start.

Fourth, verify the description field contains language that matches the kind of request the developer is making. This is the failure mode in roughly nine cases out of ten. The fix is to expand the description with the phrasings the developer actually uses, following the four-part framework from Chapter Six (canonical statement, trigger language, boundary, input expectation).

A useful diagnostic technique when the skill is installed and the description appears reasonable but the skill still does not fire is to ask Claude directly. Type into the session: why did the [skill-name] skill not fire on my last request. Claude will often produce a useful answer, naming the gap between the request and the description. That answer is then a precise instruction for what to add to the description. After making the change, restart the session for the change to take effect, and try again. Within two or three iterations, almost every triggering problem can be solved by description refinement.

A second skill failure mode is over-firing. The skill fires on requests that should not have triggered it. The cause is almost always a description that is too broad, lacking boundary language. The fix is to add explicit exclusions to the description, naming the kinds of requests that look superficially similar to the skill's territory but should not activate it.

A third failure mode is correct firing but incorrect output. The skill activates as expected, but what it produces does not match what the body specifies. This is almost always a sign that the body is ambiguous in a way the developer did not notice when writing it. The fix is to read the body as if reading it for the first time, find the place where instruction and output diverge, and tighten the language. Side-by-side good and bad examples are extraordinarily effective for fixing this.

Subagent problems

Subagent problems share a structure with skill problems, with one important addition: subagents have a system prompt that does most of the shaping work, and most subagent failures trace back to that system prompt being too vague.

If the subagent does not get dispatched at all, the cause is almost always a description that does not match the request. The diagnostic is identical to the skill triggering procedure. Type the failed request again, then ask Claude why the subagent was not dispatched, and refine the description based on the answer.

If the subagent gets dispatched but produces generic output, the cause is almost always a system prompt that gives the subagent too much latitude. The fix is to add specificity: name the method as a numbered procedure, name the output format down to the section headings, and add a what-you-do-not-do section that explicitly forbids the failure modes the developer has already encountered.

If the subagent gets dispatched and produces output in the right shape but with the wrong substance, the cause is often a missing skill. The subagent has the role definition but not the project-specific knowledge. The fix is either to add a skill that encodes the project knowledge (which the subagent will then load when relevant) or, less ideally, to put the project knowledge directly into the subagent's system prompt.

If the subagent does not respect its tool restrictions, the cause is usually a typo in the tools field of the frontmatter. The values are case-sensitive and must match the tool names exactly: Read, Write, Edit, `MultiEdit`, Bash, Grep, Glob. A typo silently grants the full tool set.

A useful introspection technique is the `/agents` slash command. Run it in a session; it lists every subagent available, shows the description and tool restrictions, and confirms the file is being read correctly. If the developer expects a subagent to exist and `/agents` does not show it, the problem is at the file or YAML level rather than at the description level.

Hook problems

Hook problems are the most subtle of the three categories, because hooks fail silently more often than they fail loudly. A misconfigured hook does not produce a visible error in most cases; it simply does not run. The four-step diagnostic procedure for hooks is worth committing to memory.

First, verify the hook is loaded. Run the `/hooks` command inside a Claude Code session. The interactive editor lists every event and the hooks bound to each one. If the new hook does not appear, the configuration file

has a problem, usually a JSON syntax error or a misnamed event. Many hook problems disappear at this step.

Second, verify the hook is firing. Add a small log statement to the hook itself, writing a timestamp to a file every time the hook fires. Make an action that should trigger the hook. Check the log file. If the log file has no new entry, the matcher is wrong; the hook is bound to an event or tool name that does not match what is happening. Adjust the matcher and try again.

Third, verify the hook is succeeding. Read the hook's standard output and standard error in the log. If the hook is exiting with a non-zero code due to a missing tool or an unexpected JSON shape, fix the cause. A common case is that a tool the hook depends on (like `jq` or `npx`) is not installed on the developer's system, or has been moved out of the path.

Fourth, verify the effect of the hook. The hook may be firing, succeeding, and still not producing the intended outcome because of a logic error in the script or because the hook's output is not being interpreted as expected. For hooks that emit JSON to control the lifecycle, validate the JSON shape against the documentation before assuming the hook is broken.

The `/doctor` slash command is worth knowing about as a complementary diagnostic. It reports on the current installation and surfaces common configuration problems including malformed settings files, missing dependencies that hooks rely on, and version mismatches. Running `/doctor` at the start of any debugging session gives the developer a quick read on whether the problem is local to her configuration or environmental.

A specific subtlety worth naming: hook configuration changes do not affect a running session. After editing the hooks file or any `settings.json`, the developer must restart Claude Code or open a new session for the change to take effect. Many otherwise-correct hook configurations appear broken because the developer is testing them in the same session in which she made the change.

Pipeline problems

When several hooks compose into a pipeline, the diagnostic complexity rises but the procedure does not change. Walk each hook in the pipeline through the four-step procedure individually. The hook that fails one of the four steps is the failing component. Pipelines almost never fail because of

pipeline-level issues; they fail because one hook in the pipeline fails, and the failure cascades.

A second pipeline-specific concern is order. Hooks bound to the same event run in the order they appear in the configuration. If the second hook in a pair depends on the first hook having modified the file system, and the first hook is misordered or misconfigured, the second hook will run on stale state. The fix is to verify the order matches the dependency, and to add a small assertion at the start of any hook that depends on another hook's effect.

Hook Events Quick Reference

“The reference is the part you reread. The argument is the part you read once.”

a textbook author, anonymous

This appendix is a compact reference to every hook event Claude Code currently supports, what each event fires for, what input it receives, and the most common pattern of use. The reference is intended to be looked up rather than read straight through. Skim it once to learn what is here, and then return to it when designing a new hook to verify which event fits.

Event	Behavior, input, and use
PreToolUse	Fires: Fires immediately before Claude invokes a tool, with the tool name and input available. Input: JSON object with tool name, tool input, and session metadata. Use for: Gates that block, modify, or defer tool calls. Security policy enforcement, dangerous-command blocking, file-path protection. Exit 2: Blocks the tool call. The standard error output is surfaced to Claude as feedback. Four richer outcomes (allow, deny, ask, defer) are available via JSON output.
PostToolUse	Fires: Fires immediately after a tool call has completed successfully. Input: JSON object with the tool name, the input, and the result. Use for: Guards that enforce consistency: formatters, linters, post-edit validators, audit logging of every action Claude takes. Exit 2: Cannot block, since the action has already happened. Surfaces the standard error output to Claude as feedback, which can prompt corrective work in the next turn.
UserPromptSubmit	Fires: Fires when the user submits a prompt, before Claude processes it. Input: JSON object with the prompt text and session context.

	Use for: Prompt-level filtering, optional injection of additional context, and policy enforcement on what kinds of requests are allowed. Exit 2: Blocks the prompt entirely. The user sees the rejection message.
Stop	Fires: Fires when Claude finishes generating a response. Input: JSON object with the session state at the moment Claude is ready to stop. Use for: End-of-turn validators, session summary writers, end-of-turn audit logs. Exit 2: Refuses the stop. Claude is required to continue working until a stop succeeds.
SubagentStop	Fires: Fires when a subagent completes its work, just before its result returns to the main session. Input: JSON object with the subagent name and its final output. Use for: Validating subagent output against expected shape, logging subagent activity. Exit 2: Refuses the subagent's stop. Use sparingly.
SessionStart	Fires: Fires when a new session begins or a saved session is restored. Input: JSON object with session metadata. Use for: Loading ambient project state into the session's starting context. Exit 2: Output written to standard output is injected into Claude's starting context.
SessionEnd	Fires: Fires when a session terminates. Input: JSON object with session metadata and final state. Use for: Session-level audit logging, daily summary writing, cleanup tasks. Exit 2: Has no blocking effect at this stage; the session is ending regardless.
Notification	Fires: Fires when Claude needs to alert the user about something. Input: JSON object with the notification payload. Use for: Producing desktop notifications, sending alerts to chat platforms. Exit 2: Does not affect the lifecycle. Notification hooks are the canonical notifier-pattern hooks.
PreCompact	Fires: Fires just before Claude compacts its context to save space.

Input: JSON object with the current context state. Use for: Capturing important context before it is compacted away, writing checkpoint summaries. Exit 2: Output to standard output is preserved into the compacted context.

A practical guideline for choosing among these events: the event whose name most directly matches the moment when the work needs to happen is almost always the right one. `PreToolUse` for prevention. `PostToolUse` for correction. `Stop` for end-of-turn validation. `SessionStart` for ambient state injection. With practice, the right event becomes obvious at the moment the developer recognizes which kind of work she is trying to enforce.

Beyond the Three Pillars: Plugins and Agent Teams

“The framework is the foundation. What sits on top of the foundation is the work of the next decade.”

a software architect, anonymous

The three pillars covered in this book, skills, subagents, and hooks, are the durable core of Claude Code customization, and they are sufficient on their own to build the engineered environments that the case studies in Part Five demonstrated. They are not, however, the entirety of what Claude Code now supports. Two additional capabilities have emerged in the more recent releases that are worth knowing about, both because they extend the architecture in useful directions and because the developer who is fluent in the three pillars will find the additions easy to absorb. This appendix introduces them at a level sufficient to recognize when they apply and to know where to look for deeper coverage.

The first additional capability is the plugin system. The second is agent teams.

Plugins

A plugin, in the Claude Code sense, is a packaged collection of skills, subagents, hooks, and supporting assets that can be installed as a unit. The plugin system formalizes a pattern that this book has already taught informally: the developer who builds a coherent set of components for a specific kind of work is, in effect, building a plugin, even if she has not packaged the components for distribution. The plugin system gives that pattern a formal home.

A plugin is a folder that contains a manifest file describing what the plugin provides, the components themselves arranged in their conventional subfolders, and any documentation the author wants to ship with it. The plugin can be installed from a local path, from a Git repository, or from a registry, and once installed it appears alongside the developer's hand-built

configuration. The plugin's components are activated through the same triggering and dispatch mechanisms covered in Parts Two through Four: skills fire when their descriptions match, subagents are dispatched when their roles fit, hooks bind to lifecycle events. From the model's perspective, a plugin is invisible; the components inside the plugin behave exactly as components installed directly would behave.

The strategic value of plugins is twofold. The first value is distribution. A developer who has built a thoughtful environment for a specific niche, such as technical writing, financial analysis, or React development, can package it as a plugin and share it with peers or sell it. The second value is consumption. A developer who recognizes that her work overlaps significantly with an existing plugin can install the plugin and inherit its components, saving herself the time of building the same components from scratch. Both ends of the exchange are useful, and the plugin ecosystem is in the early phase of populating with real, useful packages.

The reader who finishes this book and wants to participate in the plugin ecosystem has two natural entry points. The first is to package one of her own most successful sets of components as a plugin and publish it. The second is to browse what is already available, install one or two plugins that fit her work, and observe how the plugin author shaped the components to fit a niche. Either approach develops the fluency that makes the plugin system a real extension of the developer's environment rather than a separate concern.

Agent Teams

The second extension is agent teams. The orchestrator pattern from Chapter Twelve introduced the idea of dispatching several subagents in parallel and synthesizing their outputs. Agent teams formalize that pattern. An agent team is a named, configured group of subagents that work together on a single class of task, with explicit coordination logic that specifies how the team communicates and how the final output is assembled.

A team differs from an ad-hoc parallel dispatch in two respects. First, the team is a named, persistent configuration; the developer defines it once and then dispatches the entire team with a single invocation, rather than orchestrating individual subagents on every request. Second, the team can

include explicit communication patterns between its members. A simple team is the parallel-fan-out pattern from Chapter Twelve, where each subagent contributes an independent perspective and a synthesizer combines them. A more elaborate team can have sequential dependencies, with one subagent's output becoming the input to the next, or even feedback loops where a reviewer subagent flags issues and a writer subagent revises in response.

The teams capability is most valuable for workflows that the developer runs often enough to deserve their own configuration. A weekly review like the one in the solo-founder case study from Chapter Twenty-One is a natural candidate; rather than dispatching the financial-analyst, the marketing-writer, and the product-strategist subagents individually each Friday, the developer can define a weekly-review team that runs all three with a single command and produces the unified review automatically. The development cost of the team configuration is small, and the savings on every future invocation accumulate.

A reasonable rule of thumb for when to reach for an agent team rather than ad-hoc orchestration: if the same combination of subagents has been dispatched together more than three times in a month, the combination has earned its place as a named team. The team configuration captures the pattern explicitly and removes the small overhead of remembering which subagents to dispatch on every future invocation.

A final observation is worth making about plugins and agent teams in relation to the rest of the book. Both capabilities sit on top of the three-pillar architecture rather than replacing it. A plugin is a packaged set of skills, subagents, and hooks. An agent team is a coordinated invocation of subagents. The underlying components are the same components covered in Parts Two through Four, and the design disciplines for those components are unchanged. The reader who has internalized the three pillars will find both extensions natural to learn. The reader who has not will find both extensions confusing, because the extensions assume the underlying pillars are already understood. This is the right ordering: pillars first, extensions second, and the extensions become useful precisely because the pillars are solid.

Both capabilities are evolving, and the official documentation is the right primary source for current details. The treatment in this appendix is meant to put the capabilities on the reader's mental map and to indicate when reaching for them makes sense. The deeper coverage will mature as the ecosystem matures, and the reader who has finished this book has the architectural foundation to absorb that coverage as it appears.

Key Terms Index

“A book without an index is a city without street signs.”

a librarian, anonymous

The index below is a curated list of the technical terms and architectural concepts the book has used most often, with a brief gloss for each one. The index is intended to serve as a quick lookup when the reader encounters a term in a later chapter and wants to recall its meaning, or when she returns to the book months later and needs to refresh her vocabulary. The list is alphabetical and the glosses are short on purpose; for the full treatment of any term, the reader should consult the chapter where the term was introduced.

Agent loop. The think, act, observe cycle that the language model executes on every turn. Skills and subagents shape what happens inside the loop; hooks shape what happens at its boundaries. See Chapter Two and Chapter Fifteen.

Agent team. A named, persistent configuration of subagents that work together on a single class of task, with coordination logic specifying how the team communicates. See Appendix C.

allowed-tools. The optional frontmatter field in a `SKILL.md` file that restricts the set of tools Claude can use while the skill is active. Reaches its purpose by narrowing the model's capability surface to exactly what the skill needs. See Chapter Four.

Ambient versus targeted knowledge. The architectural distinction between knowledge that is true at all times in a project (which belongs in `CLAUDE.md`) and knowledge that is true only when a specific kind of work is being done (which belongs in a skill). See Chapter Two and Chapter Eight.

.claude/. The convention directory at the root of a project, or in the user's home folder, that holds Claude Code configuration. Inside it live the `skills/` and `agents/` subdirectories, the `settings.json` file that configures hooks,

and (by convention) a `hooks/` folder for the shell scripts that `settings.json` references. See Chapter Two.

CLAUDE.md. The special file Claude Code reads at session start whose contents are added to the system context. The right place for ambient project knowledge that should apply to every interaction. See Chapter Two.

Composition pattern. The architectural pattern in which skills activate inside subagents, producing capability and specialization simultaneously. The deepest leverage in the three-pillar framework. See Chapter Fourteen.

Description. The frontmatter field in any skill or subagent definition that determines whether the component fires or is dispatched on a given request. The single most consequential design decision in any component. See Chapter Six.

Deterministic outer / probabilistic inner. The architectural pattern in which the language model operates probabilistically inside the agent loop, while hooks impose deterministic constraints at the loop's boundaries. The central design principle of the book. See Chapter Three.

Gate. A hook design pattern in which a `PreToolUse` hook evaluates a tool call and either allows or blocks it. The mechanism by which wishes become rules. See Chapter Sixteen.

Guard. A hook design pattern in which a `PostToolUse` hook runs after a tool call and enforces consistency on its result. The mechanism by which conventions become automatic. See Chapter Sixteen.

Hook. A user-defined shell command that Claude Code executes automatically at a specific lifecycle event. The third pillar of the framework, providing deterministic enforcement. See Part Four.

hookSpecificOutput. The JSON output field in a hook that carries richer control instructions to Claude Code, including the four `PreToolUse` outcomes (allow, deny, ask, defer) and the modified tool input. See Chapter Fifteen.

Notifier. A hook design pattern in which a hook produces output to a system other than Claude Code itself, such as a desktop notification or an audit log entry. See Chapter Sixteen.

Orchestrator pattern. The multi-subagent pattern in which the main session decomposes a task, dispatches several subagents in parallel, and synthesizes their outputs into a unified result. The most common pattern of subagent composition. See Chapter Twelve.

Plugin. A packaged collection of skills, subagents, hooks, and supporting assets that can be installed as a unit. The formal mechanism for distributing a coherent set of components. See Appendix C.

Progressive disclosure. The loading strategy in which Claude Code reads only the descriptions of available skills at session start, then loads a skill's body only when the skill activates, and only reads bundled files when the body refers to them. The mechanism that lets large skill libraries scale without paying a context tax. See Chapter Four.

Skill. A reusable capability defined by a folder containing a `SKILL.md` file. The first pillar of the framework. See Part Two.

`SKILL.md`. The Markdown file inside a skill folder that contains YAML frontmatter declaring the skill's metadata and a body containing the instructions Claude follows when the skill is active. See Chapter Four.

Subagent. A separate Claude process with its own system prompt, restricted tool set, and isolated context window, dispatched by the main session for specific kinds of work. The second pillar of the framework. See Part Three.

Three-pillar framework. The book's organizing architecture: skills for capability, subagents for specialization, hooks for enforcement. The framework that any sophisticated Claude Code environment uses in some combination. See Chapter Three.

`@agent-name`. The explicit invocation syntax for a subagent. Typing `@agent-name` in a prompt dispatches the named subagent immediately, bypassing description-based delegation. See Chapter Ten.

The terms above cover the vocabulary the book has been building since Chapter One. Reading the index in one pass is a quick way to confirm that the central concepts have settled into working memory. Returning to it later, when one of the terms surfaces in a new context, is how the index earns its place in a reference book. The reader who has finished the book has, by this

point, a working command of every term above; the index is a backstop for the moments when one of them needs to be looked up rather than recalled.